

Desenvolvimento de Jogos 3D com Unity

2026

Iniciando

Entre no Unity Hub

Selecione Projects

Clique no botão [New]

Selecione Jogo 3D

Digite o nome do projeto
(Sistema-Solar)

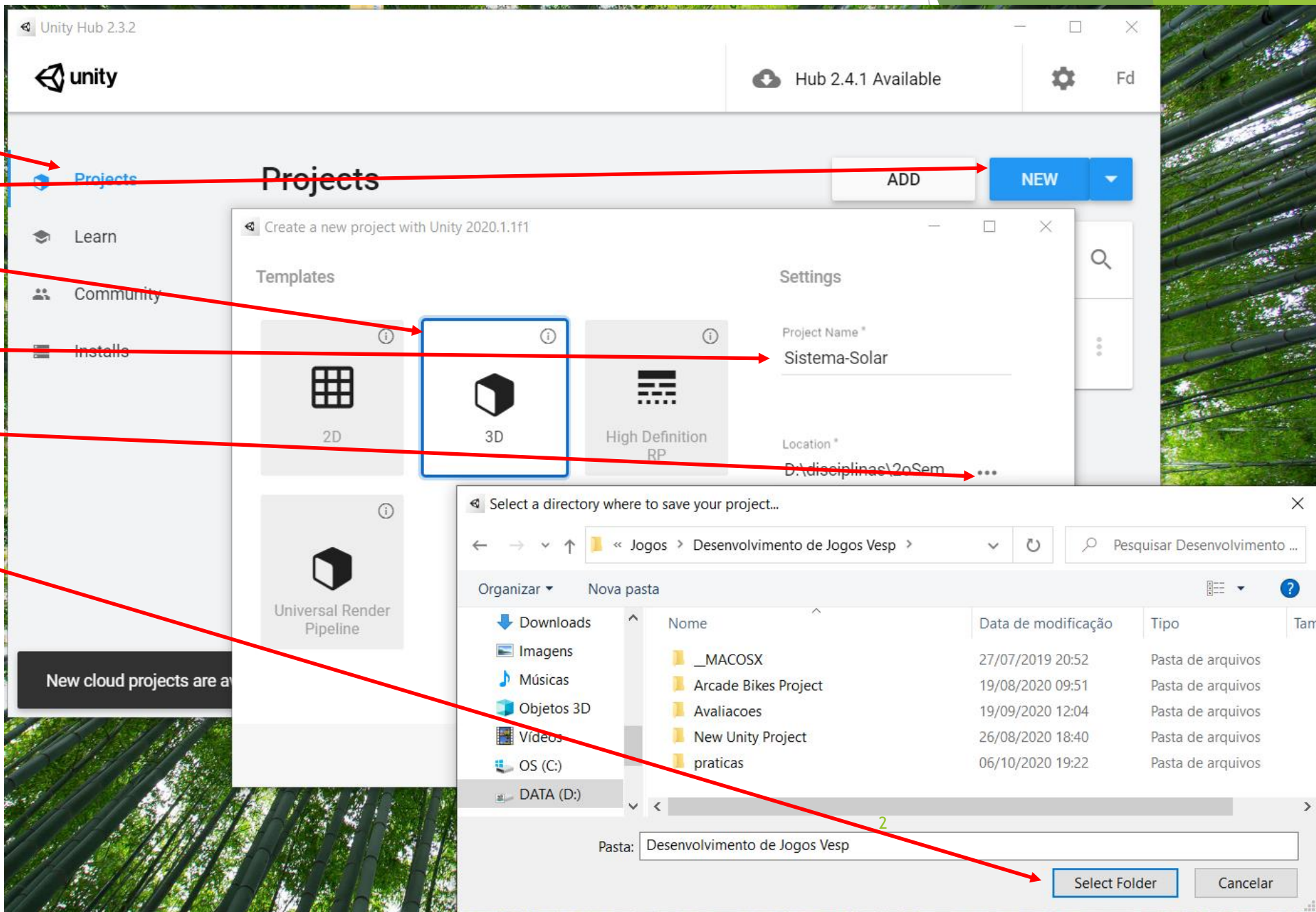
Clique nas ... para
escolher a pasta

Selecione a pasta onde
criará o jogo.

Pressione [Create] no
Hub.

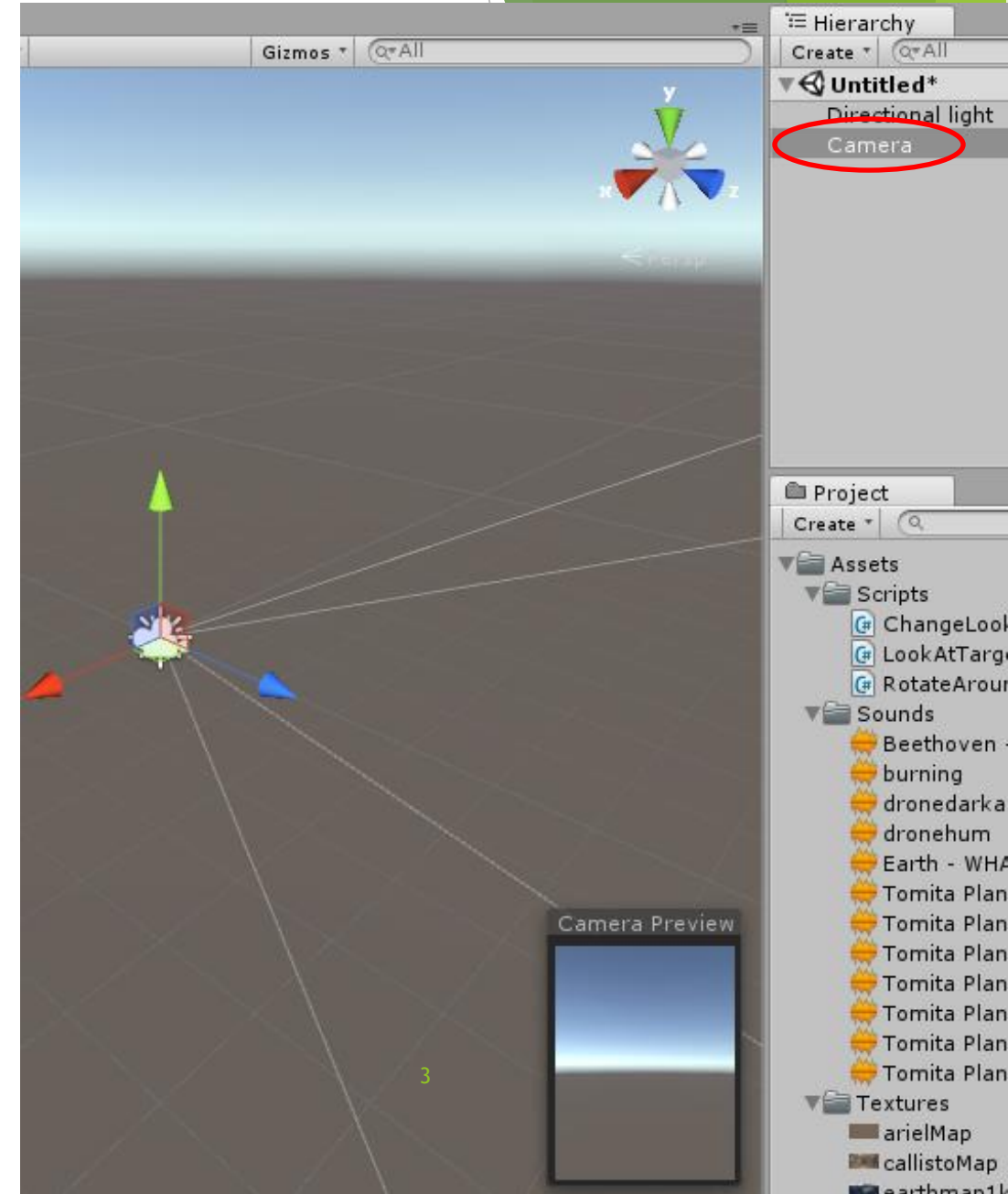
Unity criará uma pasta
com o nome do projeto
que você digitou, dentro
da pasta que selecionou.

Abra o projeto no Hub.



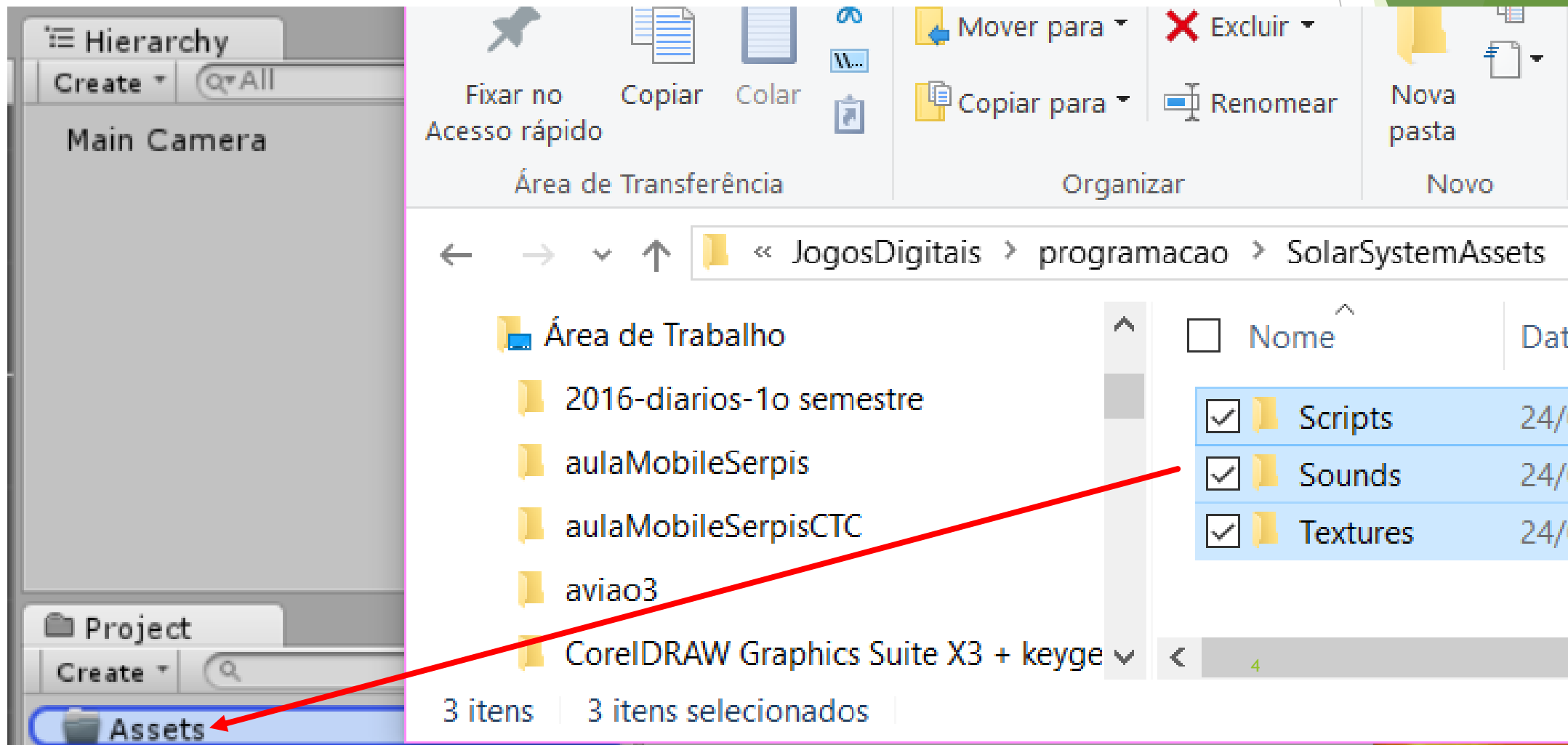
Sistema de Coordenadas

- ▶ Clique na câmera da cena (ou no objeto Main Camera da Hierarquia) e aproveite para observar o objeto no canto superior direito da cena, indicando os 3 eixos coordenados.
- ▶ Estamos trabalhando em 3D, de forma que esse objeto nos mostra para onde cada eixo aponta na direção positiva.



Importação dos assets para seu projeto

- ▶ Numa pasta em que você costuma colocar materiais desta disciplina, descompacte o arquivo de assets do Sistema solar disponibilizado pelo professor (pasta SolarSystemAssets, por exemplo).
- ▶ Arraste para a pasta Assets da janela Project as três pastas de recursos fornecidas pelo professor:

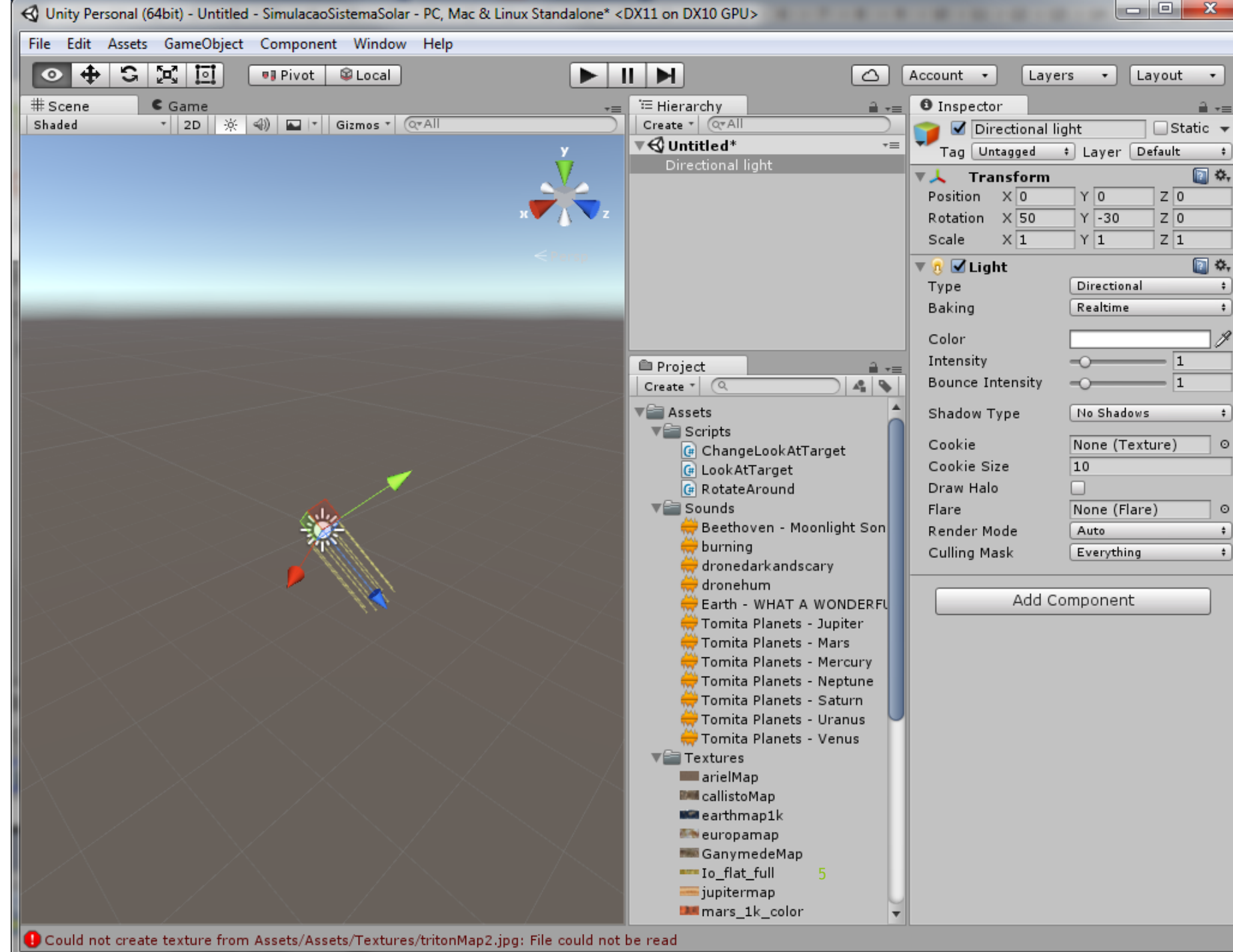


Editor do Unity

Resultado da Importação

Já tínhamos visto no estudo de 2D que essas pastas foram alocadas dentro da pasta Assets do projeto e que Unity criou arquivos de configuração (*.meta) para cada arquivo importado.

Vemos na figura ao lado os arquivos importados dentro de cada pasta.

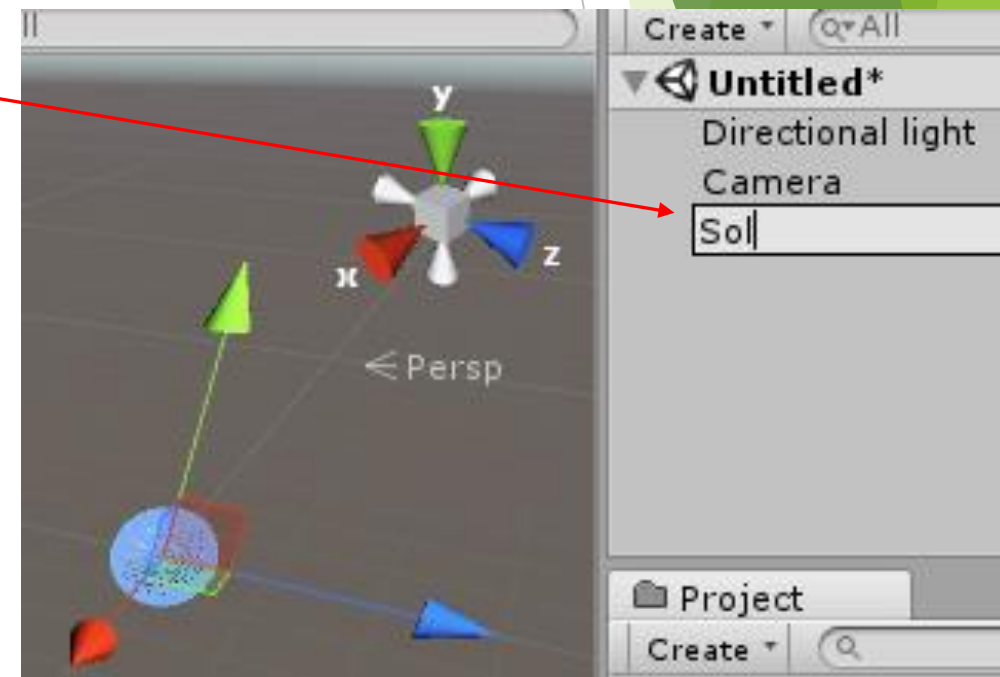
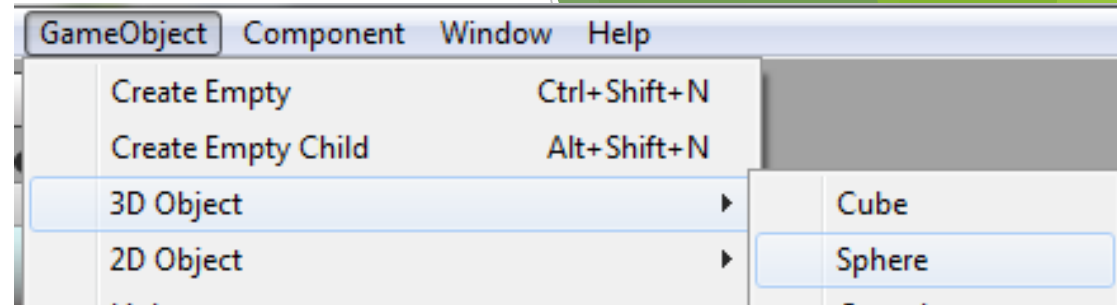


Configurando o espaço de nosso jogo

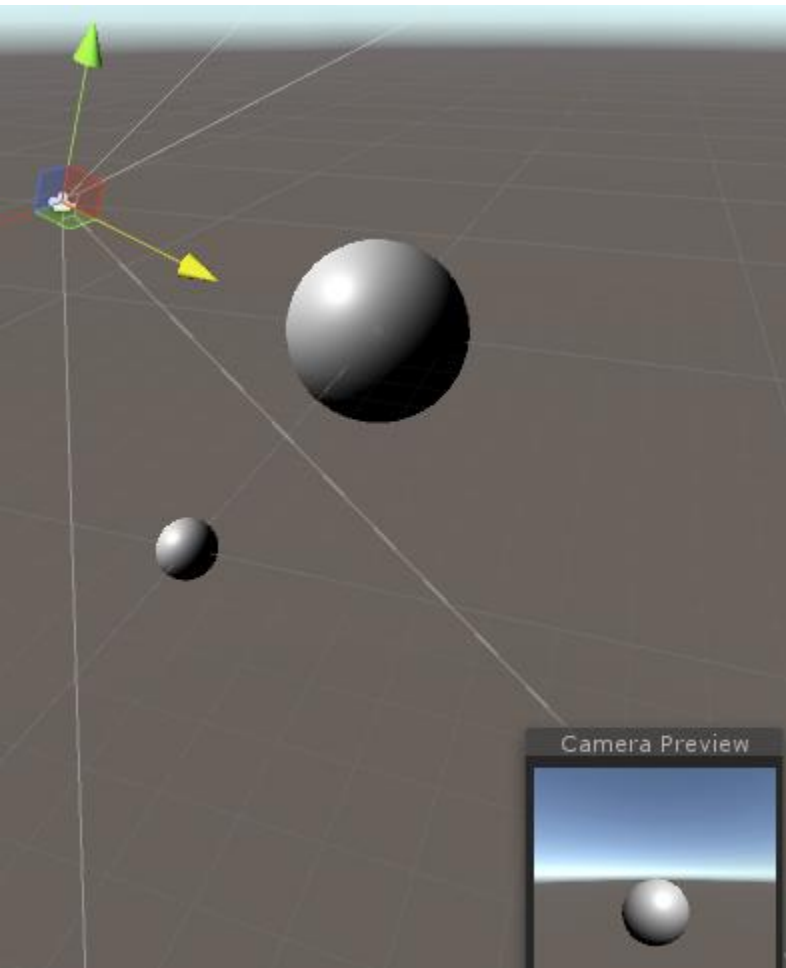
- ▶ Criar e posicionar o “Sol”
- ▶ Criar e posicionar a “Terra”
- ▶ Criar e posicionar a “Lua”
- ▶ Definir órbitas → modificar suas escalas, rotação e translação

Incluindo o Sol

- ▶ Selecione no menu: GameObject | 3D Object | Sphere.
- ▶ Aparecerá uma esfera na cena e na lista de objetos da Hierarquia. Faça Reset de suas coordenadas.
- ▶ Clique e segure na palavra Sphere da Hierarquia para mudar seu nome para Sol:
- ▶ Você pode usar a ferramenta  para mover a sua visualização dos objetos.
- ▶ Pressionando a tecla [Alt] com essa ferramenta ativada, você poderá mudar a orientação da sua visualização (ângulos de visada).
- ▶ Nesse caso, o cursor fica com o formato de um olho.
- ▶ Essas atividades não mudam os transforms dos objetos, mudam apenas o nosso ponto de visualização da cena.
- ▶ Pressione a tecla F para **focalizar** a cena no objeto selecionado na Hierarquia.

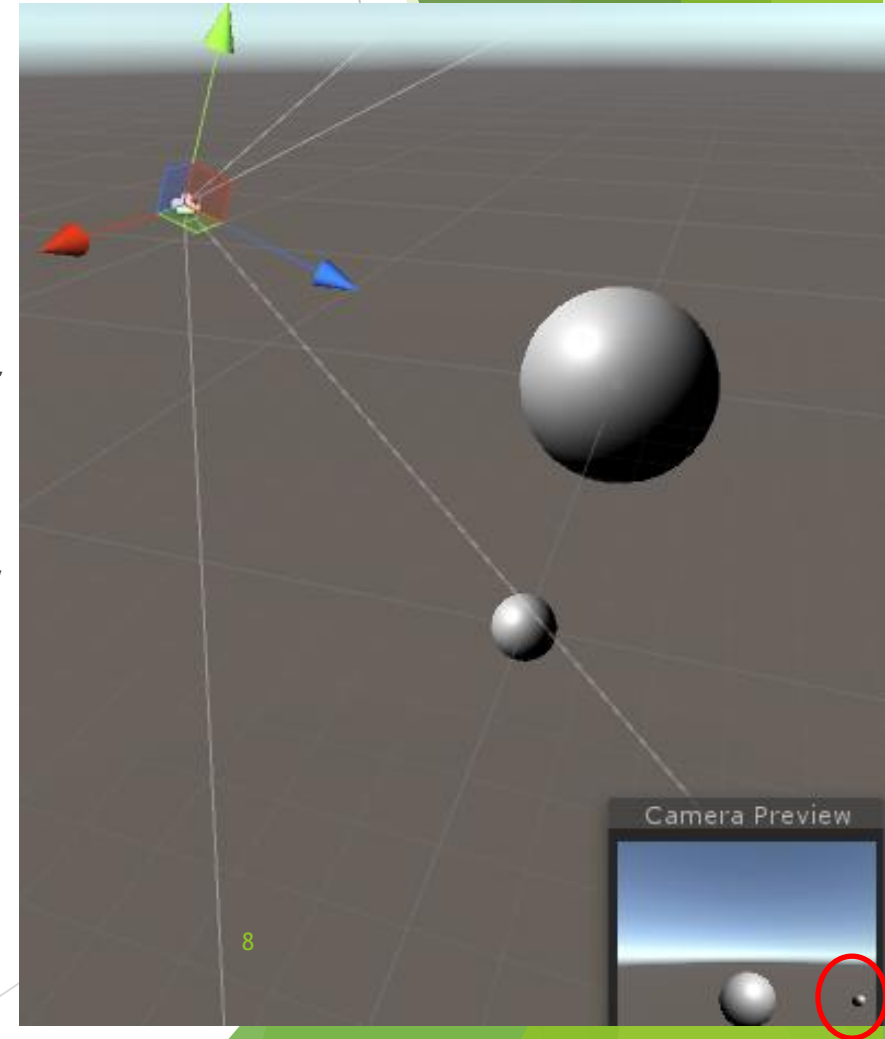


Escalando o Sol e incluindo a Terra



- Mude a escala do Sol para (5, 5, 5).
- Inclua mais uma esfera, chame-a de Terra na Hierarquia.
- Ela ficará centralizada na origem das coordenadas, dentro do Sol.

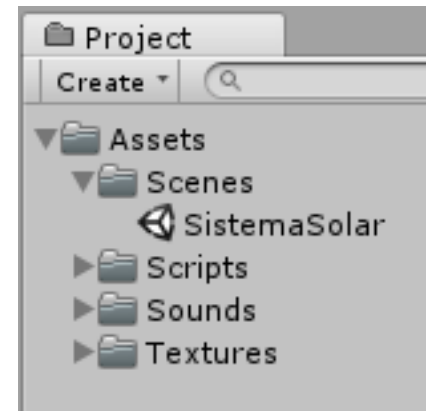
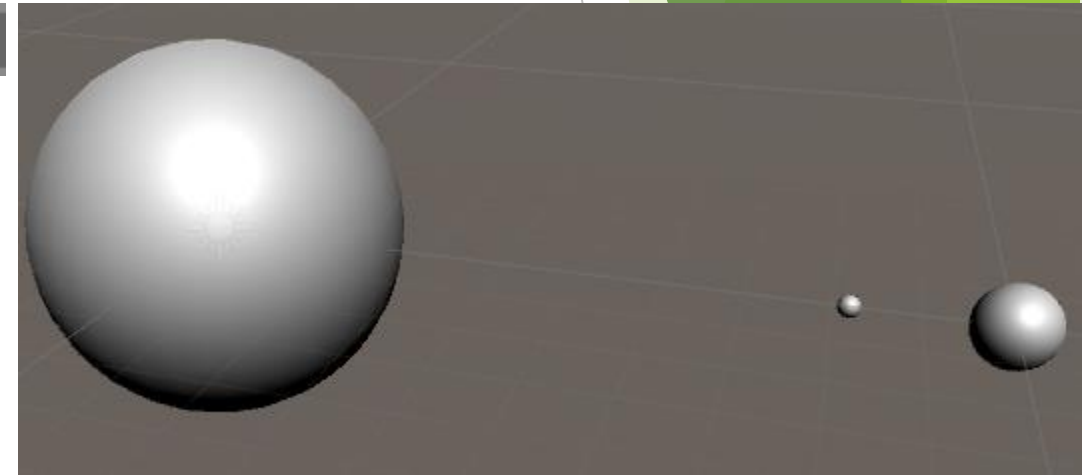
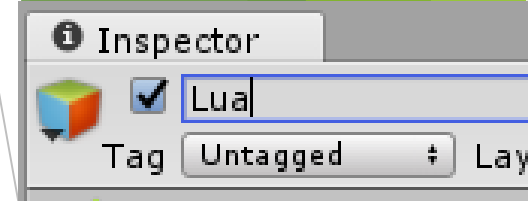
- ▶ Na janela Inspector, mude o transform Position da Terra para (8, 0, 0). Isso fará com que a Terra fique mais longe do Sol.
- ▶ Você pode usar a ferramenta de translação ou o teclado no Inspector para determinar a posição de qualquer objeto.
- ▶ Aumente a propriedade FieldOfView da Camera para 100 graus, para ver toda a expansão do Sistema Solar que construímos.



- ▶ Se, mesmo assim, a Terra ainda não aparecer, mova a câmera para que ela fique mais longe do centro, até que a Terra apareça, pequenina, ao lado do Sol. Se preciso, gire a câmera.

Criando e escalando a Lua

- ▶ Clique sobre a Terra em Scene ou Hierarchy, selecione a opção Edit | Duplicate ou pressione [Ctrl-D] para criar uma cópia da esfera da Terra.
- ▶ Mude o nome de Terra (1). Mude seu nome para Lua. Isso pode ser feito no Inspector ou na Hierarquia.
- ▶ Mude a escala (Transform / Scale no Inspector) da Lua para **(0.27, 0.27, 0.27)**.
- ▶ Arraste a Lua no eixo x usando a ferramenta de translação  de modo que ela fique entre o Sol e a Terra.
- ▶ Mude a câmera para a posição **(0,0,-10)** se já não estiver lá.
- ▶ Zere a rotação da camera → Rotation = **(0, 0, 0)**
- ▶ Salve a cena, usando File | Save Scene as, na pasta Scenes, com o nome SistemaSolar. Apague o arquivo da cena anterior (SampleScene).



Atribuindo comportamentos aos objetos

- ▶ Comportamento → scripts em C#
- ▶ Relacionamento Pai/Filho entre objetos
- ▶ Diferença entre Coordenadas do Mundo e Coordenadas Locais
- ▶ Abra a pasta Scripts e clique em RotateAround
- ▶ Esse é um Script pronto que faz um objeto girar ao redor de outro.
- ▶ Clique duas vezes para abrir o Visual Studio e editá-lo.

```
public class RotateAround : MonoBehaviour {
```

```
    public Transform target; // the object to rotate around  
    public int speed; // the speed of rotation
```

```
    void Update () {
```

```
        // RotateAround tem 3 parâmetros, o 1o é o ponto ao redor do qual girar  
        // o 2o é um vetor que indica o eixo de rotação (up aponta para cima)  
        // o 3o é velocidade angular (ângulo em graus a girar por segundo)
```

```
        transform.RotateAround(target.transform.position,  
                                target.transform.up,  
                                speed * Time.deltaTime);
```

```
    }
```

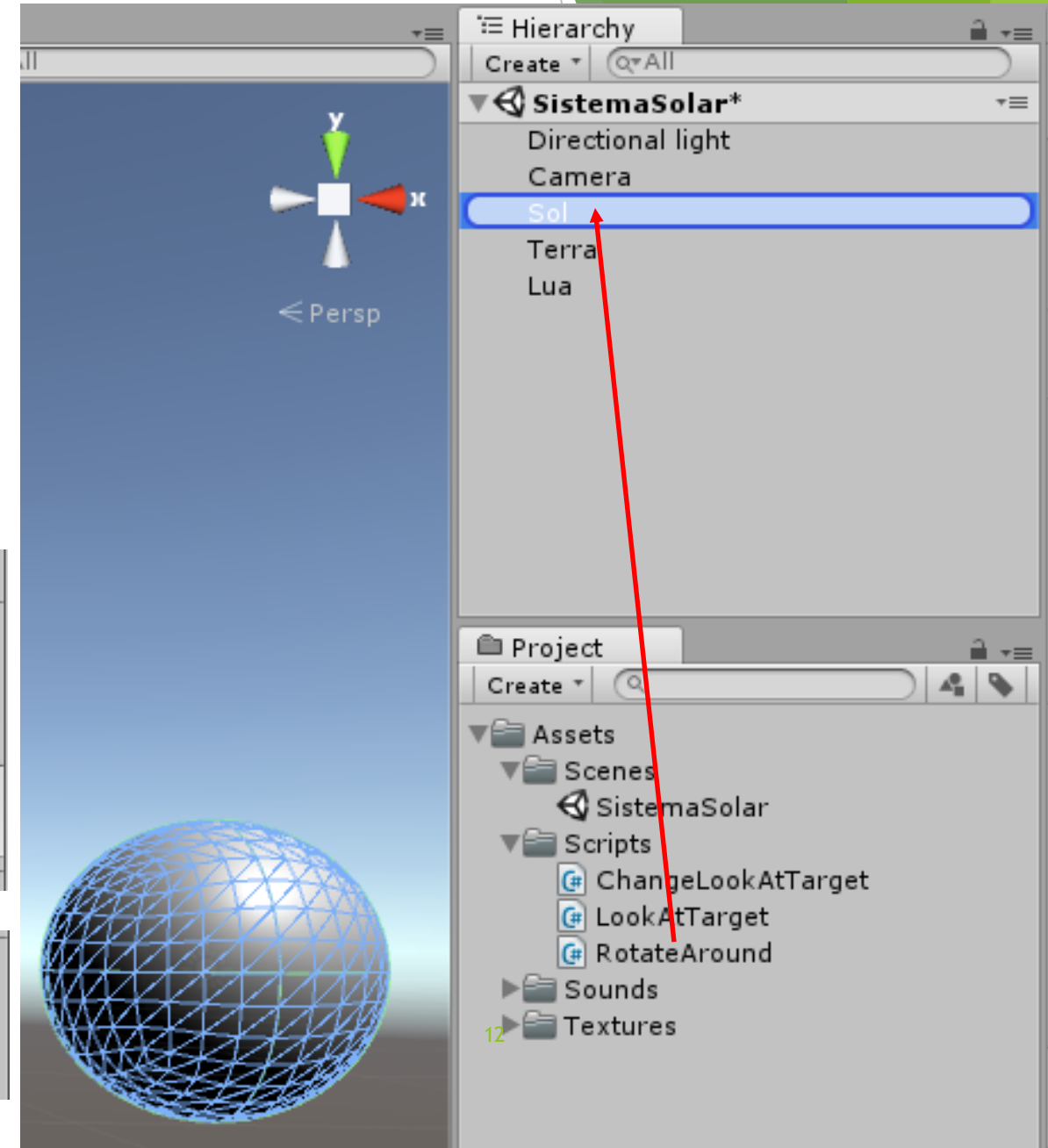
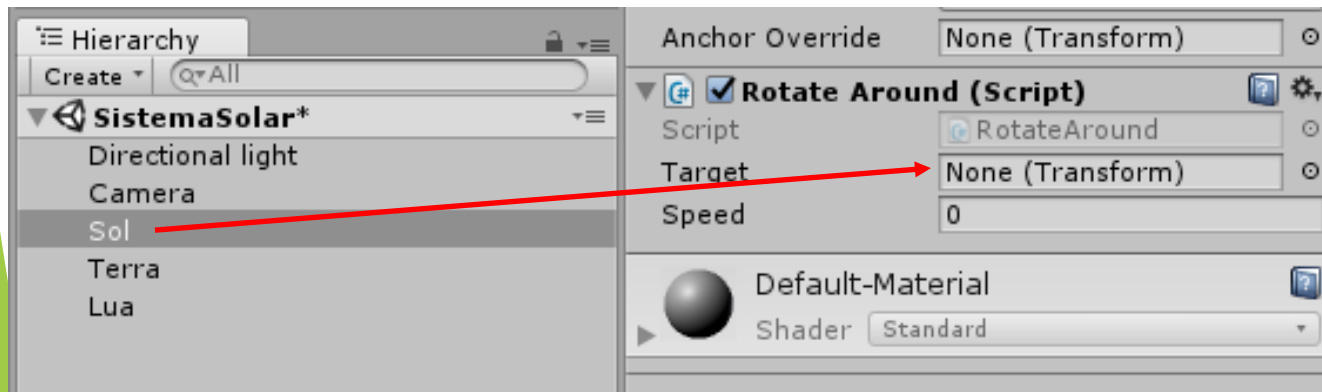
Variáveis globais públicas aparecem como campos que podem receber valores e referências a objetos no Inspector

Time.deltaTime adapta a velocidade do jogo à da GPU do computador

```
using UnityEngine;
using System.Collections;
public class RotateAround : MonoBehaviour
{
    public Transform target; // the object to rotate around
    public int speed;        // the speed of rotation
    void Start()
    {
        if (target == null)
        {
            target = this.gameObject.transform;
            Debug.Log ("RotateAround target not specified. Defaulting to parent GameObject");
        }
    }
    void Update ()
    {
        // RotateAround takes three arguments, first is the Vector to rotate around
        // second is a vector that axis to rotate around
        // third is the degrees to rotate, in this case the speed per second
        transform.RotateAround(target.transform.position,target.transform.up,speed * Time.deltaTime);
    }
}
```

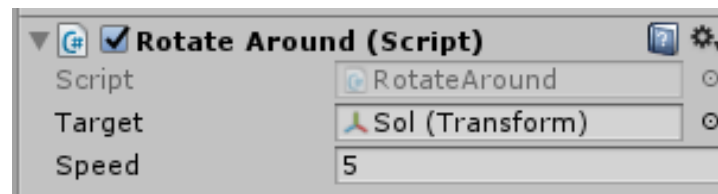
Atribuindo comportamentos aos objetos

- ▶ Para associar um script a um objeto, arraste-o sobre o objeto na Scene, ou na Hierarquia ou arraste-o sobre o Inspector, próximo ao botão [Add Component].
- ▶ Observe, na figura abaixo, que as **variáveis globais** Target e Speed do script RotateAround estão disponíveis para edição no Inspector.
- ▶ **Arraste** o objeto Sol da Hierarquia para a variável global Target e digite 5 em Speed (5 graus por segundo).



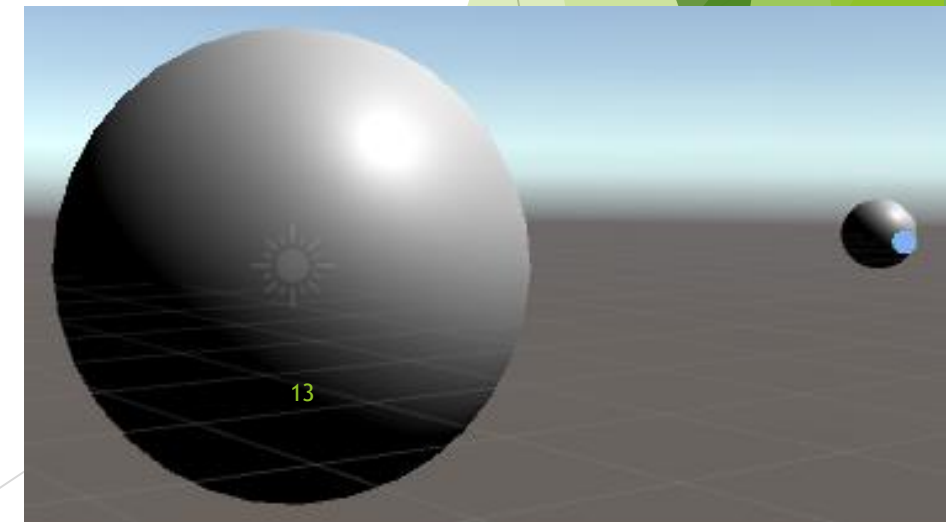
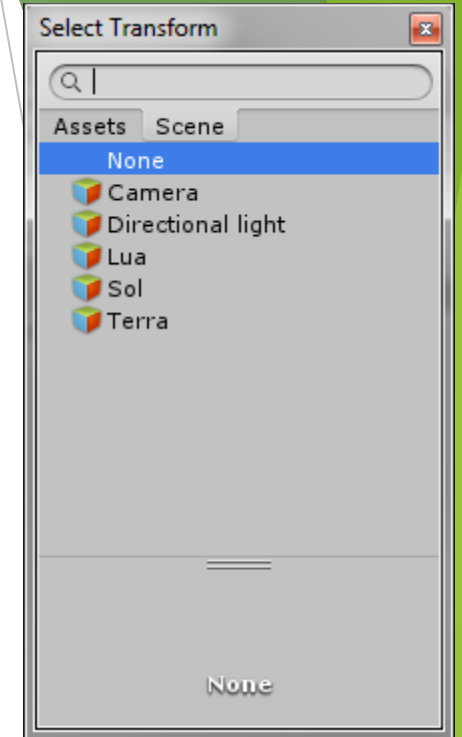
▶ Execute o jogo, pressionando o botão 

▶ Veja girar em Scene.



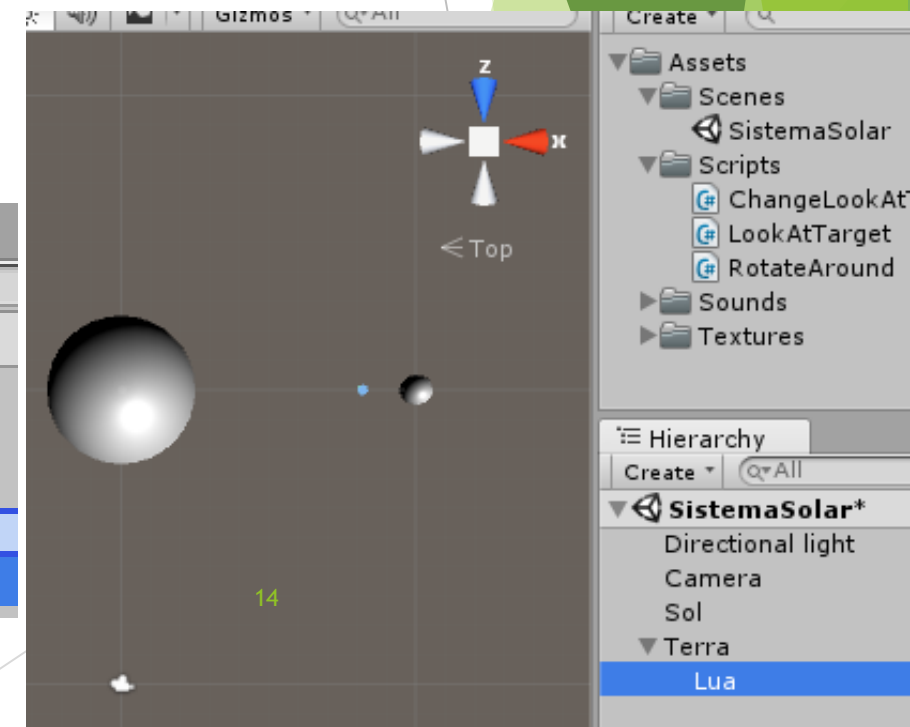
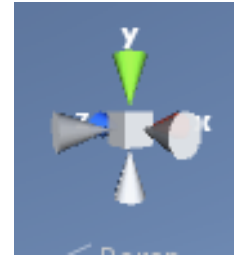
Fazendo a Terra e a Lua girarem

- ▶ Siga o mesmo procedimento e faça com que a Terra e a Lua girem ao redor delas mesmas. Atribua velocidade 15 para a Terra e 5 para a Lua.
- ▶ Execute o jogo e durante a execução, saia da janela Game e entre na janela Scene. Mude os valores de speed de qualquer um dos objetos e note que ele aumenta ou diminui sua rotação na janela Scene. Dessa maneira, você pode testar os melhores valores durante a execução, numa prototipação.
- ▶ Se você pressionar [Enter] na variável Target, aparecerá a janela “Select Transform”, com todos os objetos da classe Transform disponíveis na cena atual e poderá escolher o objeto desejado a partir dessa janela.
- ▶ Vamos agora fazer a Lua girar em torno da Terra.
- ▶ Para isso, arraste novamente o script RotateAround para a Lua e informe que o Target é a Terra, com Speed 15.
- ▶ Execute e veja a Lua girando ao redor da Terra.
- ▶ Faça também a Terra girar ao redor do Sol, com Speed 10.



Fazendo a Terra e a Lua girarem

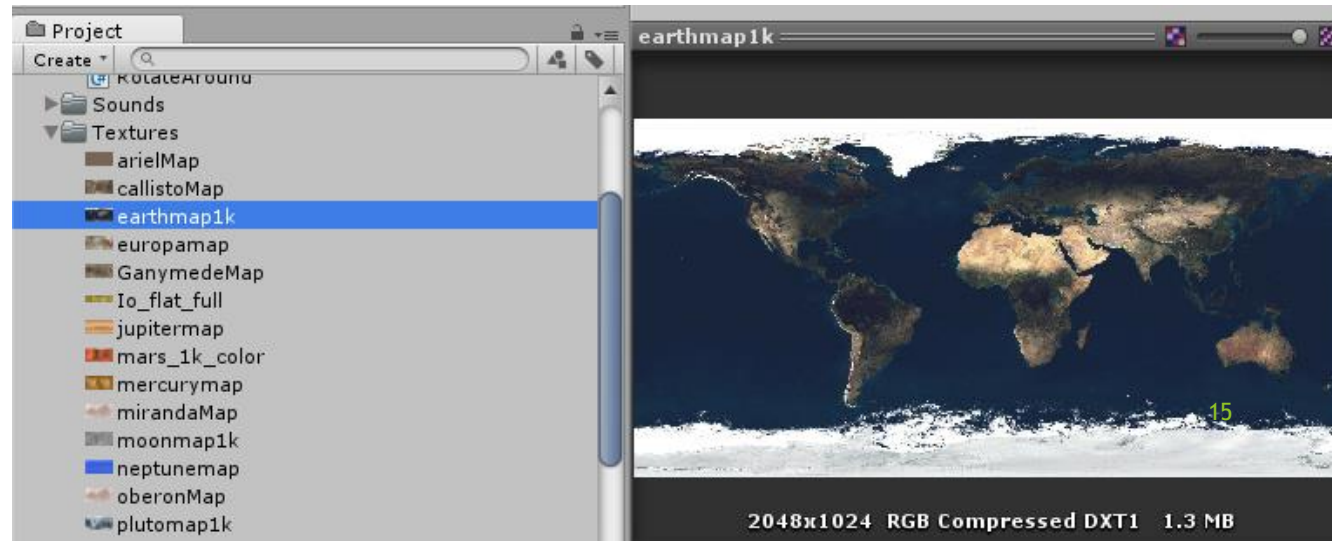
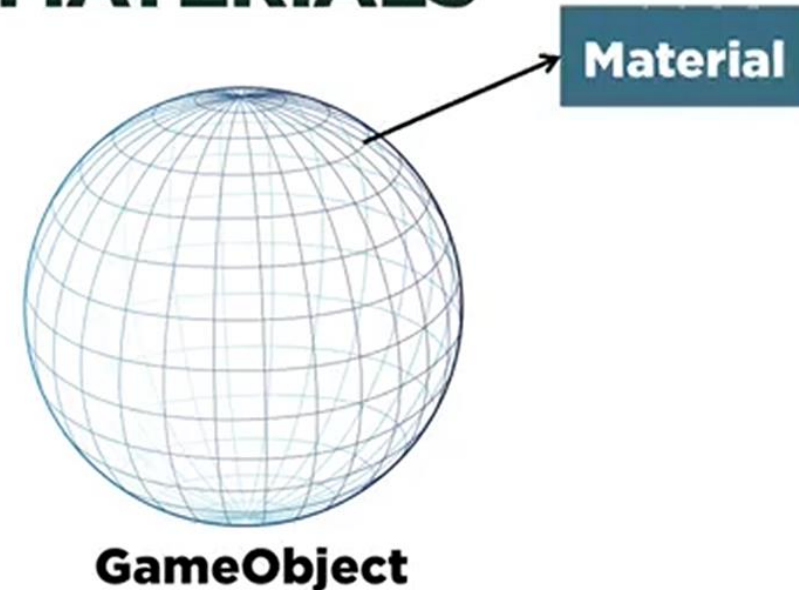
- ▶ Ao realizar as operações anteriores, você deve ter percebido que a Lua age de forma estranha, aparentemente “caçando” a Terra enquanto esta gira ao redor do Sol.
- ▶ Clique no mostrador do eixo Y para ver a cena de cima, e execute o jogo. Em seguida volte à janela Scene para ver a execução de cima. Perceba o comportamento da Lua. Pare a execução e retorne à Scene.
- ▶ Esse comportamento da Lua acontece porque a Terra saiu do lugar, ao girar ao redor do Sol. O sistema de coordenadas da Lua deixou de ter como referência a posição da Terra.
- ▶ Para resolver isso, temos que fazer com que a Lua passe a ser uma “filha” da Terra do ponto de vista hierárquico. Isso é feito arrastando, na janela Hierarchy, a Lua sobre a Terra.
- ▶ Execute novamente e perceba que, agora, o movimento dos 3 astros parece correto.



Materials

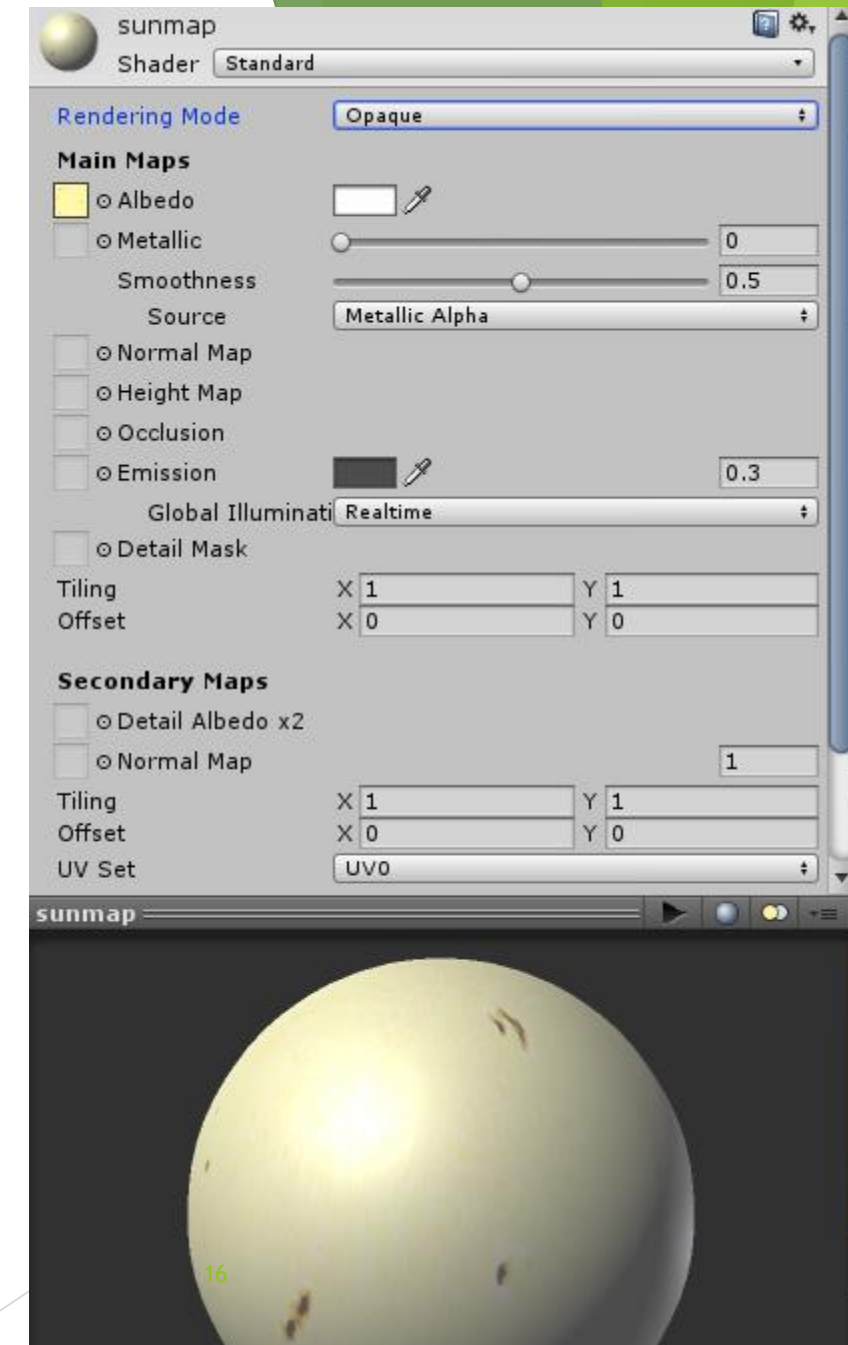
- ▶ Um GameObject é geralmente feito de uma geometria tridimensional, representada aqui como um wireframe (modelo de arame).
- ▶ Um **Material** é como um papel de embrulho que envolve um GameObject, **tornando-o visível**.
- ▶ Um shader é basicamente uma fórmula que define como o material é gerado, dando sua aparência, brilho, opacidade, espelhamento, textura, dentre outras características visuais.
- ▶ Um dos shaders mais comuns é a textura, que é uma imagem sobreposta ao material para dar a aparência visual ao objeto.
- ▶ Vamos criar materiais para o Sol, Terra, Lua e demais astros de nosso simulador.

MATERIALS



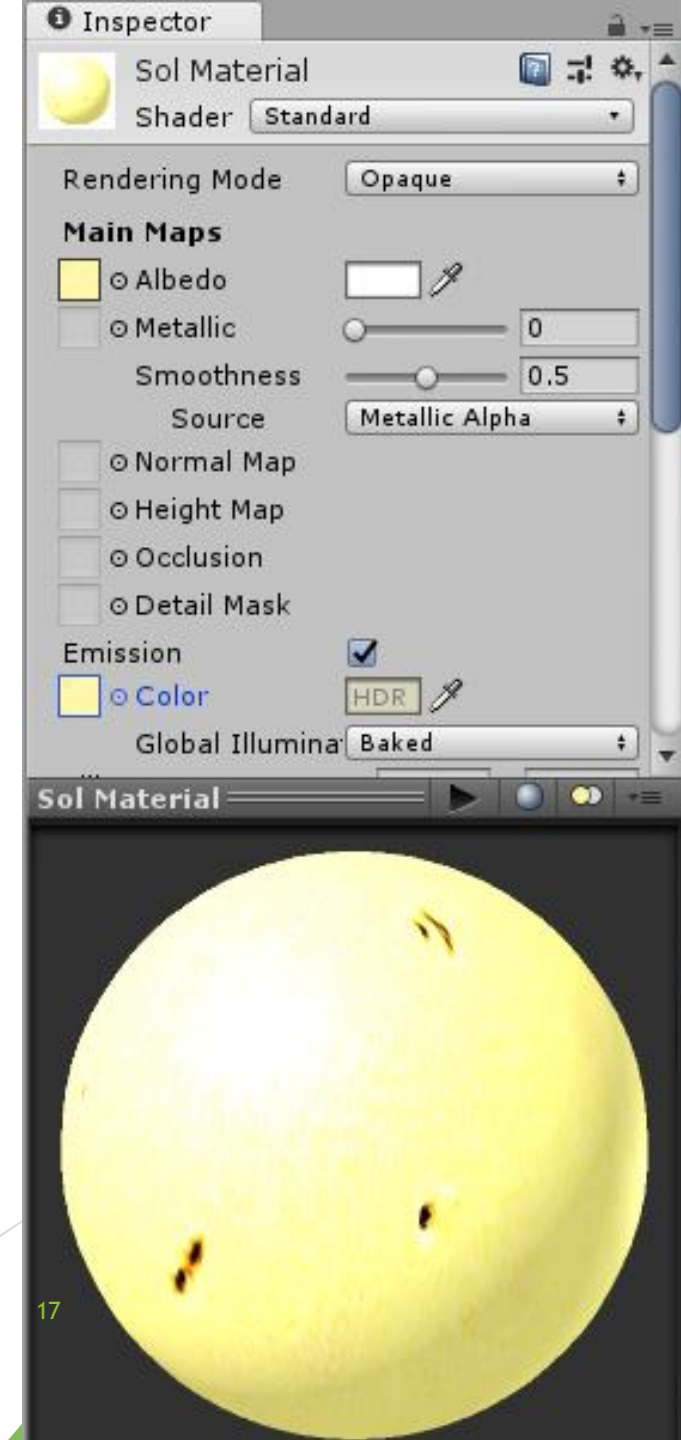
Materials e Textures

- ▶ Se você arrastar a Textura SunMap para o objeto Sol no Inspector, notará que o Unity criou uma pasta Materials automaticamente. Dentro dessa pasta, criou um arquivo de Shader informando como esse novo material se adequa ao objeto Sol.
- ▶ Você pode modificar os atributos do material e faremos isso logo mais.
- ▶ Crie o Material da Terra e o da Lua, arrastando seus respectivos mapas para as esferas de cada astro.
- ▶ Na janela Project, modifique os nomes dos componentes de Materials para Sol Material, Terra Material e Lua Material.
- ▶ Você pode usar um Material já criado em diferentes objetos. Por exemplo, ter uma Terra Bebê se colocar o Terra Material no objeto Lua.
- ▶ Se modificar um Material usado em vários objetos, a visualização de cada um deles passará a ser feita de acordo com a nova configuração do material.



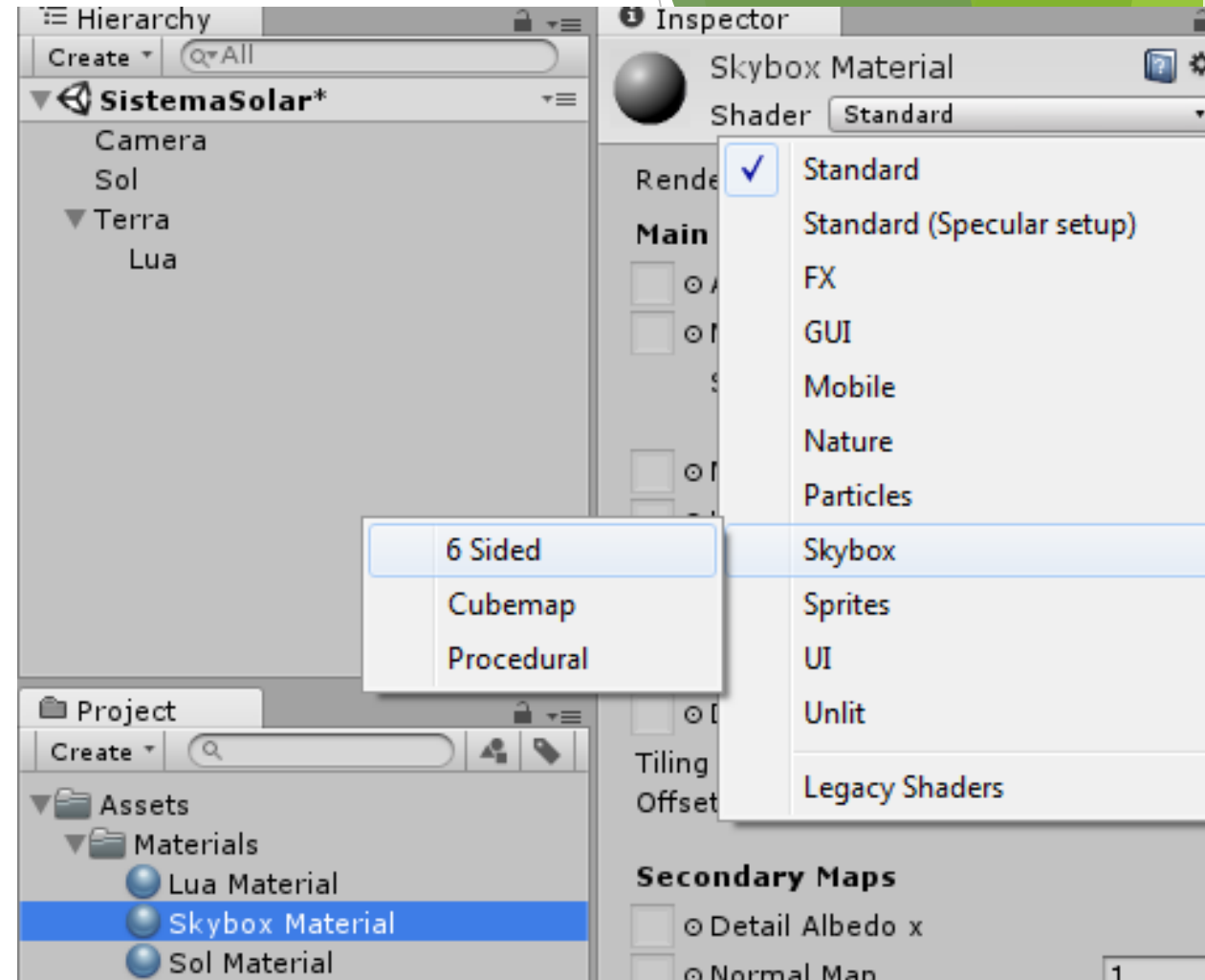
Materials e Textures

- ▶ Se executarmos nosso jogo, veremos que o Sol possui sombras e que ele realmente não emite luz alguma. Está sendo iluminado pela Directional Light que adicionamos no início do nosso projeto.
- ▶ Remova essa Directional Light e isso fará o aspecto da cena mudar radicalmente.
- ▶ Modifique o Shader do **Sol Material** para que a propriedade **Emission** passe a ter efeito.
 - ▶ Clique no checkbox Emission para ativar sua configuração.
 - ▶ Clique no círculo ao lado esquerdo da palavra Color e selecione a textura sunMap, de forma que as manchas solares apareçam.
 - ▶ Clique no quadro à direita de Color (HDR) e escolha uma tonalidade amarela para a emissão de luz.



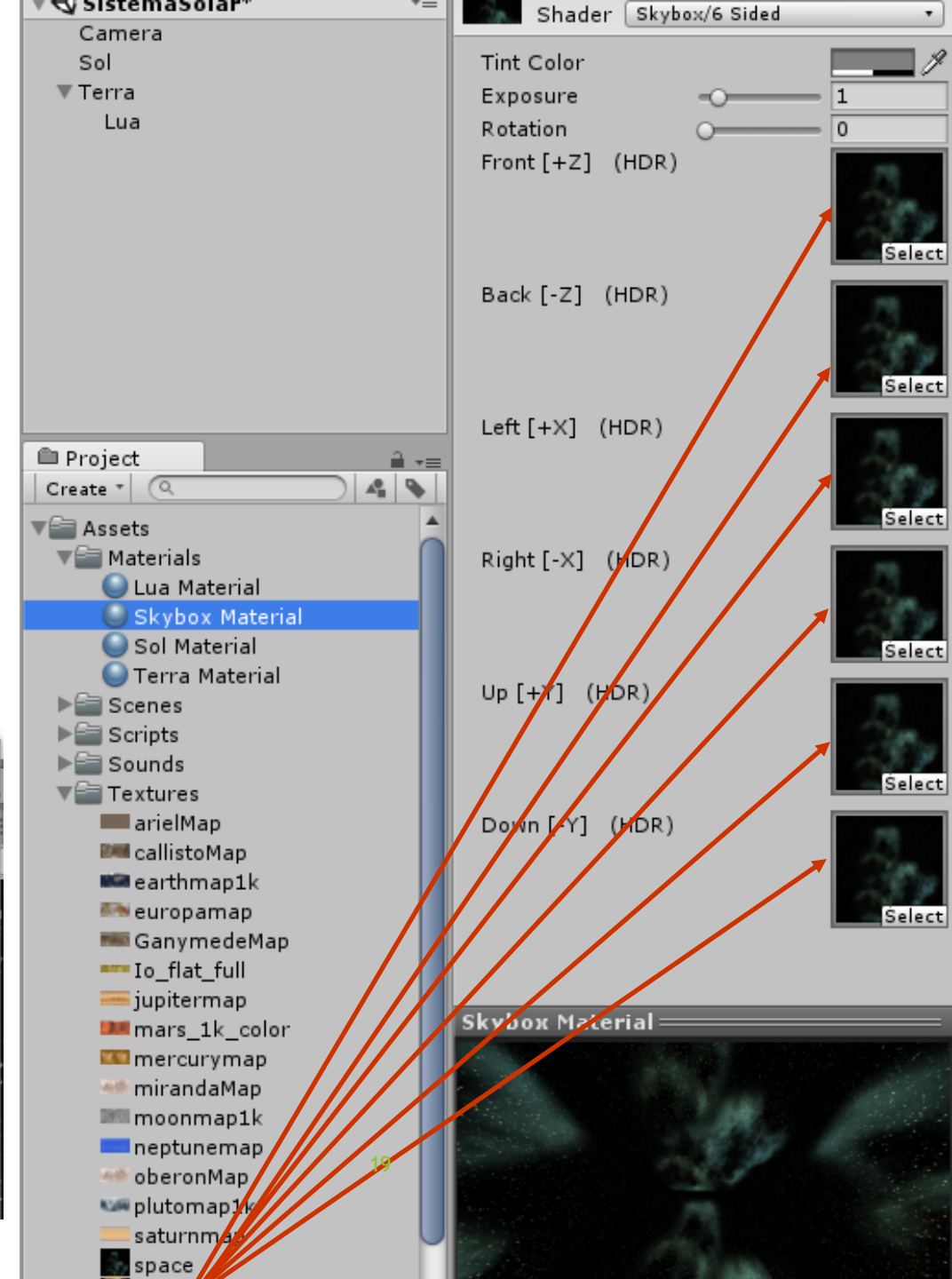
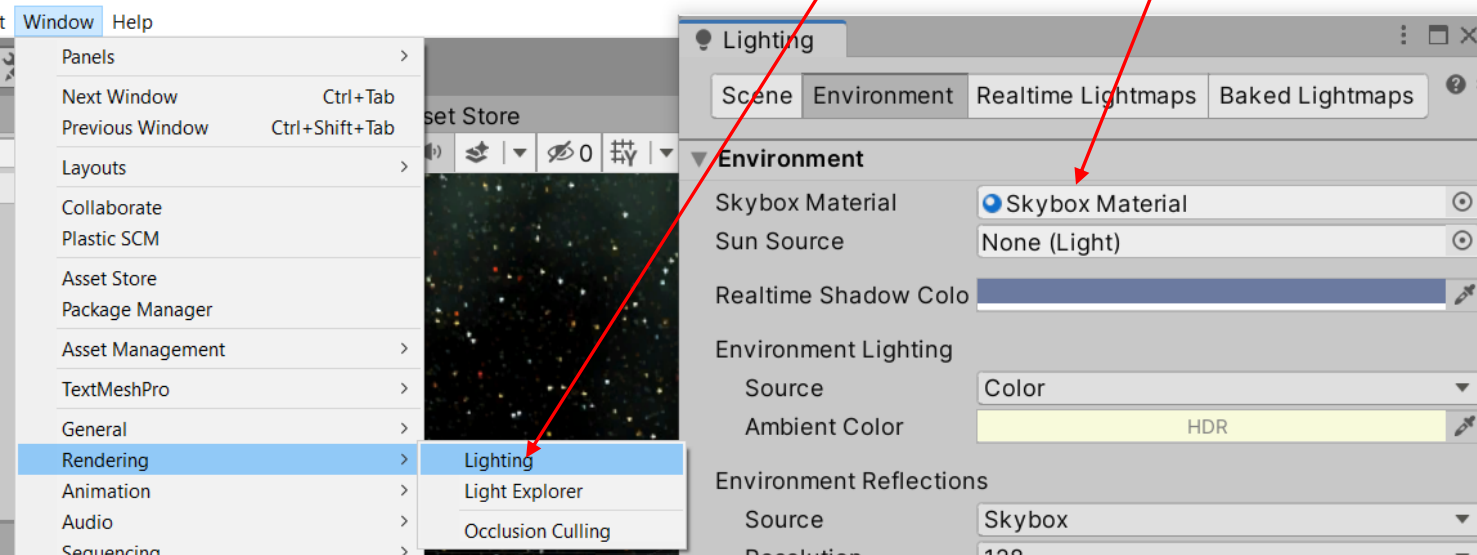
Criando um novo Skybox

- ▶ Na janela Project, na pasta Materials, crie um novo Material, chamando-o de SkyBox Material.
- ▶ Na parte superior do Inspector, observe o tipo de Shader, que é Standard no momento. Abra a lista de Shaders e escolha Skybox e, em seguida, 6 Sided.
- ▶ Criaremos um skybox com 6 lados, como se nossa cena estivesse dentro de um cubo (e estará, para efeito do skybox).



Criando um novo Skybox

- ▶ Selecione o Skybox Material na pasta Materials, de forma que no Inspector apareçam os 6 lados do cubo de skybox
- ▶ Arraste a textura (pasta Textures) space para cada um dos lados.
- ▶ O Skybox está pronto. Agora temos que colocá-lo em cena.
- ▶ Vá no menu Window | Rendering | Lighting | Guia Environment | item Skybox Material.
- ▶ Nesse item, escolha como skybox o material Skybox Material que criamos agora há pouco.



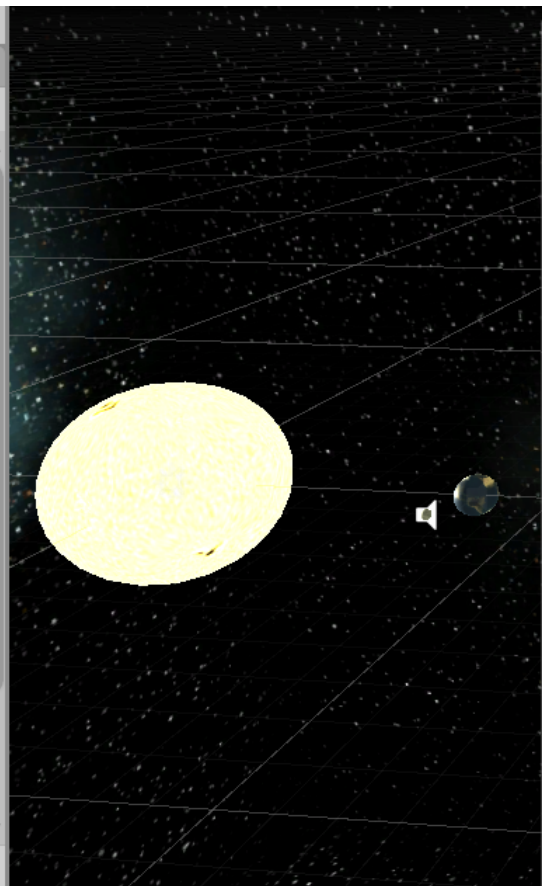
Project

Assets

- Materials
 - Lua Material
 - Skybox Material
 - Sol Material
 - Terra Material
- Scenes
- Scripts
 - ChangeLookAtTarget
 - LookAtTarget
 - RotateAround
- Sounds
 - Beethoven - Moonlight Sonata (FULL burning)
 - dronedarkandscary
 - dronehum
 - Earth - WHAT A WONDERFUL WORLD
 - Tomita Planets - Jupiter
 - Tomita Planets - Mars
 - Tomita Planets - Mercury
 - Tomita Planets - Neptune

Assets/Materials/Skybox Material.mat

Visual Studio Editor Package version 2.0.11 is available, we strongly encourage you to update



Lighting

Scene Environment Realtime Lightmaps Baked Lightmaps

Environment

Skybox Material Skybox Material

Sun Source None (Light)

Realtime Shadow Color

Environment Lighting

Source Color

Ambient Color HDR

Environment Reflections

Source Skybox

Resolution 128

Compression Auto

Intensity Multiplier 1

Bounces 1

Other Settings

Fog

Halo Texture None (Texture 2D)

Halo Strength 0.5

Inspector

Skybox Material (Material)

Shader Skybox/6 Sided

Tint Color

Exposure 1

Rotation 0

Front [+Z] (HDR)

Back [-Z] (HDR)

Left [+X] (HDR)

Right [-X] (HDR)

Skybox Material

Iluminação (Lighting)

- ▶ Objetivos da Iluminação
 - ▶ Iluminação → ver os objetos do mundo do jogo
 - ▶ Modelagem → revelar as qualidades estruturais e dimensionais dos objetos do jogo, bem como sombreamento, relacionamentos entre objetos e a hora do dia
 - ▶ Foco → definir contrastes e dirigir a atenção do usuário
 - ▶ Estilo Visual

Iluminação (Lighting)

Tipo de Luz	Descrição	Propósito Primário
Ambiente	Luz refletida	Iluminação da cena
Direcional	Luz solar (raios)	Iluminação e modelagem
Pontual	Lâmpada	Iluminação e modelagem
Spot	Luz de flash	Modelagem e foco

Luzes não tem geometria, vemos seus resultados mas não a luz por si própria.

Se você quiser ver uma luz, deverá ter um modelo 3D dessa luz e colocar a luz de tempo real dentro dela para, literalmente, acendê-la.

Luz usam o processador intensivamente. Quanto mais luzes tivermos, mais diminuirá o desempenho do jogo.

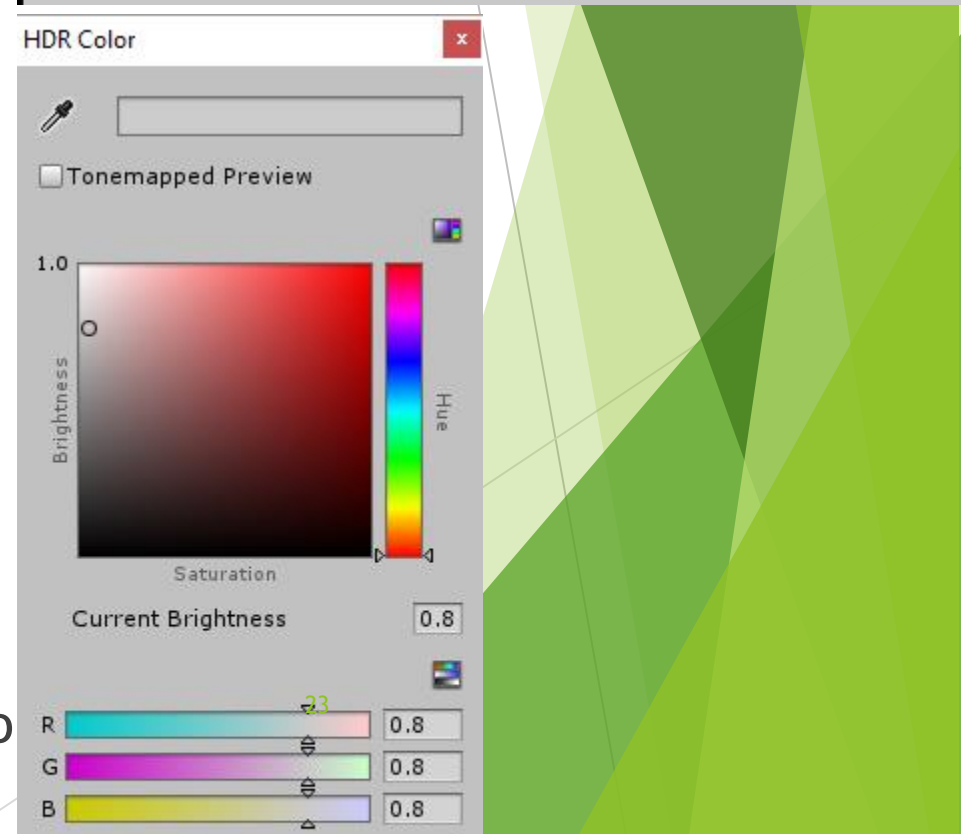
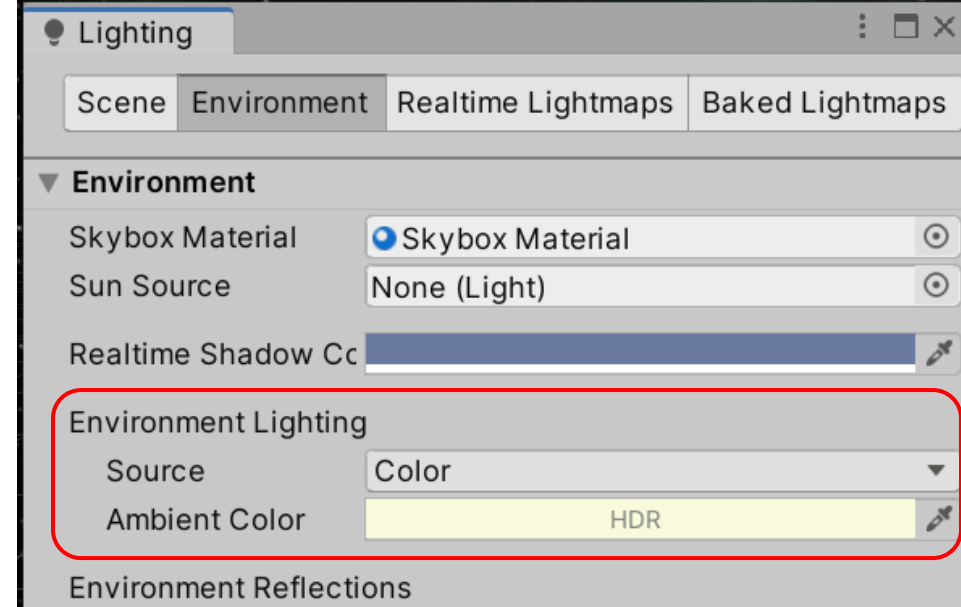
Luzes permitem criar efeitos especiais como sombras.

Esses efeitos consomem muito tempo de processamento.

Temos que ser judiciosos na utilização de iluminação se quisermos bom desempenho.

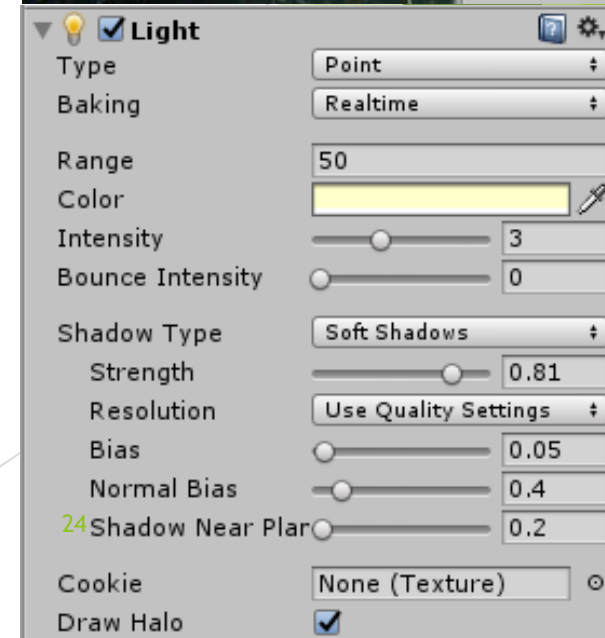
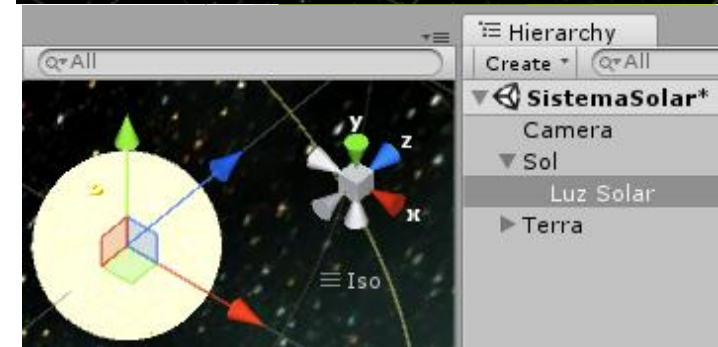
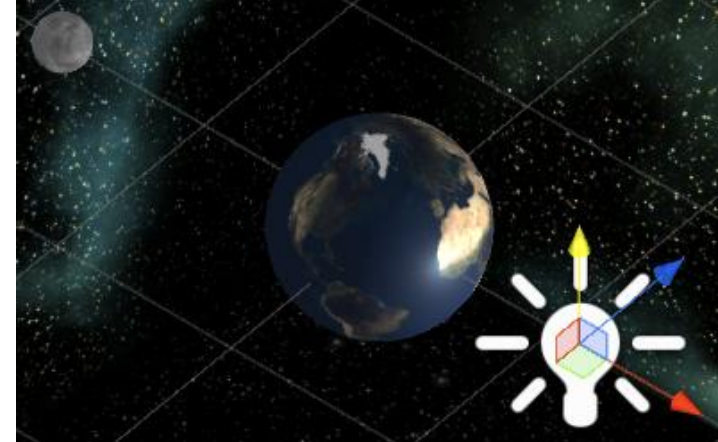
Iluminação do Sistema Solar

- ▶ Vamos ajustar a luz ambiente do nosso Sistema Solar e colocar uma iluminação no interior do Sol, para que este pareça estar emitindo luz em direção aos planetas e luas.
- ▶ No caso do objeto Sol, ele parece emitir luz mas isso é apenas um efeito visual criado na propriedade Emission do seu Material. Os planetas continuam escuros.
- ▶ Podemos modificar a luz ambiente na opção Window | Rendering | Lighting | Guia Environment. Onde vemos a propriedade Environment Lighting | Source igual a Skybox, selecione **Color**. A cor Ambient Color que vemos é Cinza, com intensidade igual a 1.
- ▶ Clique em Ambient Color para exibir a ferramenta HDR Color. Configure as medidas de RGB para (0.8,0.8,0.8) e observe que a Terra e a Lua passam a ter texturas visíveis, e que a luz ambiente ficou acinzentada. Você pode mudar os valores para ver o resultado. Por exemplo, deixar a Terra avermelhada.
- ▶ Vamos criar uma Luz Pontual (Point Light) e colocá-la no centro do Sol, para que este emita essa luz sobre o restante da cena.



Iluminação do Sistema Solar

- ▶ GameObject | Light | Point Light cria a Luz Pontual. Faça reset nela.
- ▶ Você sabe que luzes são como quaisquer outros GameObjects, de forma que existem no mundo do jogo, possuem transforms também e podem ser posicionadas onde desejarmos e são visíveis por si mesmas.
- ▶ Inclua um objeto Point Light, mova-o para perto da Terra e da Lua para ver seus efeitos sobre esses objetos e, depois, mova-o para o centro do Sol. Mude seu nome para Luz Solar e o subordine ao Sol na Hierarquia.
- ▶ Mude a propriedade Range da Luz Solar para 50, de forma que ela ilumine até 50 metros de distância no nosso modelo do sistema solar.
- ▶ Altere a cor da Luz Solar para que ela emita um amarelo claro.
- ▶ Na Luz Solar, mude a propriedade Intensity para 3 e Shadow Type para Soft Shadow. Coloque Draw Halo como true. Haverá sombras quando executar. Afaste a câmera para ver o halo na janela Scene enquanto executa o jogo.
- ▶ Mude a variável Speed do Script da Lua ao redor da Terra para -10. Se você usou os mesmos valores dados anteriormente, a Lua ficará sempre sobre a mesma posição entre a Terra e o Sol, mas não haverá sombras quando executar.



Áudio

- ▶ Em Unity, áudio funciona de maneira bastante simples.
- ▶ Você tem um componente AudioListener que, como um microfone, obtém sons de certos locais no ambiente.
- ▶ Em geral, o AudioListener é ligado à câmera, e você somente pode ter um único AudioListener ligado na cena a cada momento, embora possam existir vários deles na cena.
- ▶ O AudioListener obtém o som emitido por um AudioSource.
- ▶ Unity suporta Som 3D posicional, ou seja, obtém o som a partir da posição do objeto que emite (direita, esquerda, etc). A distância do emissor influi no volume do som.
- ▶ Observe o componente Camera. Ele já possui um AudioListener.
- ▶ Arraste o arquivo Burning da pasta Sounds para o Sol na Hierarquia. Isso criará um componente AudioSource.
- ▶ Mude o volume para 0.5 e marque Loop para que o som se repita.
- ▶ Faça o mesmo com a Terra, associando o arquivo “Earth - WHAT A WONDERFUL WORLD - Instrumental Version - complete” com volume 0.5 e Loop. Execute.



Áudio

- ▶ Em um jogo real nós poderemos controlar o AudioListener programaticamente para ligá-lo ou desligá-lo
- ▶ Pode-se também alternar o jogo de um AudioListener para outro se tivermos várias câmeras no jogo. Por exemplo, num jogo de corrida, você pode alternar a câmera entre cada carro e uma vista geral, apresentando um efeito sonoro diferente para cada uma delas.
- ▶ <http://producaodejogos.com/como-produzir-audio-para-games/>
- ▶ <http://www.audiomicro.com/free-sound-effects/free-foley>
- ▶ [https://pt.wikipedia.org/wiki/Foley_\(cinema\)](https://pt.wikipedia.org/wiki/Foley_(cinema))

Câmeras

- ▶ Em alguns aspectos, a câmera 3D do Unity pode ser vista conceitualmente como uma câmera real, pois para configurar sua visualização você precisa posicioná-la apropriadamente e rotacioná-la de forma que ela aponte para o local que você deseja ver.
- ▶ Numa engine de games, a câmera é uma projeção de uma geometria 3D sobre um plano 2D do monitor de vídeo.
- ▶ Você pode simular efeitos como foco, flare e olho de peixe, mas eles exigem muito do processador.
- ▶ Há dois tipos de projeção que você pode criar no seu jogo:
- ▶ Perspectiva → como uma câmera real, jogos de 1ª e 3ª pessoas
- ▶ Ortográfica → jogos 2D de plataforma, por exemplo
- ▶ O frustrum (pirâmide) de visualização determina o que será visualizado na tela.
- ▶ Na visão perspectiva, a câmera exibe tudo que estiver entre uma distância mínima e uma distância máxima, colocando os tamanhos em escala de acordo com a distância.
- ▶ Na visão ortográfica, os tamanhos relativos se mantêm com a distância.

Câmeras

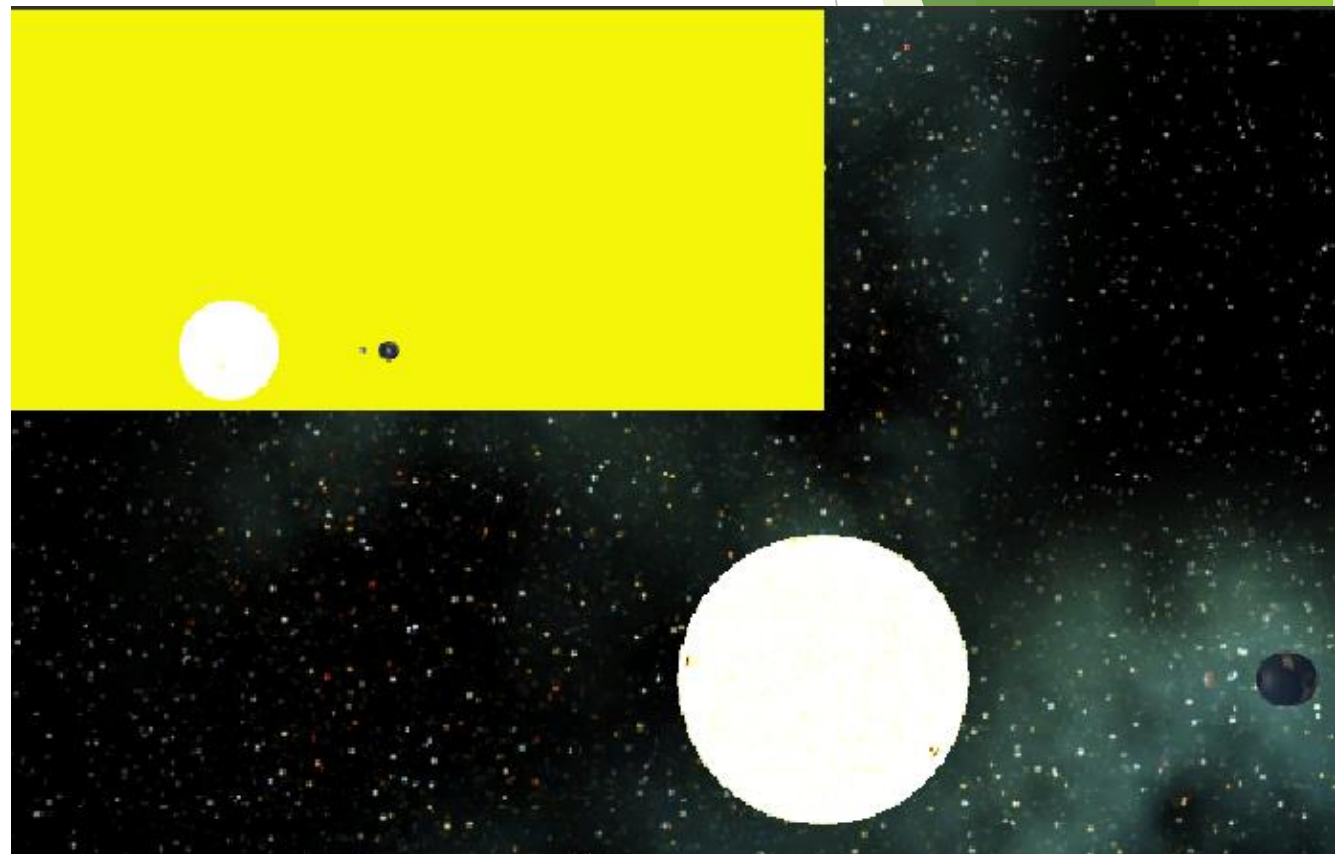
- ▶ Agora, vamos modificar nosso jogo para termos os seguintes recursos:
 - ▶ Entender algumas das configurações da câmera
 - ▶ Controlar a câmera de perspectiva
 - ▶ Criar um mini-mapa através do uso de uma segunda câmera, em visão ortográfica.
- ▶ Clique na Câmera da cena. Observe que o Frustum de Visualização aparece como uma pirâmide deitada, com a ponta na posição da câmera. Modifique a propriedade Far para 120, de forma que possamos diminuir a distância em que objetos ficam visíveis à frente da câmera. Se você mudar Far para 33, por exemplo, a Terra desaparecerá quando passar ao lado do Sol, se a câmera estiver posicionada em $Z = -32$ ou algo parecido.
- ▶ Mude também o Viewport Rect, para que a largura (W) fique valendo 2 e a altura (H) continue valendo 1.

Câmeras

- ▶ Vamos agora criar uma outra câmera em nosso jogo, com visualização ortográfica. Vá em GameObject, selecione Camera.
- ▶ Uma nova visualização ocorreu, e não é exatamente a que tínhamos antes. Queremos que essa câmera fique em outra posição, vendo a cena de cima.
- ▶ Selecione a câmera inicial e, em GameObject, selecione a opção [Align View to Selected].
- ▶ Selecione a segunda câmera e, em GameObject, selecione a opção [Align with View]. Isso fará com que a segunda câmera fique alinhada no mesmo local onde a primeira câmera aponta.
- ▶ Mude o nome da segunda câmera para Camera Mini Mapa, pois a usaremos para mostrar um mini-mapa em nossa cena.
- ▶ Observe que ela possui um AudioListener e que, como somente um AudioListener pode estar ativo na cena, temos que desligá-lo, clicando no checkBox ao lado esquerdo das palavras Audio Listener na janela Inspector.
- ▶ Mude Projection para Orthographic e note como a visualização da cena muda.
- ▶ Se executar, o tamanho dos objetos não muda conforme eles passam na cena.

Câmeras - criando o mini-mapa

- ▶ Clique no triângulo verde da cena (eixo y) para mudar a visualização para sendo vista de cima.
- ▶ Clique na Camera Minimapa e, em GameObject, selecione [Align with View].
- ▶ Para projetarmos a visualização da Camera Minimapa sobre a visualização da câmera principal, temos que fazer um empilhamento de visualizações.
- ▶ Selecione a Main Camera e, na propriedade Render Type, mantenha Base. Isso fará com que esta visualização seja a que fica na base da pilha de visualizações.
- ▶ Selecione agora a Camera Minimapa e, em sua propriedade Render Type, selecione Overlay. Ela ficará, assim, sobreposta a outras visualizações.
- ▶ Selecione a Main Camera e, em Stack, adicione a Camera Minimapa que, assim, ficará sobre a visualização da câmera principal.



Câmeras - criando o mini-mapa

- ▶ No ViewPort Rect da Camera Minimapa, coloque $X = 0$, $Y = 0.7$, $W = 0.5$ e $H = 0.3$.
- ▶ Associe o som DroneDarkAndScary à Lua e execute o jogo.
- ▶ Observe que os sons ficam embaralhados.
- ▶ O minimapa está amarelo aqui para enfatizar a Lua.

Alterando a visualização da Câmera

- ▶ Para permitir que a câmera faça zoom no astro em que clicarmos, associe a Main Camera com o script LookAtTarget.
- ▶ Associe o script ChangeLookAtTarget a cada um dos astros que clicaremos.
- ▶ Associe cada objeto com a variável Target desse script associado ao objeto mas, **no caso do Sol, coloque target como a Luz Solar**, para que não haja um efeito indesejado que distorce toda a imagem quando a câmera apontar o sol.
- ▶ Se você se cansou de ver a Lua estacionária em relação à Terra e sempre fazendo sombra no mesmo lugar, num eterno eclipse solar, mude a variável speed do script RotateAround novamente para 10.
- ▶ Num script, a variável **transform** sempre se refere ao objeto associado ao script. No caso de LookAtTarget, transform é a câmera, de forma que o método LookAt faz com que a câmera aponte ao objeto da variável target (Luz Solar, Terra, Lua, etc.):

```
void update() {  
    if (target != null)  
        transform.LookAt(target.transform);  
}
```

Alterando a visualização da Câmera

Em versões recentes da Unity, ao usar apenas o **New Input System**, alguns projetos apresentam problemas com APIs antigas baseadas em mouse.

Veja em:

Edit → Project Settings → Player → Active Input Handling

Valores possíveis:

- Input Manager (Old)
- Input System Package (New)
- Both

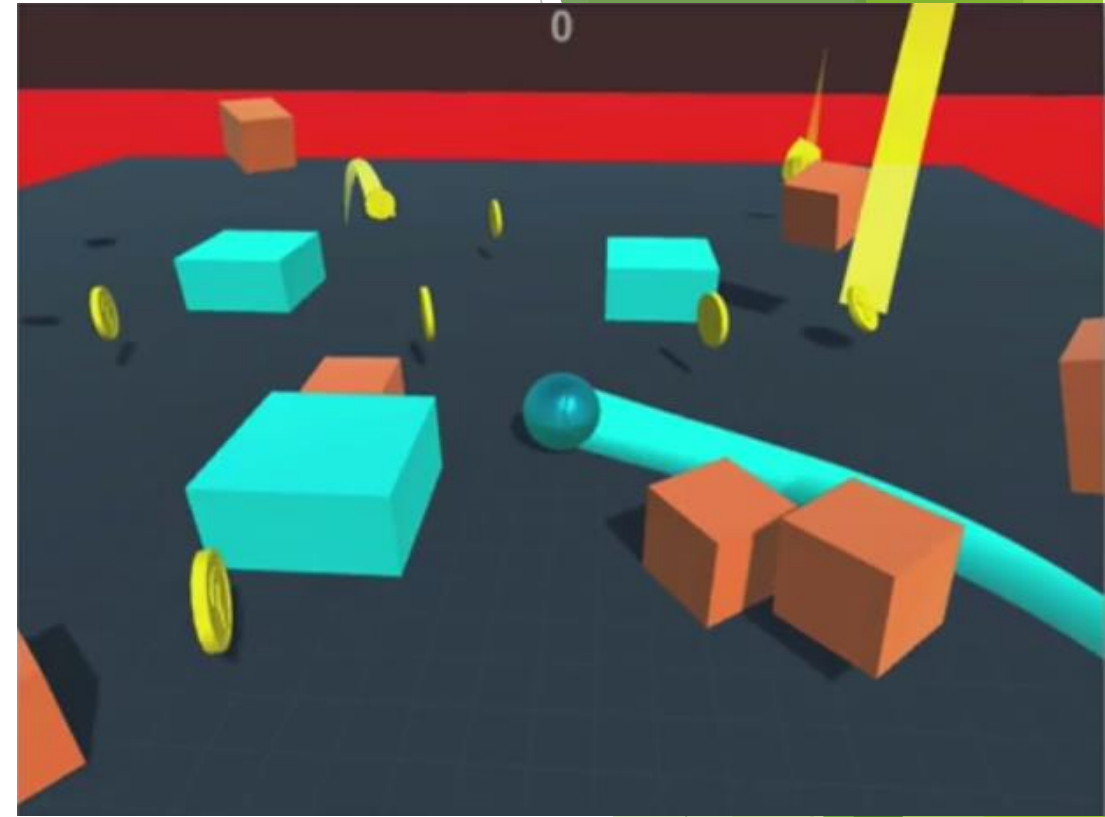
Para teste, coloque **Both**.

Projeto para avaliação - Sistema Solar

- ▶ Agora, como tarefa de **avaliação**, complemente esse projeto incluindo os demais planetas, em posições adequadas, com suas luas e inclinações.
- ▶ Pesquise os dados de cada planeta e satélite, como nome, diâmetro, distância ao Sol ou ao planeta e o ângulo de inclinação do eixo de rotação dos planetas.
- ▶ Procure fazer um simulador, não se preocupando com tamanhos e distância muito reais. O importante é fornecer uma visualização da dinâmica do Sistema Solar.
- ▶ Lembre-se dos anéis de Saturno, que Urano gira deitado e Vênus gira ao contrário em torno de si mesmo. Os satélites sempre orbitam no mesmo plano que o equador do planeta.
- ▶ Associe arquivos de áudio a cada planeta. Você pode usar outra trilha sonora, caso assim deseje. Pesquise como fazer com que apenas a música do planeta selecionado seja tocada.
- ▶ No início, a única música selecionada deve ser a do Sol (burning).
- ▶ Crie, também, uma segunda estrela (anã-vermelha), orbitando o Sol original, com seu próprio sistema de planetas. Pesquise sobre Rastros (Trail Renderer) e deixe um rastro em cada planeta desse segundo sistema solar enquanto ele orbita a estrela vermelha. (Trappist-1)

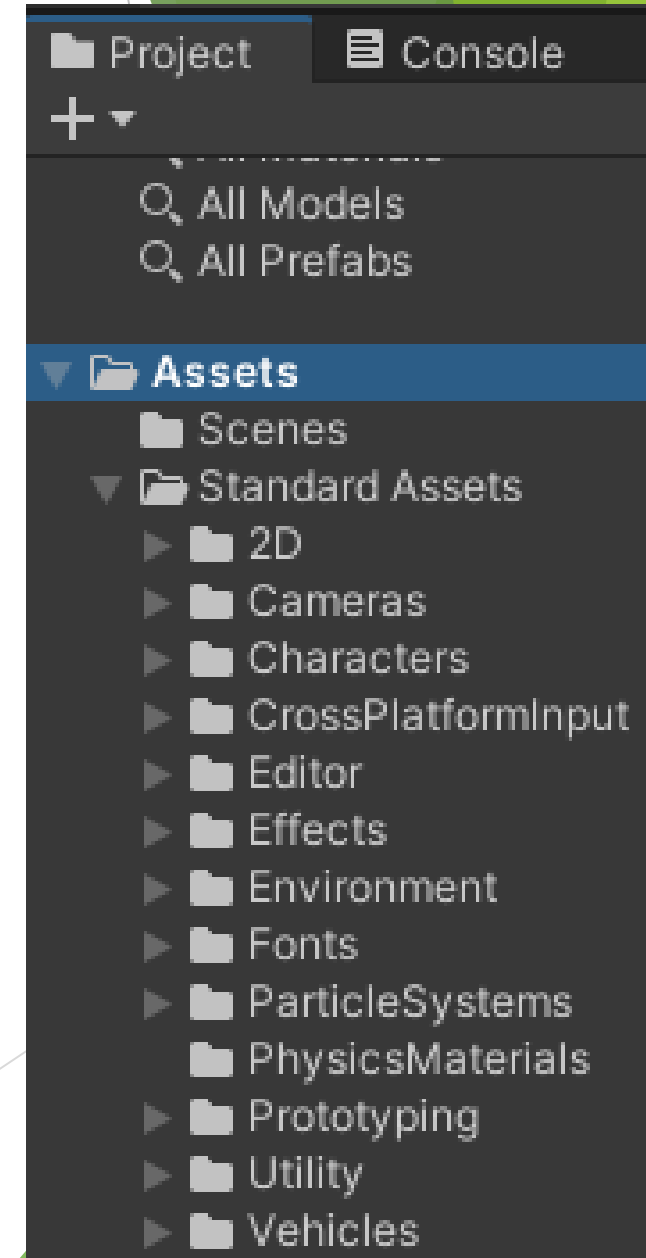
Roller Madness

- ▶ Neste novo jogo, trataremos os conceitos abaixo:
 - ▶ Importação de Pacotes de Assets
 - ▶ Uso dos Assets padrão
 - ▶ Boas práticas de organização do projeto
 - ▶ Física 3D
 - ▶ Fundamentos da Interface com o Usuário
 - ▶ Gerador de rastros
 - ▶ Sistemas de Partículas
 - ▶ Animação
 - ▶ Sistemas de Jogo : controle do player, pick-up, inimigos, placar de pontos
- ▶ Na pasta de trabalho, copie o arquivo RollerMadnessAssets.rar da pasta da nossa disciplina na rede e o descompacte. Observe que aparecerá a pasta RollerMadNessAssets, contendo a pasta Audios com arquivos wave e ogg. Há também o arquivo MSUGameScripts.unitypackage, que contém scripts C# que usaremos no jogo.



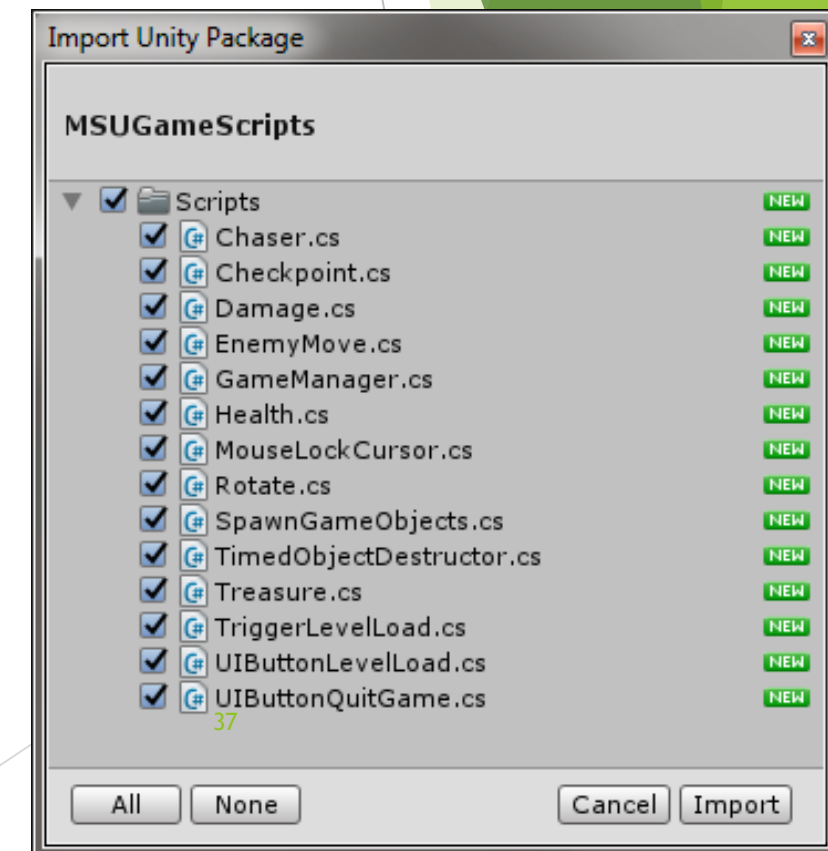
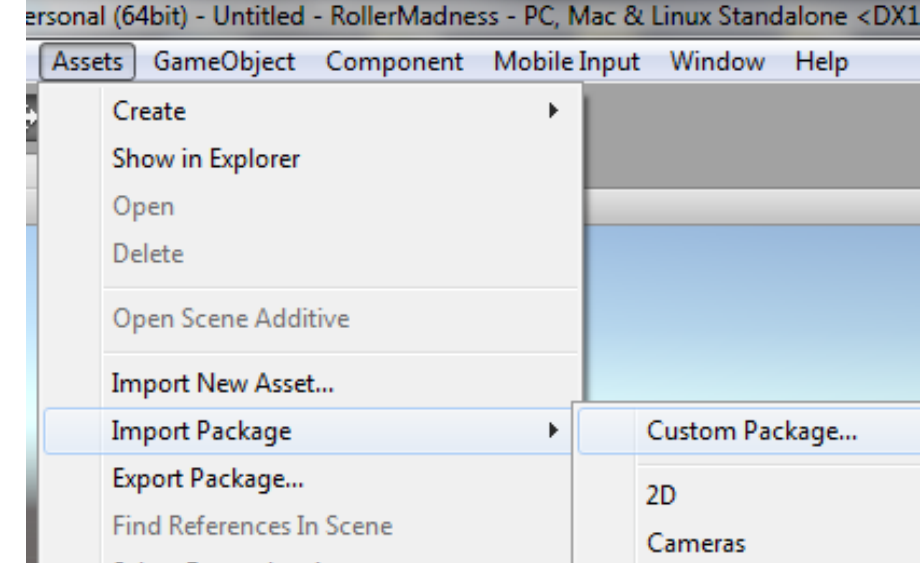
Roller Madness

- ▶ Vamos criar um novo projeto 3D, chamado RollerMadness e armazená-lo em uma pasta local do seu computador (por exemplo, Documents).
- ▶ Após criar o projeto, copie a pasta Standard Assets fornecida pelo professor para dentro da pasta Assets da janela Project. Observe os pacotes abaixo, disponíveis para nosso uso:
 - ▶ Characters
 - ▶ Cross Platform Input
 - ▶ Prototyping
 - ▶ Utility
- ▶ Como estamos importando assets, o Unity poderá demorar um pouco.



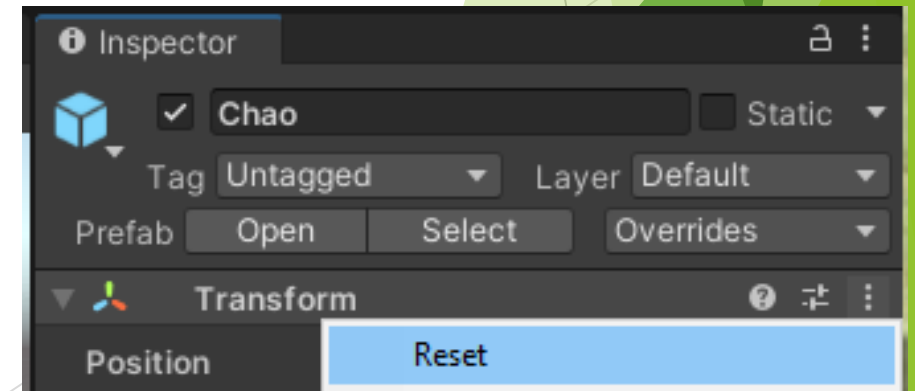
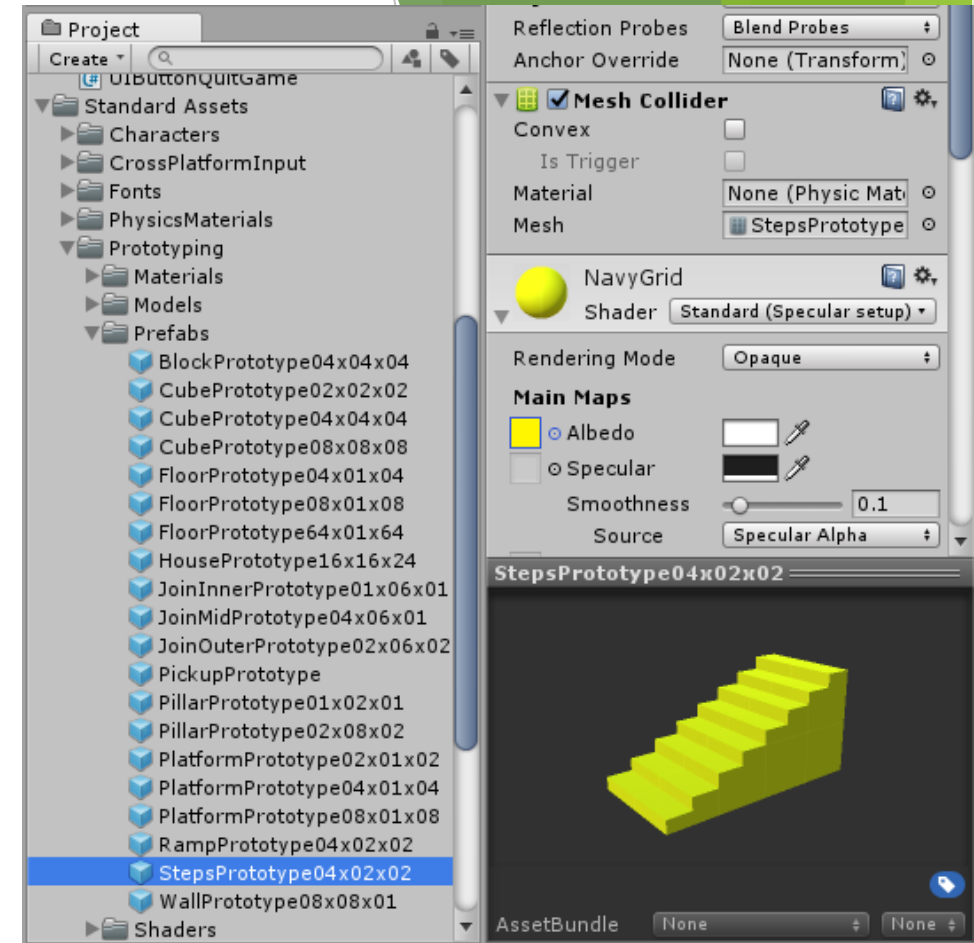
Roller Madness - Pacote de Scripts prontos

- ▶ Após isso, acesse a opção Assets do Menu e selecione a opção Import Package e Custom Package.
- ▶ Na pasta onde você descompactou RollerMadnessAssets.rar, selecione o arquivo MSUGameScripts.unitypackage.
- ▶ Clique [Import].
- ▶ Um package é um agrupamento de arquivos que pode ser usado para troca entre membros de um time de desenvolvedores de jogos ou para propósitos de arquivamento.
- ▶ No package que importamos temos scripts que controlarão os objetos de nosso novo jogo, como vemos na figura ao lado.



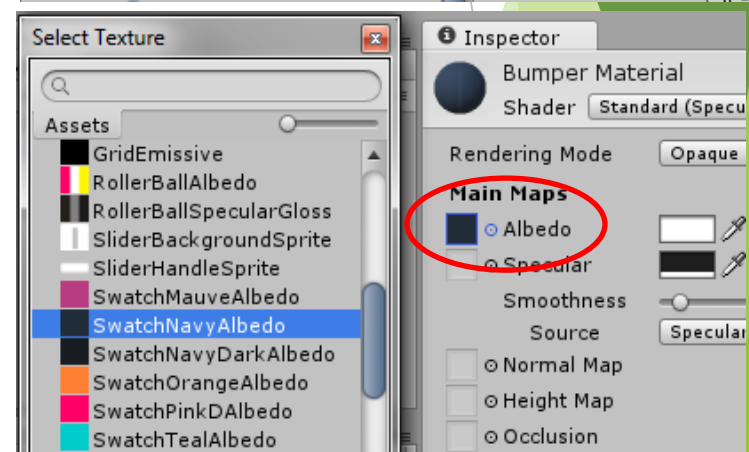
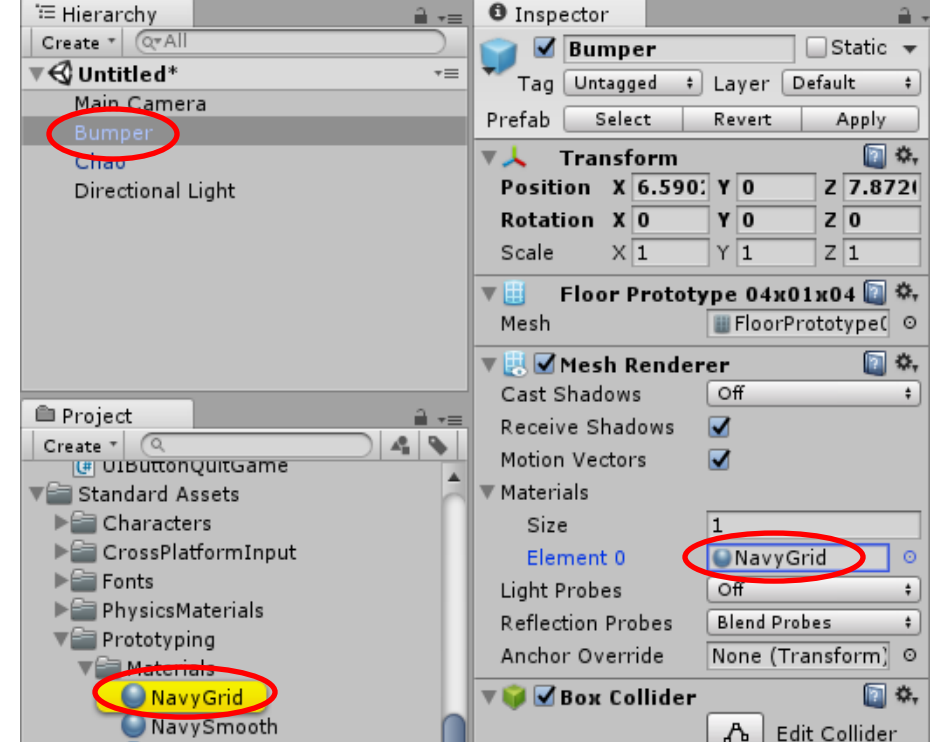
Roller Madness - Primeira fase

- ▶ Vamos criar a fase (level) base de nosso jogo, incluindo o chão e vários obstáculos.
- ▶ Vamos criar e usar objetos previamente fabricados, os prefabs.
- ▶ Usaremos vários protótipos de assets vindos com o Unity, para representar objetos que posteriormente poderão ser colocados no jogo, já com sua arte final.
- ▶ Na pasta Standard Assets existe uma pasta Prototyping | Prefabs com vários objetos prontos, como o que vemos na figura ao lado.
- ▶ Arraste para a cena o protótipo Floor, com 64 x 1 metros.
- ▶ Aparecerá um grande objeto na cena e, na Hierarquia, mude seu nome para Chao. Posicione-o na origem do mundo digitando (0,0,0) em Transform | Position ou selecione o botão  no Transform do Inspetor e clique em Reset.
- ▶ Modifique a escala do Chao para (0.5, 1, 0.5).



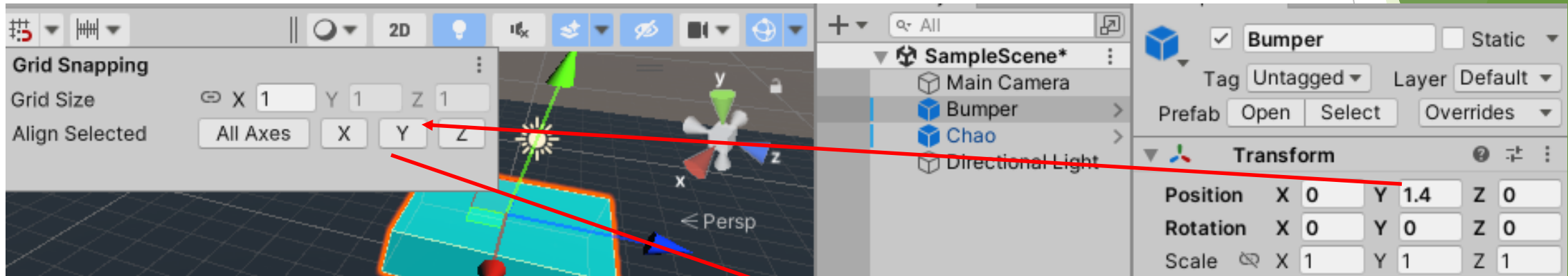
Roller Madness - Primeira fase

- ▶ Vamos colocar mais alguns assets como obstáculos na cena e modificá-los conforme nossas necessidades.
- ▶ Arraste para a cena o objeto FloorPrototype04x01x04 da pasta Prefabs. Ele ficará um pouco escondido, pois ainda tem o mesmo material cor e espessura que o Chao. Pressione a tecla F para dar zoom nesse objeto e mude seu nome para Bumper (ou para-choque).
- ▶ Na figura ao lado vemos o material no Mesh Renderer (NavyGrid). Selecione esse material na pasta Materials e o duplique (com Ctrl-D). Renomeie a cópia para Bumper Material e o selecione.
- ▶ Se você clicar no círculo ao lado da propriedade Albedo, aparecerão os materiais pré-feitos. Selecione o Swatch Teal Albedo ao invés do Swatch Navy Albedo. O Material mudará de cor.
- ▶ Arraste esse Material para o Bumper na Hierarquia.
- ▶ Faça reset no Bumper e o suba para (0,1,0).

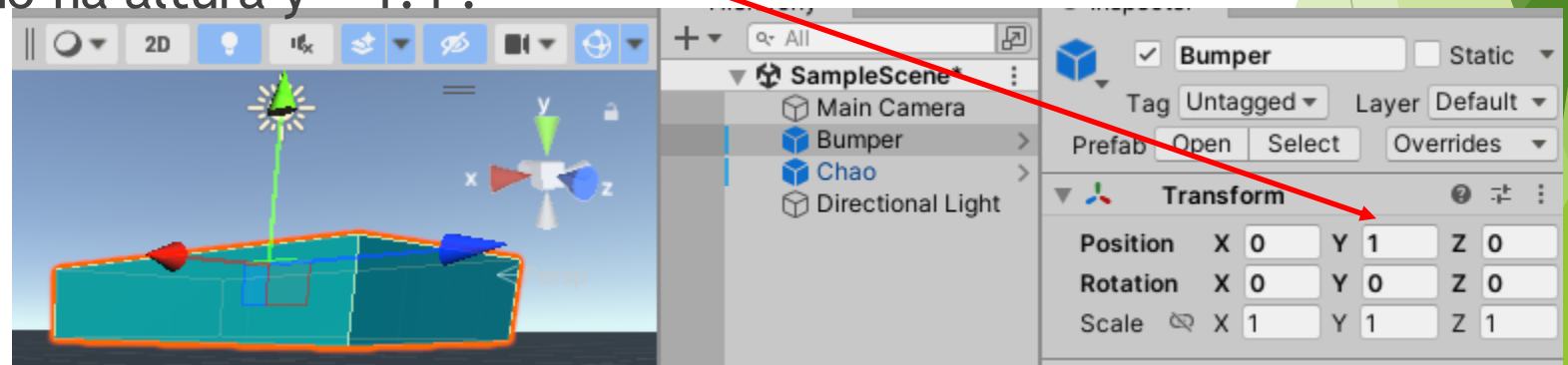


Roller Madness - Primeira fase - Snap Settings

- ▶ Para que os objetos sejam posicionados com maior controle, use as Snap Settings.
- ▶ Por exemplo, você pode mover o Bumper para cima usando a ferramenta de translação e clicar nos botões de Snap Settings. Se  em [Y], o Bumper será movido exatamente para a altura mais próxima de um múltiplo de 1 a partir de onde ele está, no eixo y.



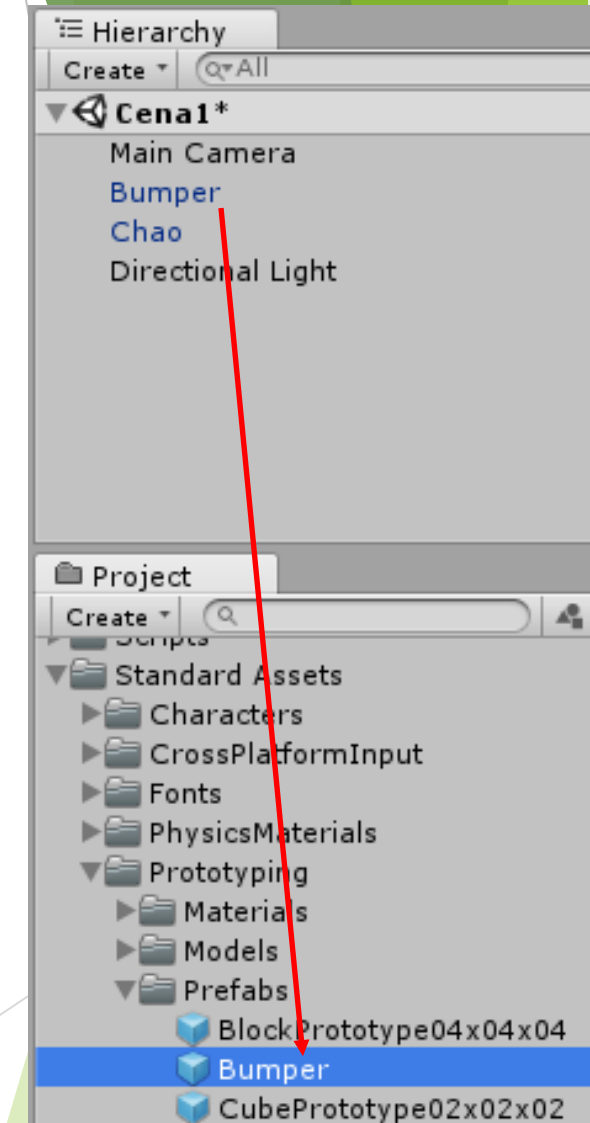
- ▶ Pressionando [Y] na janela Snap settings, o Bumper será posicionado exatamente em Y = 1, após ter sido colocado na altura y = 1.4.



- ▶ Pressionando a tecla [Ctrl] e movendo o objeto, fará com ele mude de posição de acordo com os valores de Move X, Move Y e Move Z do Snap Settings.

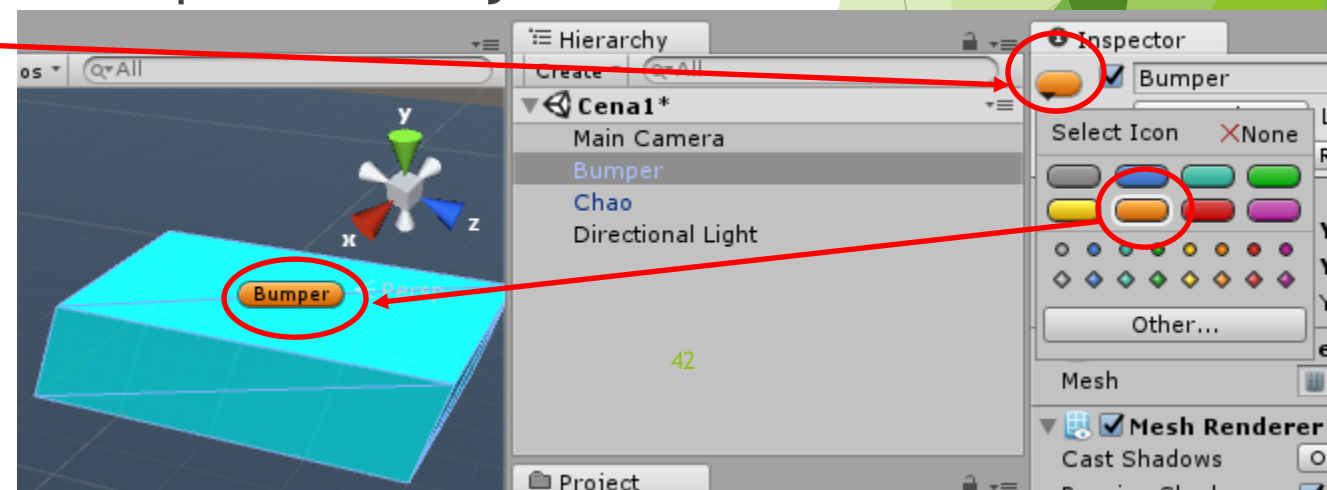
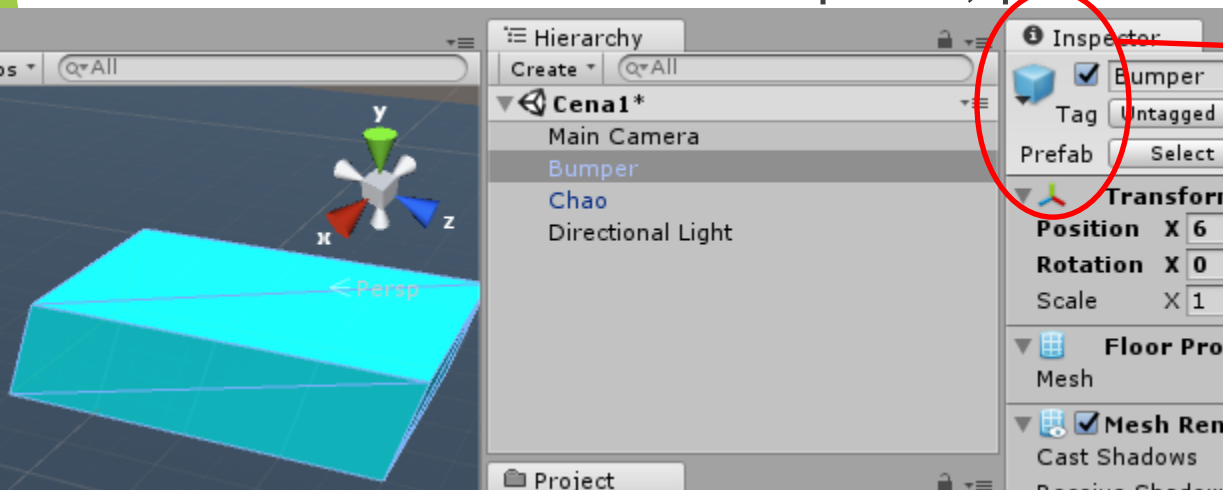
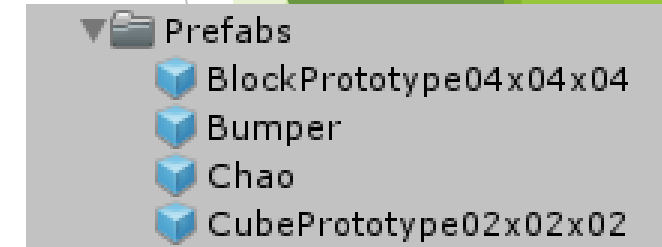
Roller Madness - Primeira fase - Criando Prefabs

- ▶ Mova o Bumper do centro para alguma outra posição um pouco afastada, mantendo-o em contato com o Chao.
- ▶ Queremos ter vários Bumpers em nosso mundo.
- ▶ Toda vez que planejarmos ter vários gameObjects que sejam muito idênticos nós devemos criar um Prefab a partir do objeto-modelo original.
- ▶ Para tornar nosso Bumper em um prefab, arraste-o da Hierarquia para a pasta Standard Assets | Prototyping | Prefabs.
- ▶ Observe que um Asset azulado apareceu nessa pasta, com o nome Bumper.



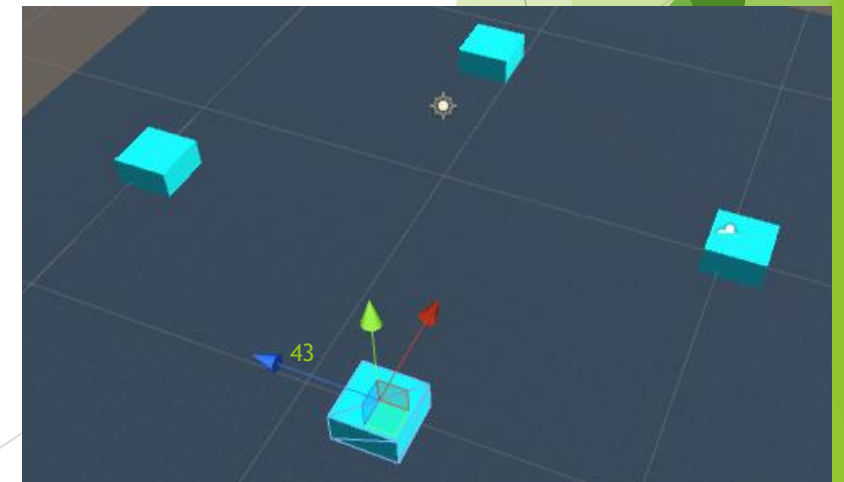
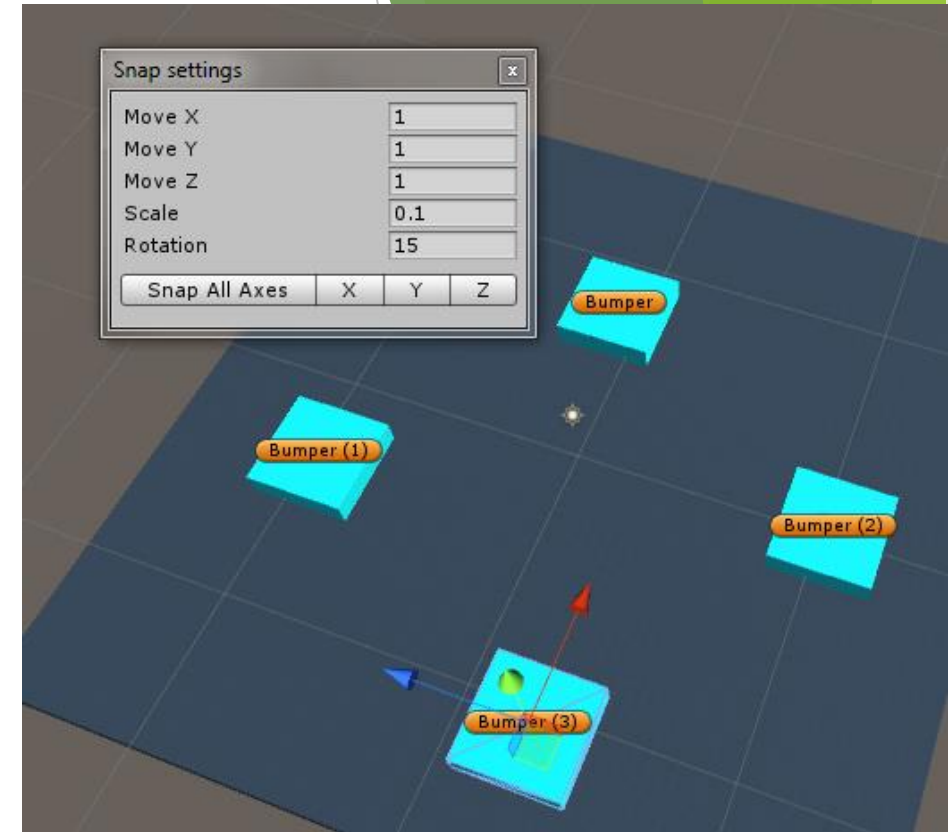
Roller Madness - Primeira fase - Prefabs

- ▶ Criemos, também, um Prefab para o Chao, mas agora de outra maneira:
- ▶ Clique no título da pasta Prefabs que usamos acima, e selecione o botão [+] abaixo da palavra Project e, em seguida, selecione [Prefab]. Aparecerá uma nova entrada na lista de prefabs. Chame-a de Chao.
- ▶ A cor cinza indica que nenhum objeto está associado ao prefab que, portanto, está vazio. A cor azul, como do prefab Bumper, indica que já há um objeto associado.
- ▶ Arraste o objeto Chao da Hierarquia para o prefab Chao, acinzentado, e ele passará a ficar azulado, indicando que um gameObject foi associado a ele e poderá ser usado como modelo para criação de outros “Chao”.
- ▶ Clicando na caixa azul no Inspector, poderá indicar o prefab do Objeto:



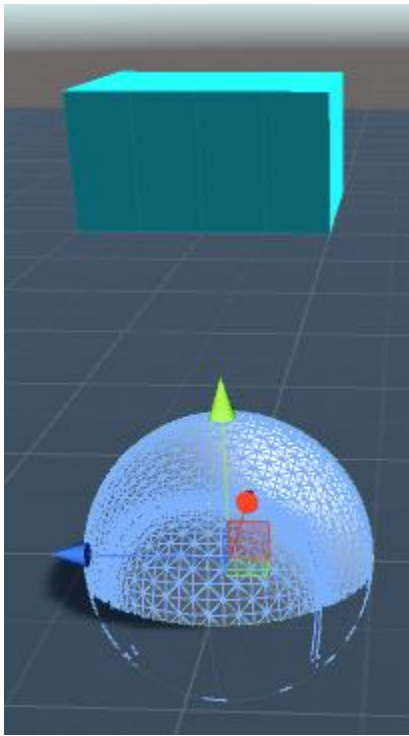
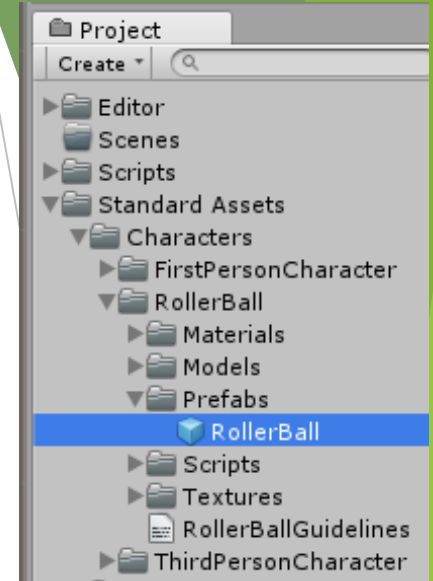
Roller Madness - Primeira fase - Prefabs

- ▶ Vamos arrastar mais alguns prefabs Bumper para a cena.
- ▶ Para posicioná-los com precisão, pode usar os Snap Settings como, por exemplo, Snap All Axes em múltiplos de 1, formando um padrão como o da figura:
- ▶ Em seguida, selecione o prefab Bumper na janela Projects para mudarmos sua escala, pois eles estão um pouco superdimensionados.
- ▶ Mude a escala em X e em Z para 0.5 metros, deixando a escala em Y valendo 1 metro.
- ▶ Esse objeto é um prefab. Caso você se arrependa e considere as alterações incorretas, basta pressionar as teclas Ctrl-Z.
- ▶ Caso considere que as alterações estão corretas e deseja que todas as instâncias do prefab passem a ter essa escala, saia da edição clicando em outro objeto.
- ▶ As alterações serão propagadas aos Bumpers da cena.

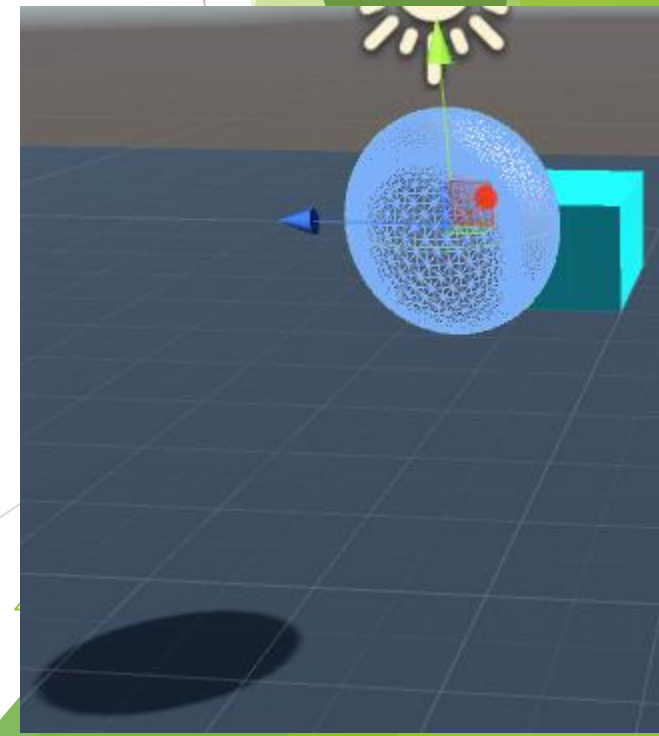


Roller Madness - Primeira fase - Player Character

- ▶ Para nosso Player Character (PC) usaremos um Asset Padrão.
- ▶ Na janela Project, abra as pastas Standard Assets, Characters, RollerBall e Prefabs.
- ▶ Arraste para a cena o prefab RollerBall.
- ▶ No Inspector, clique nos três pontinhos de Transform e, logo depois, em Reset para zerar o transform da esfera que aparecerá na cena, de forma que ela fique no centro do nosso mundo de jogo.

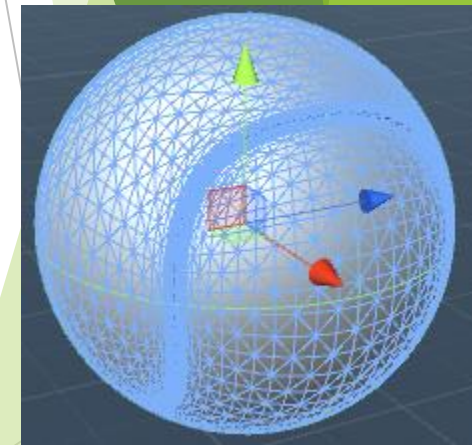


- ▶ Mude a posição Y da esfera para 2, de maneira que ela fique suspensa acima do chão.
- ▶ No inspector dessa esfera, observe os scripts já prontos : Ball e Ball User Control. Eles recebem os inputs do jogador e, com isso, movem e dão vida à bola.
- ▶ Salve e execute seu jogo, usando seu controlador de jogo ou as teclas de setas para mover a bola (espaço a faz pular).



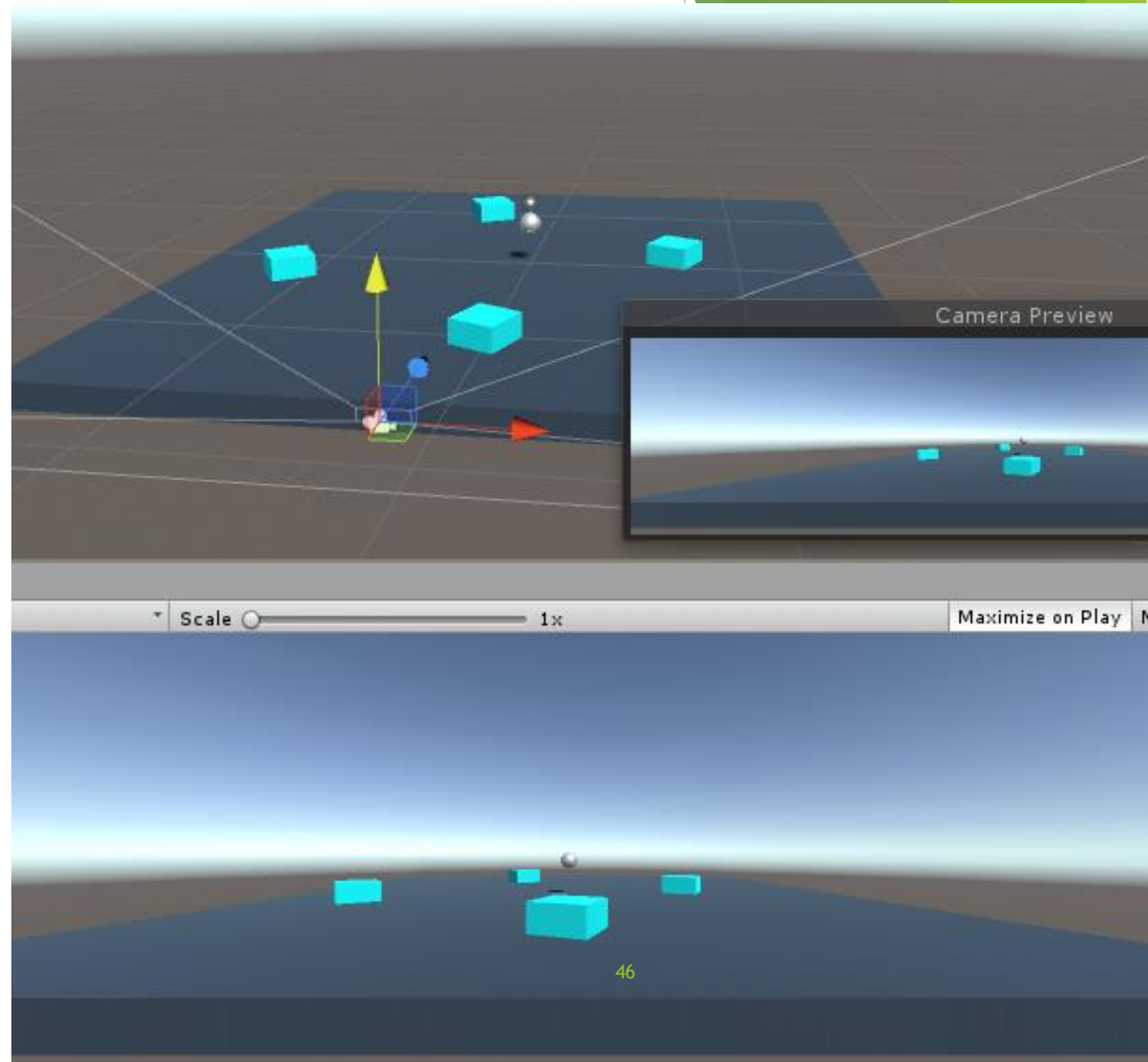
Roller Madness - Primeira fase - Câmera

- ▶ Nossos próximos objetivos são:
 - ▶ Posicionar a câmera
 - ▶ Fazer a câmera seguir nosso player character (a bola)
 - ▶ Modificar o fundo da tela visto pela câmera
 - ▶ Adicionar música à câmera
- ▶ Vamos posicionar a câmera. Clique na bola e pressione a tecla F para colocar esse objeto no foco.
- ▶ Observe o indicador de eixos da bola. O eixo z é o azul e sua seta aponta na direção positiva.
- ▶ O controlador funciona de maneira que, quando você pressiona o comando para ir para a frente, ele aplica uma força na direção positiva e a bola é empurrada na direção de z positivo.
- ▶ Assim, em essência desejamos que a câmera fique olhando na direção do eixo Z.
- ▶ Posicione a câmera um pouco atrás da bola, olhando para baixo, de forma



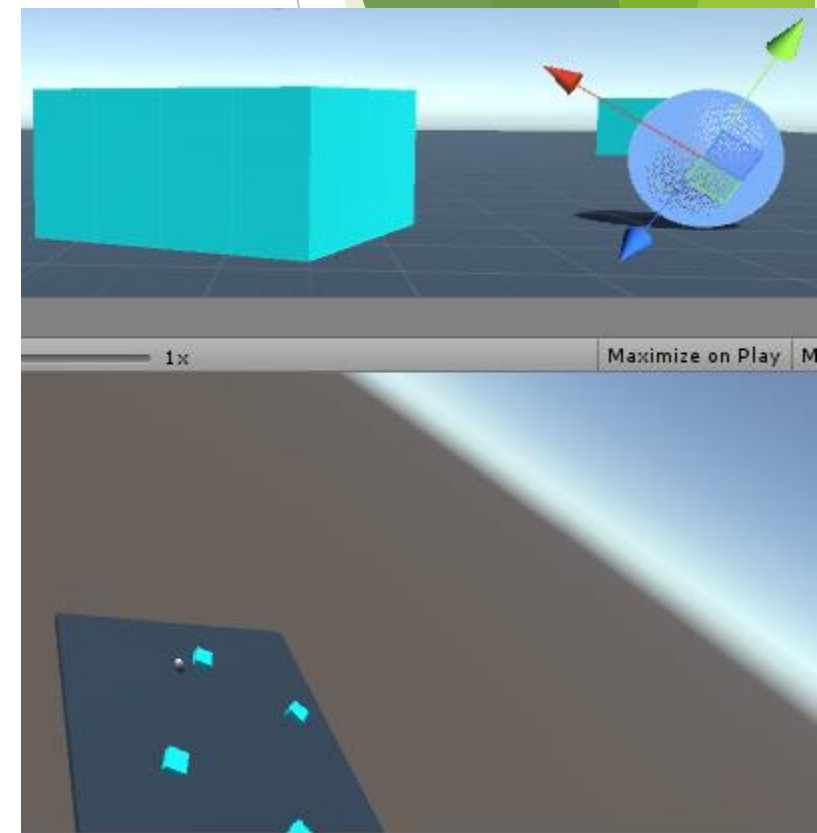
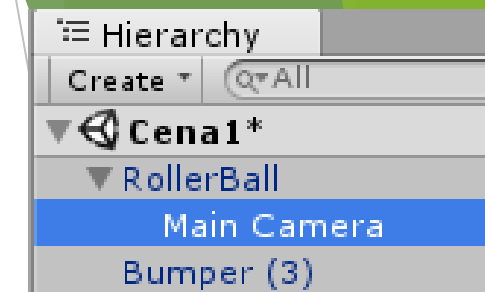
Roller Madness - Primeira fase - Câmera

- ▶ Procure mover a câmera um pouco para trás e para o lado direito, de forma que a cena possa ser visualizada de forma semelhante à da figura ao lado:
- ▶ Agora que temos a visualização da cena de aspecto próximo ao que desejamos para o jogo, precisamos mudar a própria câmera.
- ▶ Selecione a câmera na hierarquia e ative a opção `GameObjects | Align With View`. Agora, a visualização [Game] ficará igual à da câmera na scene.
- ▶ Se executarmos o jogo, teremos essa visualização no modo de jogo, mas a câmera não segue a bola.
- ▶ Como fizemos no primeiro projeto, vamos subordinar a câmera à bola, arrastando-a na hierarquia para dentro do objeto Rollerball, criando um relacionamento pai-filho para os dois e fazendo a câmera seguir a bola conforme esta se move.



Roller Madness - Primeira fase - Câmera

- ▶ Após fazer isso, salve e execute seu jogo.
- ▶ Observe que a câmera fica fora de controle. Isso ocorre porque ela está girando junto com a bola, que domina seus movimentos, pois a câmera está agora subordinada ao sistema de coordenadas da bola, o qual está girando continuamente conforme a bola gira.
- ▶ Pare a execução e recoloque a câmera fora da bola.
- ▶ Selecione a câmera e, em Standard Assets, abra a pasta Utility e inclua na câmera o script Smooth Follow. Configure a variável Target para apontar a RollerBall. Mude Distance para 6 e deixe Height valendo 5.
- ▶ Se você executar o jogo agora, verá que a câmera segue a RollerBall de maneira correta, pois está fora de seu sistema de coordenadas e segue a bola através do script Smooth Follow.
- ▶ Com a câmera selecionada, observe no Inspector a propriedade SkyBox. Mude Clear Flags para Solid Color e selecione uma cor vermelha escura (ou outra qualquer conforme seu gosto). Por exemplo, a cor (115, 0, 0, 0).
- ▶ A Câmera tem um componente Audio Listener e é um bom lugar para colocar sons, que não variarão com a distância à câmera. Clique em [Add Component], selecione Audio e Audio Source. Abra a janela Select AudioClip (círculo do lado direito da propriedade AudioClip e selecione a música Background. Mude **volume** para 0.25 e **loop** true. Execute o jogo.



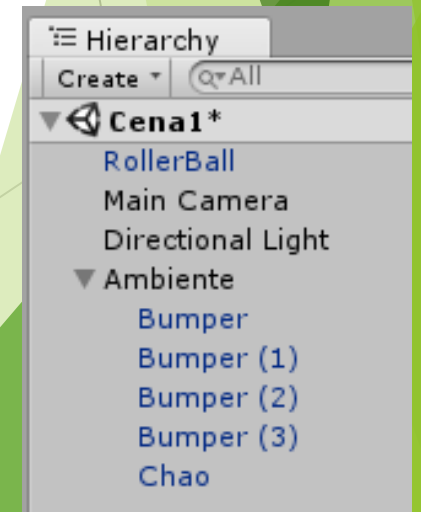
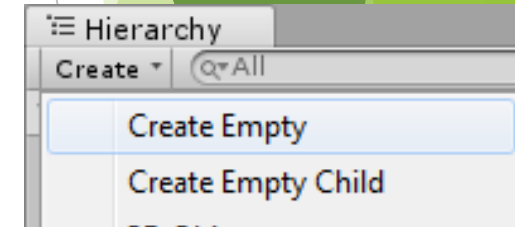
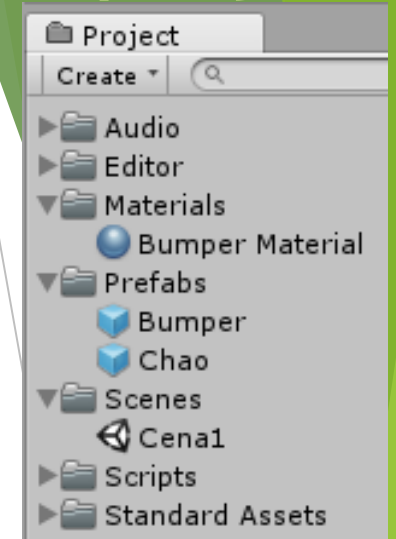
Roller Madness - Primeira fase - Organização do projeto

► Organizando Projects

- Crie as seguintes pastas no Projeto : Scenes, Materials e Prefabs
- Arraste para Materials o componente Bumper Material
- Arraste para Prefabs os componentes Bumper e Chao
- Arraste para Scenes o componente da nossa cena.

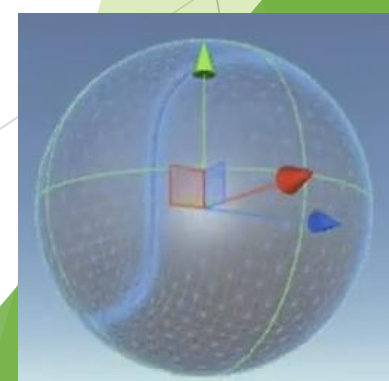
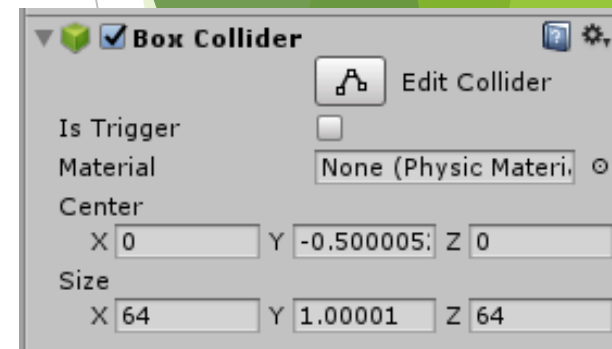
► Organizando Hierarchy

- Do ponto de vista organizacional, uma relação pai/filho entre Game Objects é parecida a uma pasta
- Podemos criar Objetos de jogo vazios (empty Game Objects) para atuar como pastas e, assim, organizar a hierarquia
- Em Hierarchy, selecione [+] e [Create Empty], mude seu nome para Ambiente. Selecione a Engrenagem no Inspector e clique em Reset, para que o objeto fique no centro das coordenadas do mundo.
- Arraste para dentro do Ambiente os 4 bumpers e o Chao.
- Assim teremos nossos objetos de jogo organizados e agrupados.



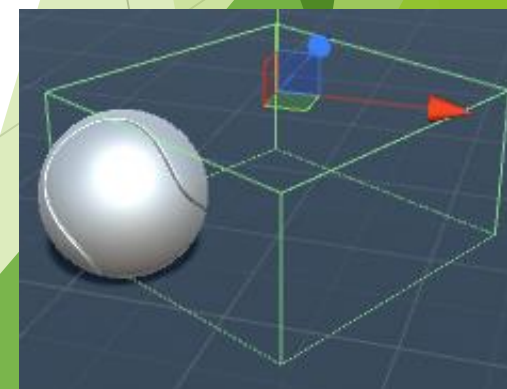
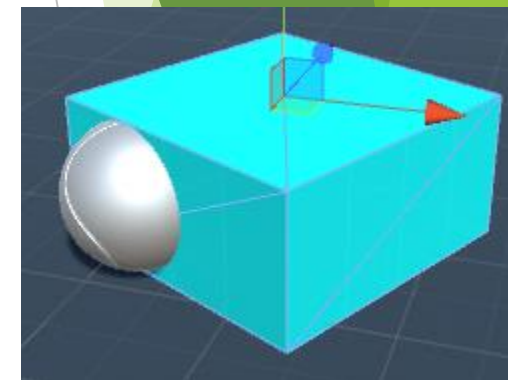
Roller Madness - Primeira fase - Física

- ▶ Física é uma parte importante de muitos jogos. Unity tem duas engines de física:
 - ▶ Física 3D = PhysX Engine → é a que usaremos em RollerMadness
 - ▶ Física 2D = Box2D Physics Engine
- ▶ Nossos objetivos agora são:
 - ▶ Entender e modificar Colliders
 - ▶ Entender e aplicar RigidBodies
 - ▶ Fazer uso de Materiais Físicos
- ▶ Na cena, clique em Chao e observe que ele tem um Box Collider.
- ▶ Clique na RollerBall e observe que ela possui um Sphere Collider, que é parte do Prefab original que obtivemos dos Assets Padrão. Observe uma esfera com linhas verdes envolvendo a RollerBall. Essa é o Sphere Collider.
- ▶ Os bumpers também possuem Box Colliders que podem ser vistos quando clicamos um dos bumpers, como uma caixa de linhas verdes envolvendo⁴⁹ o objeto.



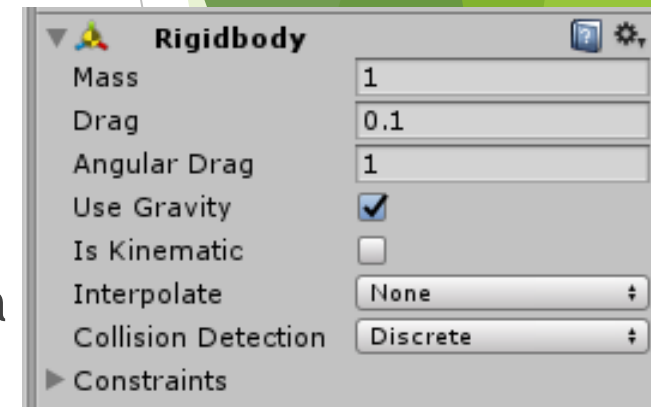
Roller Madness - Primeira fase - Física

- ▶ O componente Collider é usado para detectar colisões entre game objects.
- ▶ Você pode modificar o tamanho do collider, pressionando o botão [Edit Collider] do Inspector e movendo os pequenos quadrados verdes que aparecerão ou, então, usando a propriedade Size do Collider no Inspector.
- ▶ Por default, Unity não permite que Colliders se sobreponham.
- ▶ Portanto, quando você faz a Rollerball bater em um bumper, ela não o atravessa.
- ▶ Se você ligar a opção Is Trigger de um Collider, ele deixará de usar a detecção de colisão. Fazendo isso num Bumper, ele permitirá que a RollerBall passe dentro dele, pois não mais impedirá a sobreposição, embora ainda detecte colisões. Inclusive podemos criar um evento OnTriggerer em um script para tratar essa situação.
- ▶ Por exemplo, o bumper poderia ser um objeto invisível na cena mas que, quando tocado pelo PC, executaria outras ações, como abrir uma porta, ou mudar a iluminação da cena. Para tornar um objeto invisível, desative seu Mesh Renderer no Inspector.



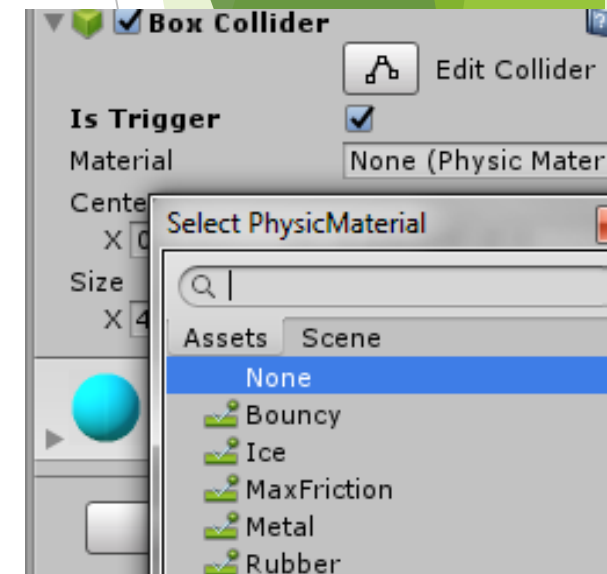
Roller Madness - Primeira fase - Física

- ▶ Se você colocar Is Trigger true para o Chao, ao executar o jogo verá que a bola passa por ele, caindo sob a ação da gravidade. No entanto, os bumpers não passaram pelo chão e caíram eternamente, ficaram onde estavam compondo a cena.
- ▶ Por que isso aconteceu? Observe a Rollerball e note que ela também possui um componente chamado Rigidbody.
- ▶ Rigidbodies permitem que colliders sejam afetados por Física.
- ▶ Assim, como a Rollerball possui um Rigidbody e uma e suas opções é Use Gravity, que está ligada, de forma que a Rollerball recebe uma força para baixo.
- ▶ Os bumpers não tem RigidBodies. É interessante colocar RigidBodies em componentes que possuam colliders e estejam se movendo de alguma maneira, como a Rollerball.
- ▶ Outra opção do Rigidbody é Is Kinematic. Quando ligada, essa opção faz com que o objeto não seja afetado pela Física, o que pode ser útil se você desejar mover o objeto pelo seu **transform**.
- ▶ Rollerball será afetada pela Física, portanto deixaremos Is Kinematic desligado. Desligue também Is Trigger do Box Collider do Chao, para que Rollerball não passe por ele.



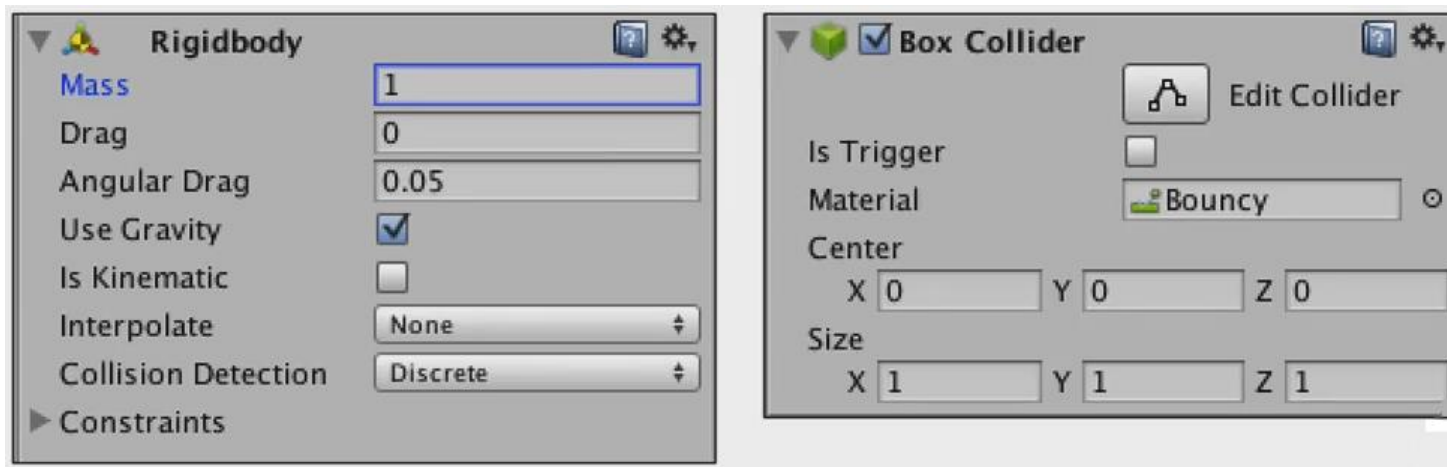
Roller Madness - Primeira fase - Física

- ▶ No Box Collider da Rollerball, observe que ela possui uma propriedade Material igual a Rubber. Quando essa bola bate em algum objeto, ela age como uma bola de borracha e volta para trás.
- ▶ Há vários tipos de Material, você pode testar o jogo com diversos e ver qual fica mais adequado à nossa bola.
- ▶ Selecione um dos bumpers e observe que ele não tem um material associado ao box collider. Clique no círculo à direita e selecione o material Bouncy. Execute o jogo e veja como a bola é rebatida quando bate no Bumper modificado.
- ▶ Pressione [Apply] no Inspector desse bumper para que todos os prefabs sejam atualizados com esse material.



Revisão de Física no Unity

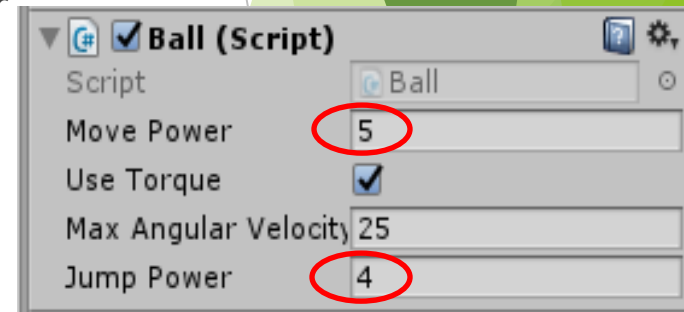
- Física é criada no Unity através de dois componentes aplicados aos game Objects:



- Rigidbody habilita física num game object, incluindo gravidade.
- Você pode especificar propriedades físicas do objeto, como massa.
- Se um Collider é ligado ao game object, o Rigidbody também poderá detectar colisões entre o objeto atual e outros.
- O Collider, mesmo sendo invisível para a câmera, determina o formato do game object e como ele interage com a física.
- Se Is Trigger for verdadeiro, os objetos podem passar através um do outro, mas o jogo pode detectar colisões (através de scripts) e atuar de acordo.
- Colliders podem ter um material físico ligado a eles para modificar suas características durante colisões, como rebatimento ou deslizamento.

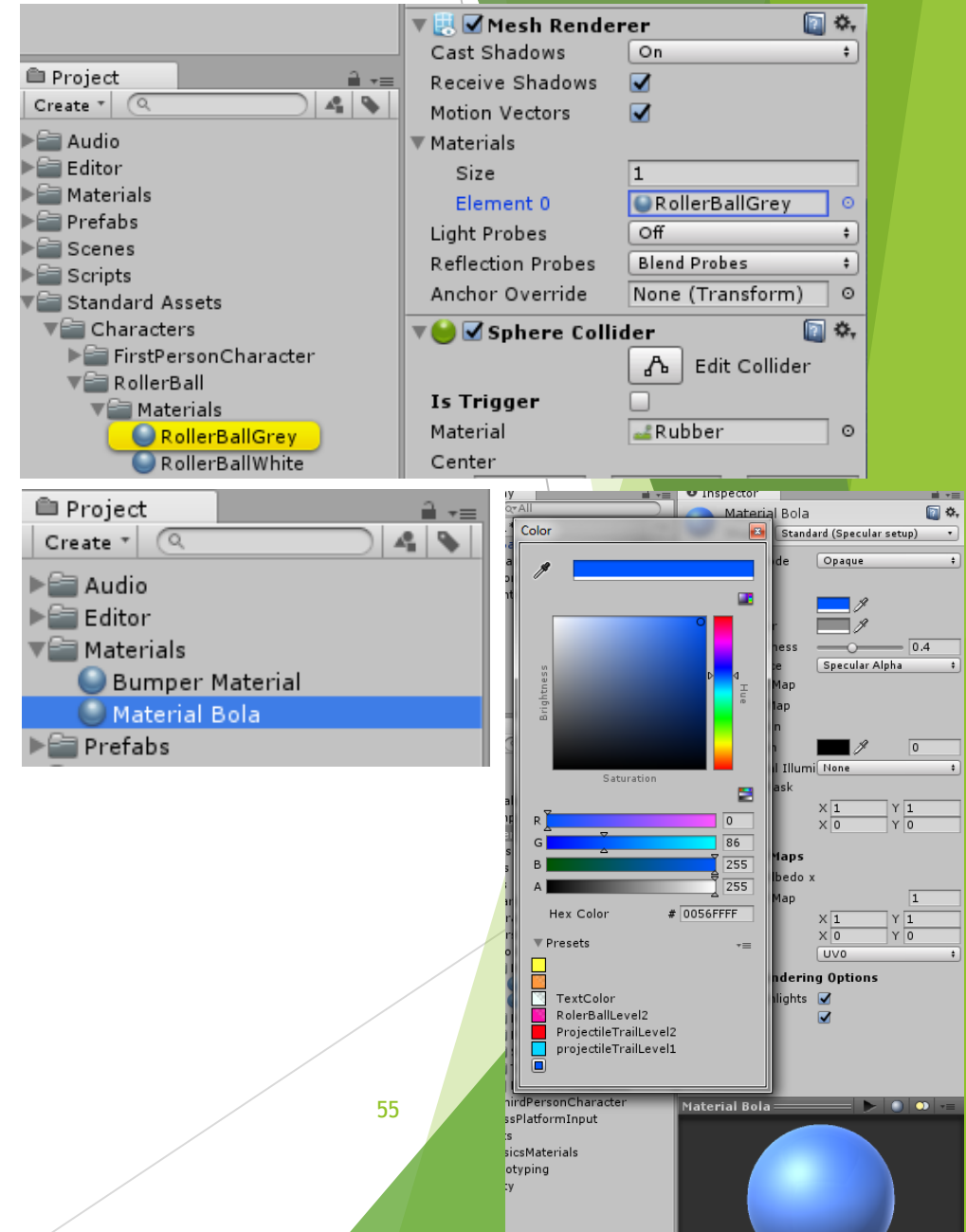
Roller Madness - Controle do Jogador

- ▶ Vamos aprimorar o movimento e a aparência de nosso jogador (a bola)
- ▶ Modificaremos a velocidade da bola, seu material e colocaremos um rastro que a segue
- ▶ O objeto RollerBall é um Asset Padrão, com características “físicas” e scripts prontos que facilitaram e tornaram mais veloz o desenvolvimento do jogo até o momento.
- ▶ Os scripts estão codificados para permitir a leitura dos valores de deslocamento pelo teclado (teclas de setas, WASD e controle de jogo), possibilitando várias experiências de jogabilidade.
- ▶ Observe no Inspector o script Ball. Ele possui uma variável chamada MovePower que define a quantidade de força aplicada ao movimento da bola.
- ▶ JumpPower indica a quantidade de força aplicada quando a bola pula. Se você não quiser que a bola do jogo pule, basta mudar esse valor para 0.
- ▶ Mude MovePower para 100 e JumpPower para 1, para que o movimento da bola fique mais controlável.
- ▶ UseTorque permite à bola derrapar enquanto a movemos. De certa maneira isso ajuda a controlá-la. Deixar UseTorque falso faz com que a bola role sem atrito e sua velocidade aumente muito, ficando fora de controle.



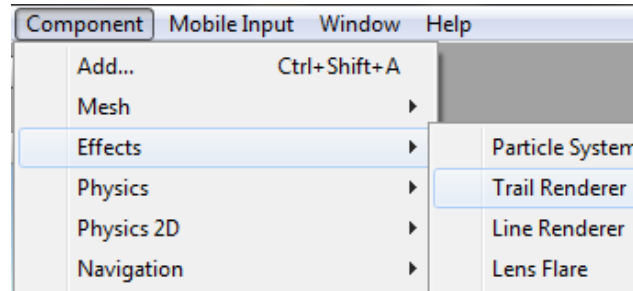
Roller Madness - Aparência do Jogador

- ▶ Clique na RollerBall na Hierarquia. Observe no Inspector a propriedade Mesh Renderer e, dentro dela, Materials. Clique na indicação RollerBallGrey e observe que o material é cinzento. Observe também que esse material é exibido na janela Project, dentro dos materiais dos Assets Padrão (pasta Characters | Materials).
- ▶ Clique em RollerBallGrey na janela Project e tecle [Ctrl-D] para duplicar esse material. Mude seu nome de RollerBallGrey 1 para Material Bola.
- ▶ Arraste Material Bola para nossa pasta Materials, fora dos Standard Assets.
- ▶ No inspector desse material, clique no Color Pick do Albedo e digite, em Hex Color, 0056FFFF (ou outra cor que goste).
- ▶ Em seguida, clique no Color Pick de Specular, e digite 00C0FFFF no Hex Color, de forma que o material fique espelhado com cor Aqua Blue.
- ▶ Arraste esse material para o objeto RollerBall na Hierarquia, para que ele assuma a nova aparência.

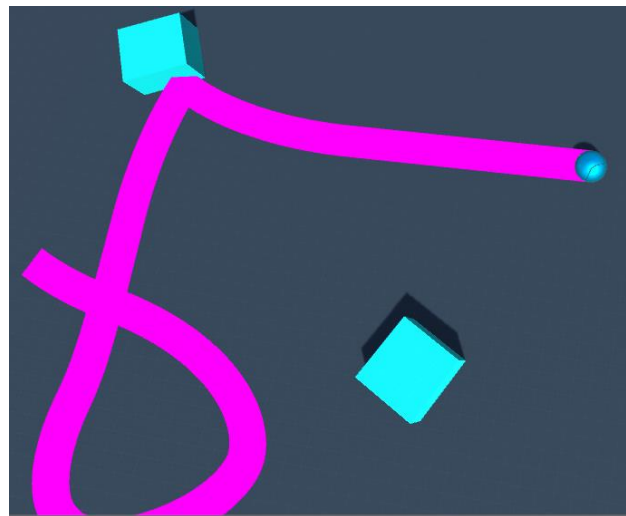


Roller Madness - Aparência do Jogador

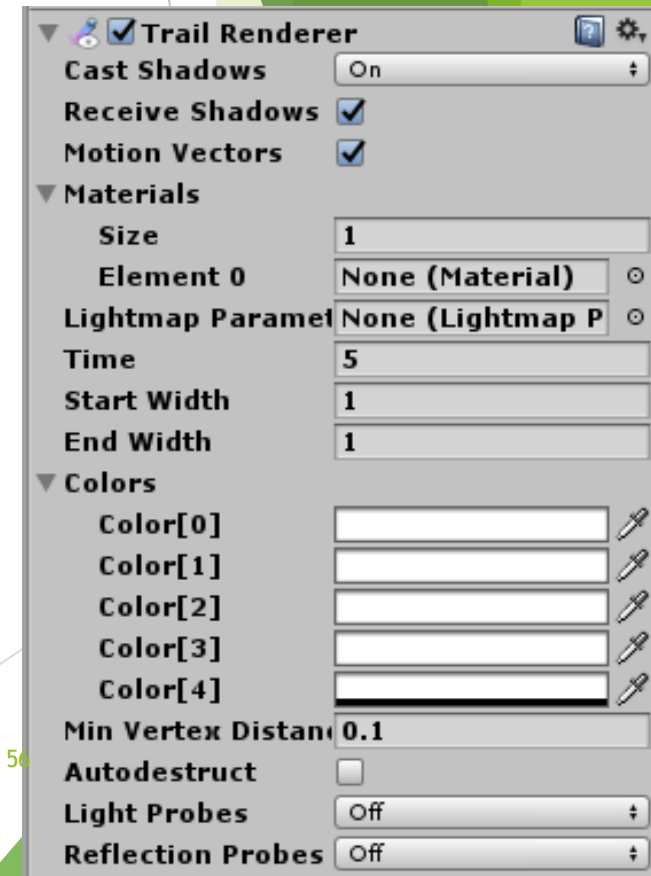
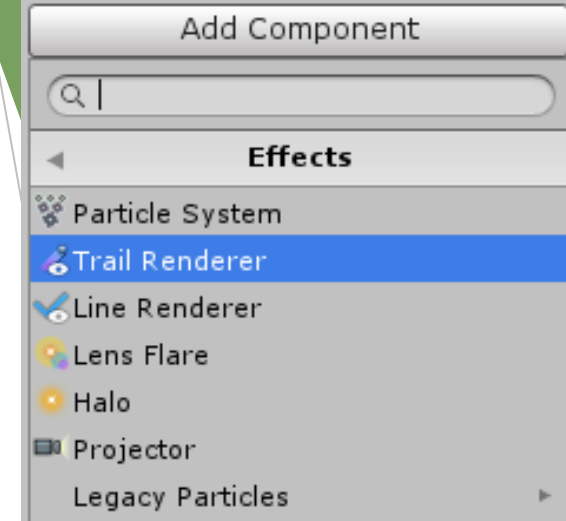
- ▶ Vamos agora adicionar um Trail Renderer (gerador de rastros) à bola.
- ▶ Com a bola selecionada na Hierarquia, clicamos em [Add Component] no Inspector, em seguida em Effects e escolhemos Trail Renderer. Ou fazemos isso pelo menu Component.



- ▶ Um Trail Renderer basicamente gera um rastro atrás de um game object conforme este se move na cena.

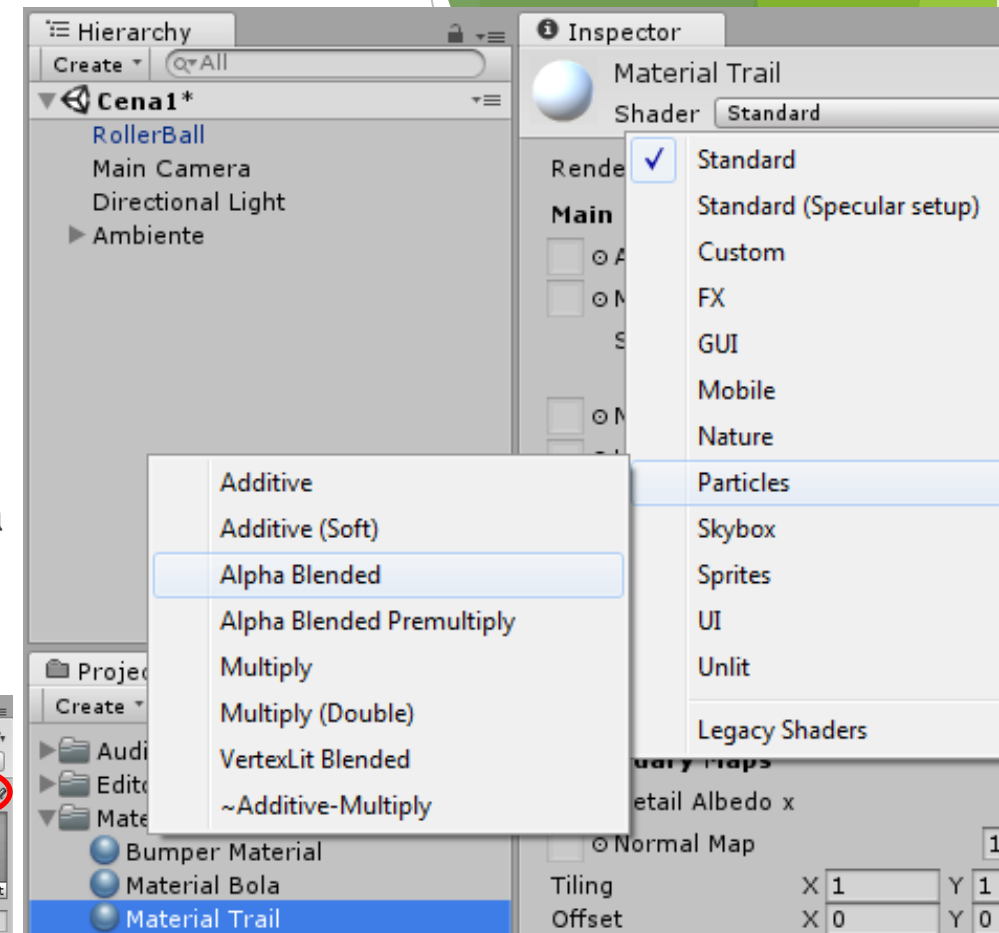
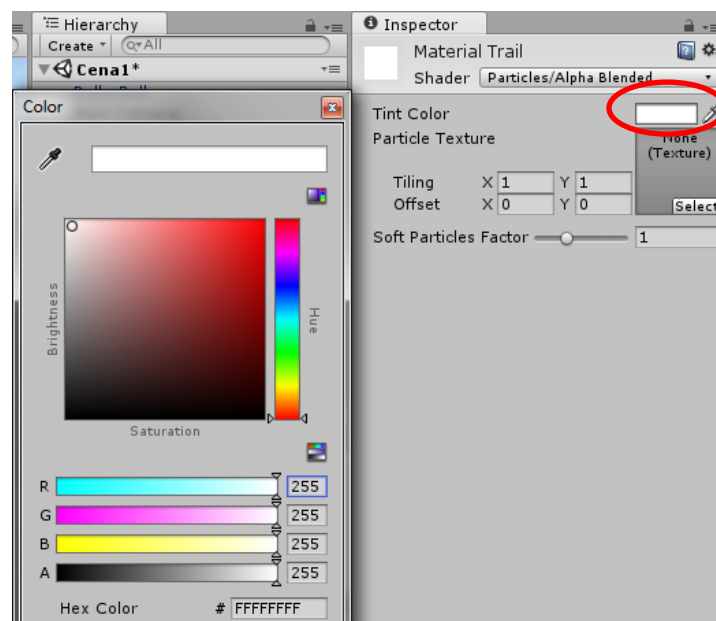


- ▶ Observe quantas propriedades esse componente possui. Ainda não o configuramos para ficar com uma boa aparência, mas vamos fazê-lo.



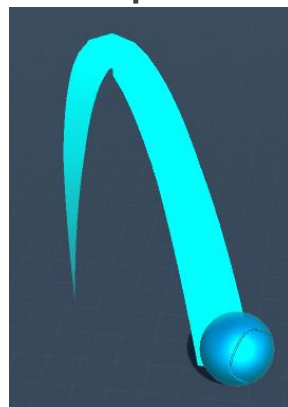
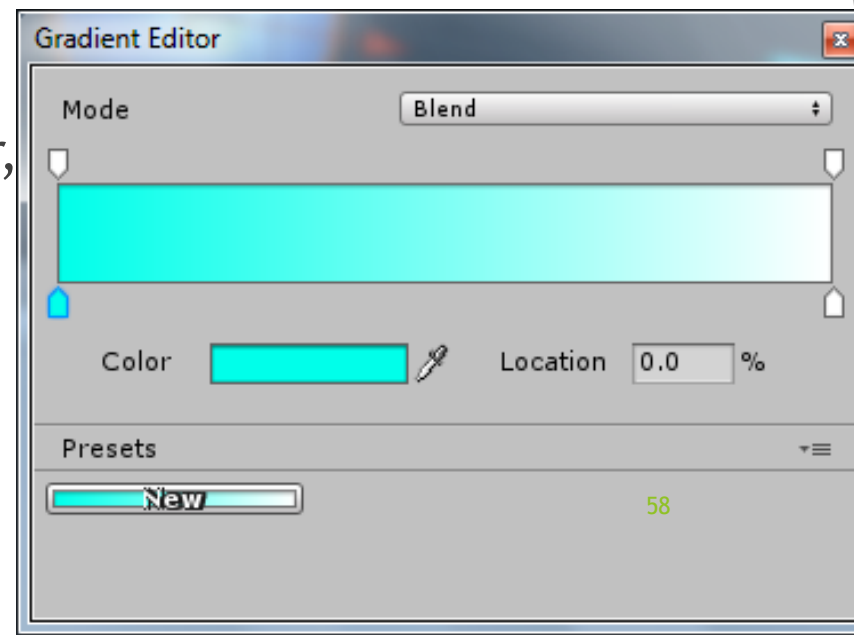
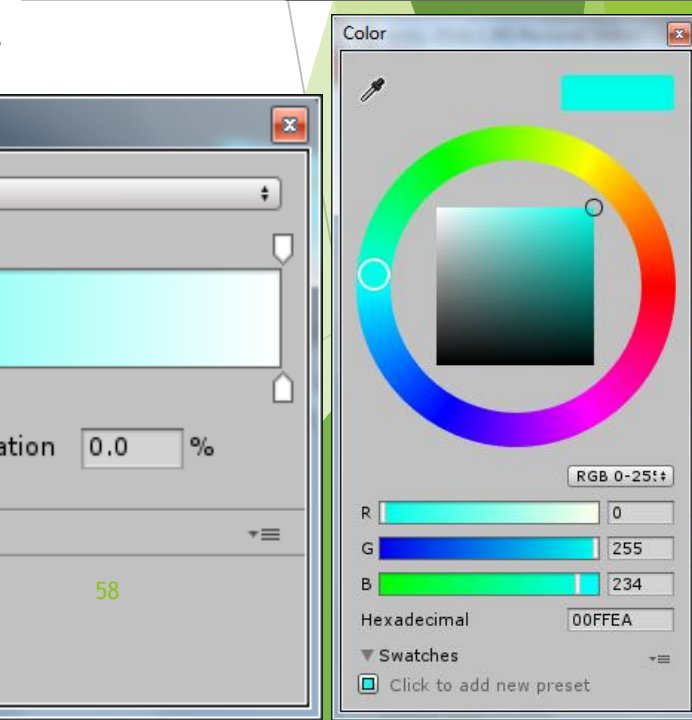
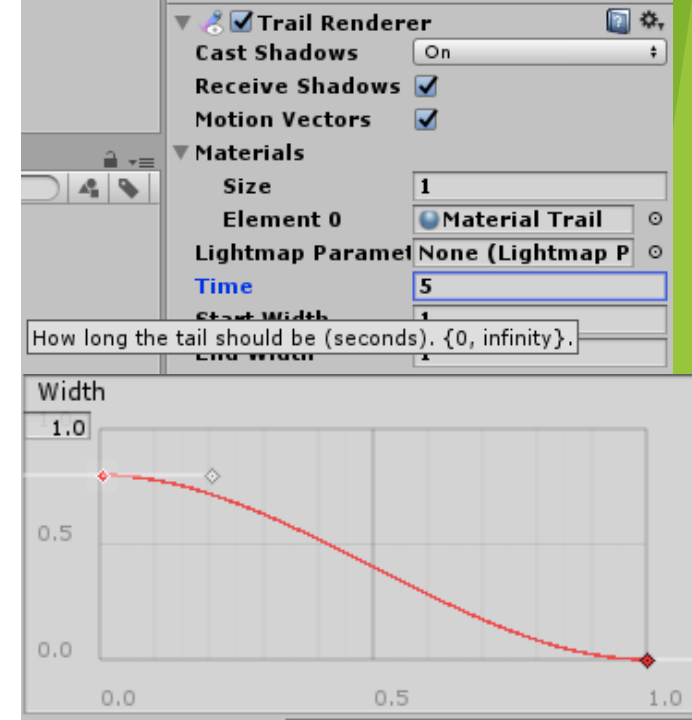
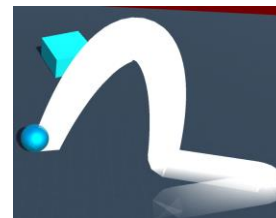
Roller Madness - Aparência do Jogador

- ▶ Uma das propriedades mais importantes é o material usado para gerar o rastro.
- ▶ Vamos criar um novo material para o rastro. Clique no botão Create da janela Project, escolha a opção Material e chame o material criado de Material Trail.
- ▶ Este será um material bastante básico, pois mudaremos apenas seu Shader.
- ▶ No Inspector desse material, mude a opção Shader de Standard para Particles e selecione Alpha Blended.
- ▶ Mude Tint Color para branco (255, 255, 255, 255).



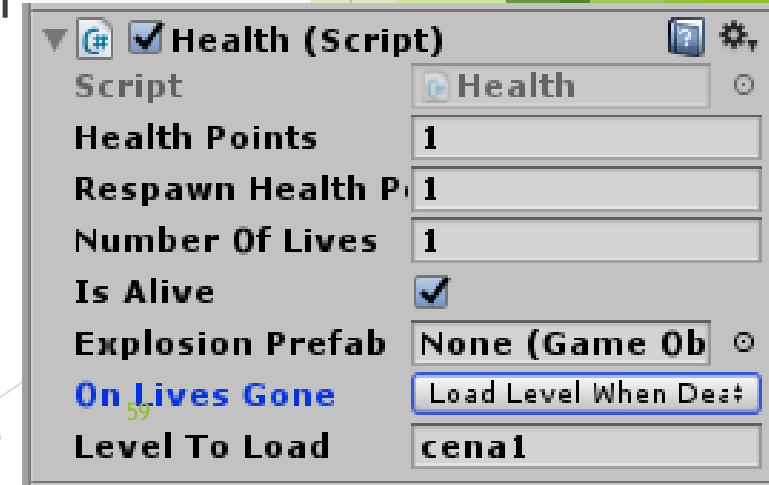
Roller Madness - Aparência do Jogador

- ▶ Agora devemos arrastar o Material Trail para a propriedade Material do Trail Renderer.
- ▶ O efeito do rastro mudou ao executar.
- ▶ Altere Time para 2 segundos.
- ▶ Na propriedade Width, clique na linha vermelha com o botão direito para selecionar a opção Add Key e crie pontos de interpolação da curva de diminuição da largura do rastro. Ao lado foi criado um ponto ao final da curva e puxado para baixo, resultando na curva apresentada.
- ▶ Assim, o rastro durará 2 segundos, começará com largura 0.8 e terminará com largura 0, de forma que aparecerá diminuindo até sumir.
- ▶ Por fim, mudaremos as cores do rastro durante sua existência.
- ▶ Clicando na propriedade Colors do Trail Renderer, aparecerá a janela Gradient Editor. Clique no Color Pick e escolha a cor 00FFEA na janela Color.
- ▶ A ferramenta montará o gradiente.
- ▶ O efeito será o da figura ao lado.



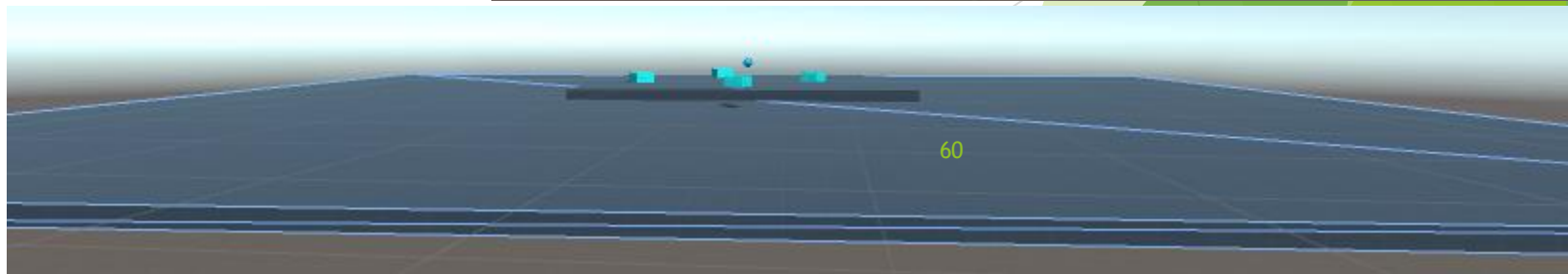
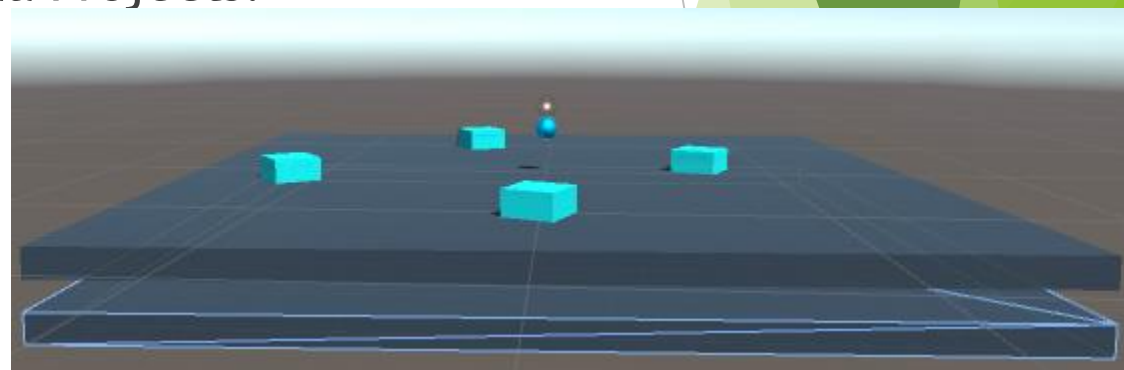
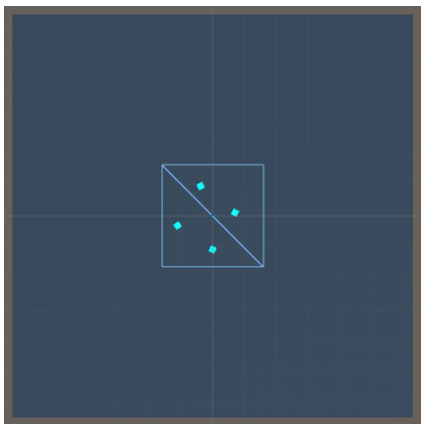
Roller Madness - Saúde e Danos

- ▶ Neste momento, se a bola cair para fora do chão, ela o fará para sempre. O ideal seria terminar ou reiniciar o jogo se isso acontecer.
- ▶ Vamos adicionar um Rastreador de Saúde e uma Zona de Danos que danifica o jogador se ele cai nela. A Zona de Danos será um game object que matará o jogador se ele colidir com ela.
- ▶ Para isso, temos dois scripts já prontos :
 - ▶ Health → rastreia a saúde de um personagem do jogo
 - ▶ Damage → danifica a saúde do personagem.
- ▶ Adicione o script Health à Rollerball. Esse script possui variáveis que informam HealthPoints do jogador, quantos HealthPoints ele terá se forgerado novamente (respawn), número de vidas. No momento, vamos manter esses valores com 1, pois o jogo será simples e se você morrer, você morreu.
- ▶ Mude a propriedade OnLiveGone para LoadLevelWhenDead, de forma que, quando o PC morrer, recarregaremos a cena.
- ▶ Em LevelToLoad, digite **cena1**, que é o nome da nossa cena atual, que será recarregada em caso de morte do PC.



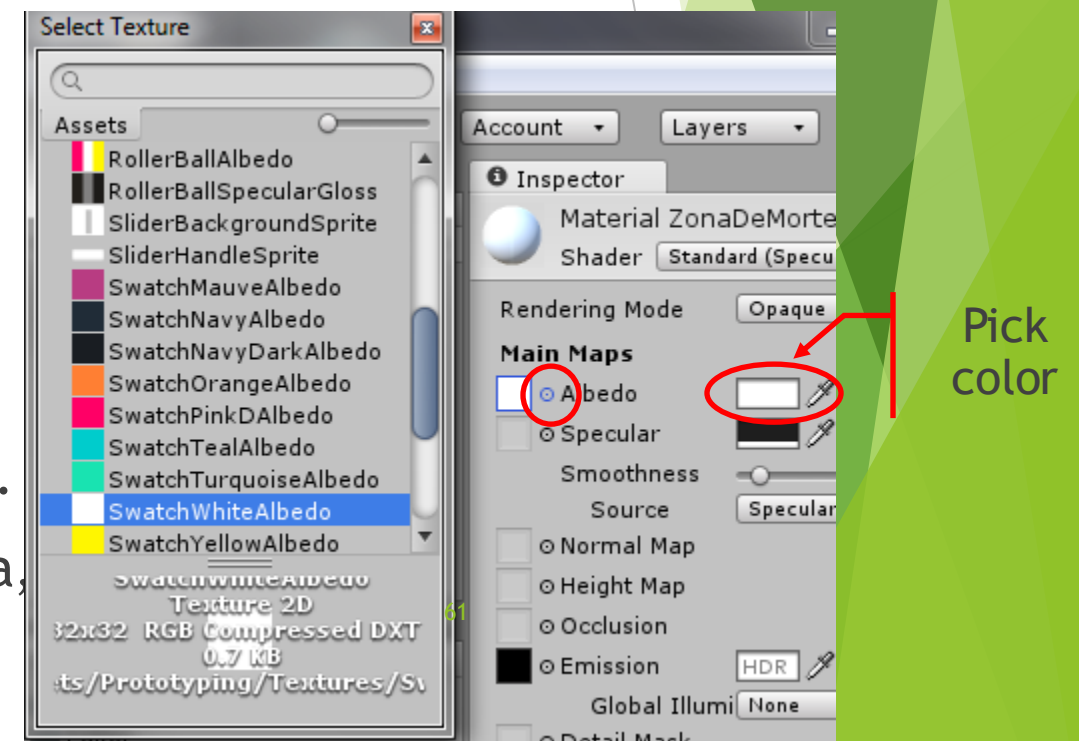
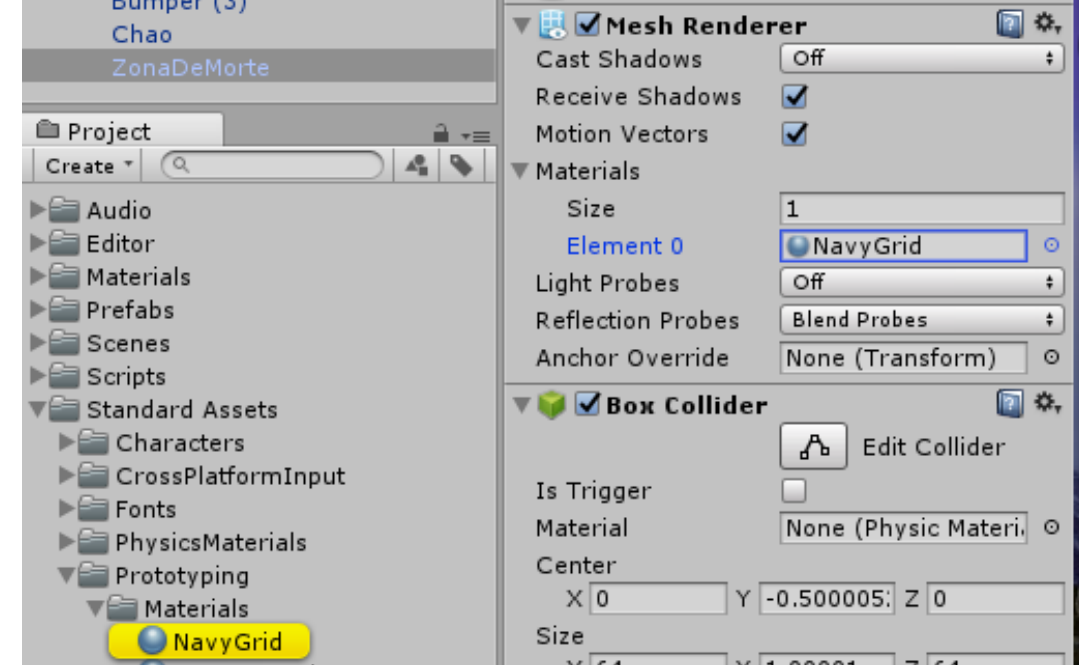
Roller Madness - Saúde e Danos

- ▶ Agora precisamos de algo que detecte quando o PC cai para fora do Chao e, então, realizar danos ao jogador.
- ▶ Há várias maneiras de conseguir isso, mas uma forma simples é criar um game object com o qual o jogador colida e, assim, dispare o evento que realiza danos.
- ▶ Clique no objeto Chao, pressione as teclas [Ctrl-D] para duplicá-lo. Mude o nome do objeto resultante para ZonaDeMorte.
- ▶ Esse objeto está, no momento, ligado ao prefab Chao. Vamos criar um prefab para ele, arrastando-o para a pasta Prefabs da janela Projects.
- ▶ Mude o transform Position do objeto ZonaDeMorte para (0, -2, 0), de forma que ele fique abaixo do Chao original. Mude sua escala para (2, 1, 2).



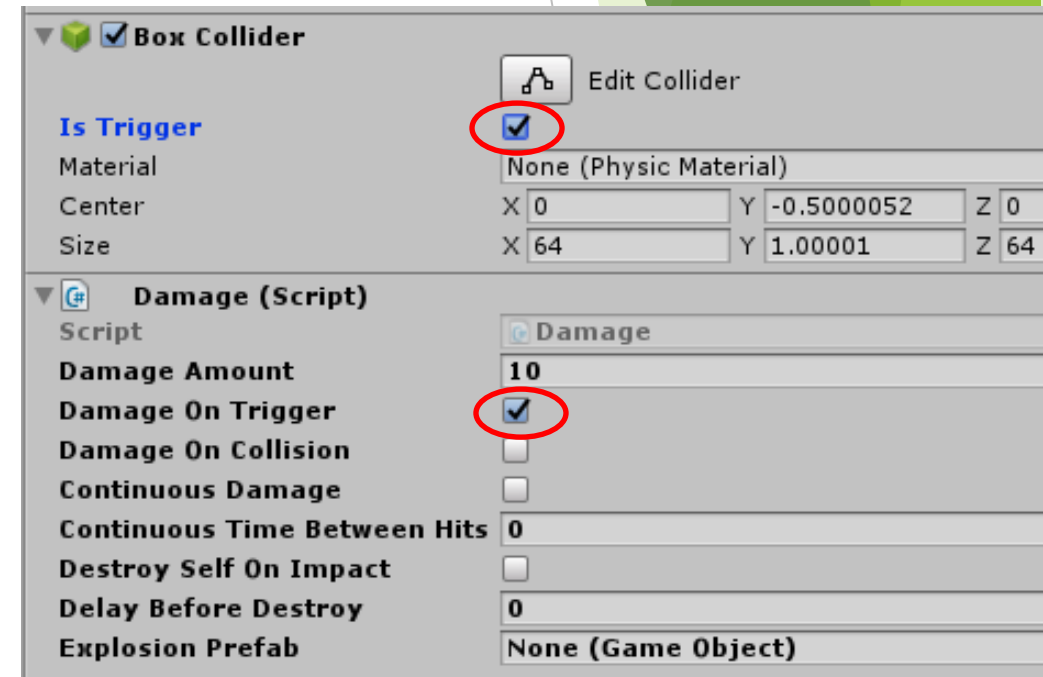
Roller Madness - Saúde e Danos

- ▶ Vamos criar um novo material para o objeto ZonaDeMorte, para diferenciá-lo do Chao.
- ▶ Clique na propriedade Mesh Renderer dele, acesse a propriedade Materials e veja que temos o Element 0 como NavyGrid. Clique em NavyGrid no Element 0 e aparecerá o Material na pasta Materials dos Standard Assets.
- ▶ Duplique esse material, chame-o de Material ZonaDeMorte e o arraste para a pasta Materials específica do nosso projeto, fora dos Standard Assets.
- ▶ Arraste esse novo Material para o objeto ZonaDeMorte, para que este passe a ter as características desse material (que modificaremos a seguir).
- ▶ Clique no Material ZonaDeMorte na pasta Materials, mude sua propriedade Albedo para SwatchWhiteAlbedo.
- ▶ Vá no Pick Color de Albedo e selecione uma cor amarela, por exemplo.



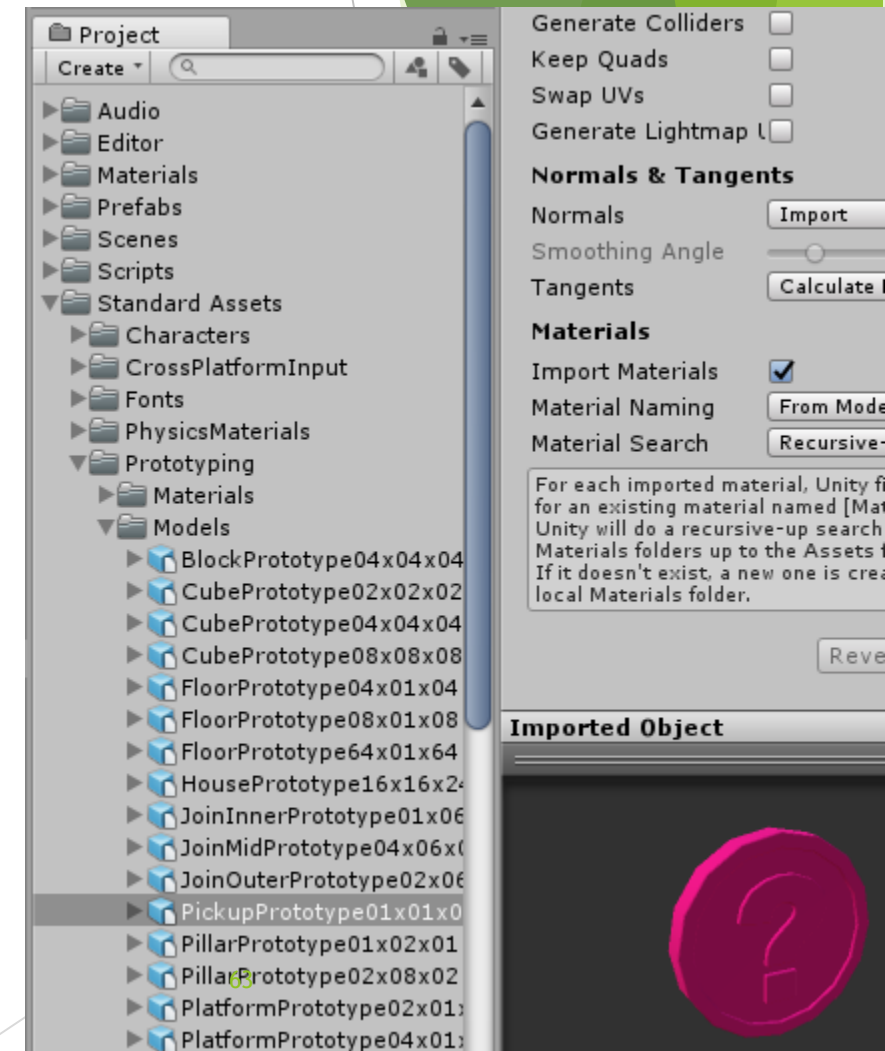
Roller Madness - Saúde e Danos

- Mude a propriedade Is Trigger do Box Collider de ZonaDeMorte para true, pois não queremos que a bola seja rebatida quando colidir com ZonaDeMorte, e sim queremos executar scripts quando ocorrerem colisões entre esse objeto e outros que o tocarem.
- Selecione o objeto ZonaDeMorte e adicione o script Damage a ele.
- Observe que a variável DamageAmount do script Damage vale 10, que é um valor maior que os HealthPoints do PC (a bola, cujo HealthPoints vale 1). Dessa maneira, o script subtrairá DamageAmount dos HealthPoints do objeto RollerBall e, como o resultado ficará negativo, matará a bola.
- A propriedade DamageOnTrigger deve valer true para que, no momento da colisão, o script danifique o objeto que colidiu com ZonaDeMorte.
- Ao executarmos o jogo, veremos que, quando a bola cai fora do Chao, ela se choca com a ZonaDeMorte e, nesse momento, a cena é recarregada, com a bola recomeçando no local original.



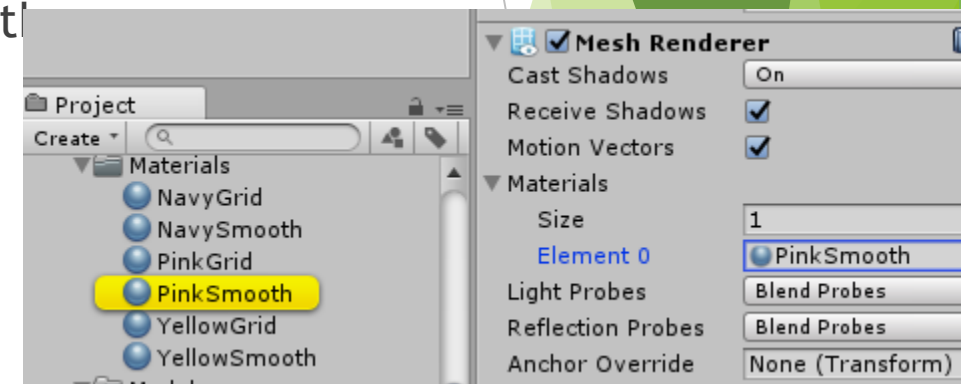
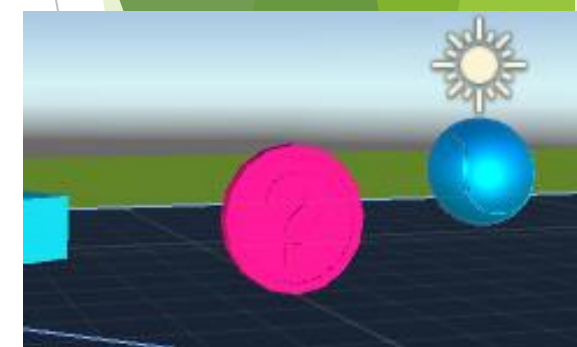
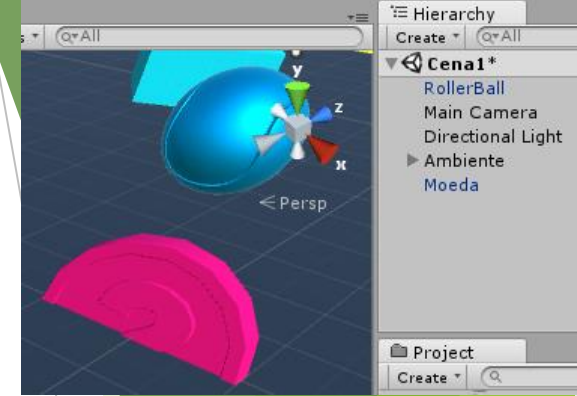
Roller Madness - Coleta de itens

- ▶ O que é um jogo sem a capacidade de coletar coisas?
- ▶ Vamos criar objetos para que nosso jogador tenha algo para coletar e, portanto, algo para fazer no jogo.
- ▶ Esses objetos serão instâncias de um prefab de moedas.
- ▶ As moedas serão geradas e colocadas na cena.
- ▶ Reconhecemos a natureza de cada objeto do jogo através da propriedade Tag.
- ▶ Para criar nossas moedas, novamente usaremos os Assets Padrão de Prototipagem.
- ▶ Em Standard Assets | Prototyping | Prefabs, há um objeto PickupPrototype mas, por algum motivo, ele está vazio e não temos como usá-los.
- ▶ Por outro lado, em Standard Assets | Prototyping | Models, há um objeto PickupPrototype01x01x01 que pode ser usado para criarmos o prefab de moedas.
- ▶ Arraste esse objeto para a cena e chame-o de Moeda.

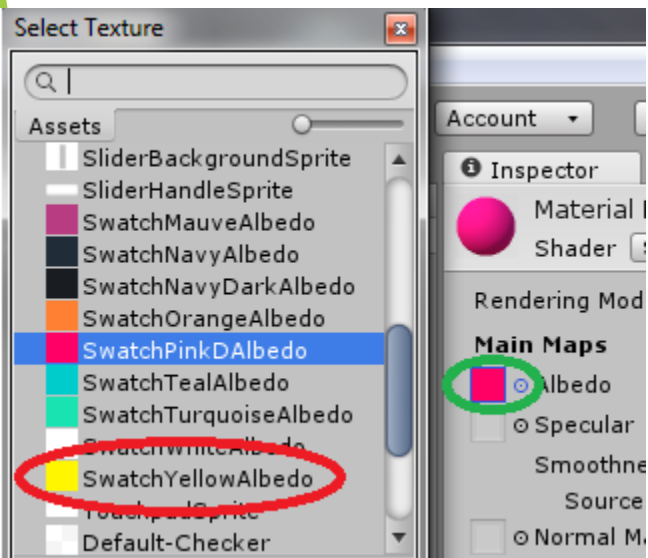


Roller Madness - Coleta de itens

- ▶ Faça o reset do transform da Moeda, de forma que ela fique na origem do sistema de coordenadas 3D.
- ▶ Pressionando a tecla [Ctrl], mude-a 0.5 unidade para cima e algumas para trás, de forma que fique sobre o Chao e afastada da bola.
- ▶ Ela ainda está grande em comparação com a bola, de forma que vamos mudar sua escala para (0.4, 0.4, 0.4) para que ela fique menor e mais difícil de ser tocada.
- ▶ Vamos mudar o material da moeda. Para isso, selecione a moeda e, no seu Inspector, clique em Mesh Renderer | Materials | Element 0. Note que aparece, na janela Project, o material do qual a moeda é “feita”, PinkSmooth.
- ▶ Faremos como das outras vezes, vamos duplicar esse material.

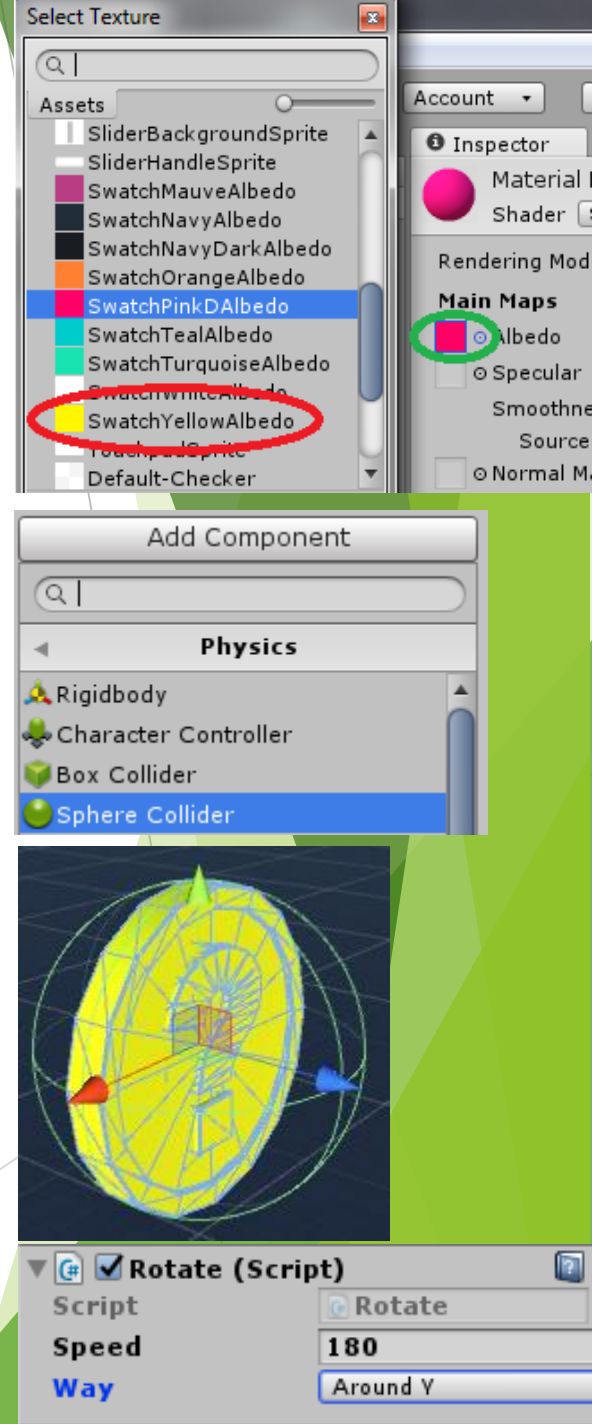


- O novo Material deve ser renomeado para Material Moeda.
- Arraste o Material Moeda para a pasta Materials do projeto.



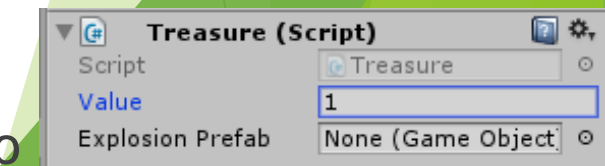
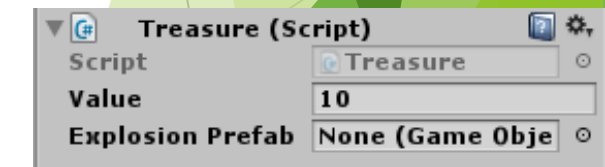
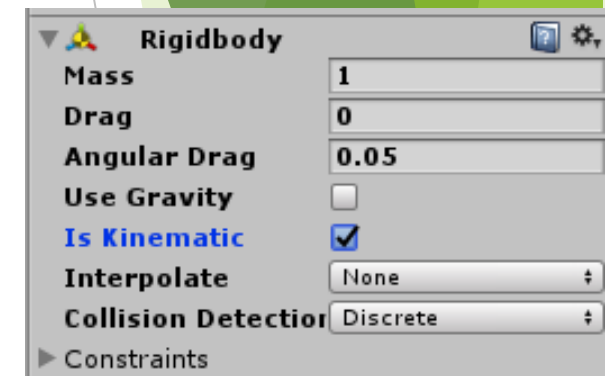
Roller Madness - Coleta de itens

- ▶ Selecione Material Moeda e, em seu Inspector, clique no círculo ao lado esquerdo da propriedade Albedo e o mude para SwatchYellowAlbedo.
- ▶ Arraste Material Moeda para o objeto Moeda na Hierarquia. A moeda ficará amarela.
- ▶ Gostaríamos que a moeda pudesse detectar colisões. Observe que no objeto Moeda não há um Collider, de forma que criaremos um.
- ▶ Selecione a Moeda na Hierarquia e, no seu Inspector, clique em [Add Component], selecione a opção Physics e, em seguida, Sphere Collider.
- ▶ A Moeda não é uma esfera, mas vamos detectar as colisões na área ao redor da moeda.
- ▶ Observe que, clicando em Sphere Collider no Inspector da Moeda, aparece uma esfera verde ao redor dela. Podemos mudar o tamanho do colisor para que ele fique um pouco mais que a moeda.
- ▶ Também será interessante que o player possa passar pela moeda ao coletá-la, de forma que vamos marcar **Is Trigger** do colisor como **True**.
- ▶ Vamos fazer a Moeda girar continuamente. Para isso, adicione o script Rotate após pressionar o botão [Add Component] | Scripts.
- ▶ Mude a variável **Way** para **AroundY** e **Speed** para **180** (graus por segundo).
- ▶ Execute o jogo para ver a moeda girando.



Roller Madness - Coleta de itens

- ▶ Vamos agora adicionar um Rigidbody à Moeda ([Add Component] | Physics | Rigidbody).
- ▶ A gravidade não irá atuar sobre a moeda mas, como ela será um objeto em movimento com um colisor, você sempre terá necessidade de um Rigidbody para que o sistema de física saiba que esse é um objeto dinâmico no mundo do jogo.
- ▶ Deixe a propriedade **Use Gravity** como **false**.
- ▶ Nosso objeto Moeda está sendo dirigido apenas pelo script Rotate, que faz com que o transform desse objeto rotacione.
- ▶ Como não estamos aplicando forças ou torque à Moeda, apenas rotacionando-a por meio de um script, vamos deixar a propriedade **Is Kinematic** igual a **true**.
- ▶ A próxima atividade que faremos sobre nossa Moeda é adicionar um script que permita ao jogo saber que ela é um objeto a ser coletado (pick up), ou seja, que a moeda é um “tesouro”.
- ▶ Assim, clique em [Add Component], selecione Scripts e adicione o script chamado **Treasure**. Mude o valor de Value para 1, ou seja, uma moeda equivale a 1 ponto.
- ▶ Execute o jogo e observe que, quando o player (a bola) colide com a moeda (o tesouro), a Moeda desaparece, ou seja, foi coletada.



Roller Madness - Coleta de itens

- ▶ O script Treasure basicamente verifica se o outro objeto que colidiu com o objeto que o possui (a Moeda) é o player. Se o outro objeto for o player, a Moeda é coletada e se destrói, saindo da cena.

```
public class Treasure : MonoBehaviour {
```

```
    public int value = 10;
```

```
    public GameObject explosionPrefab;
```

```
    void OnTriggerEnter (Collider other)
    {
```

```
        if (other.gameObject.tag == "Player") {
```

```
            if (GameManager.gm!=null)
```

```
            {
```

```
                // tell the game manager to Collect
```

```
                GameManager.gm.Collect (value);
```

```
            }
```

```
            // destroy after collection
```

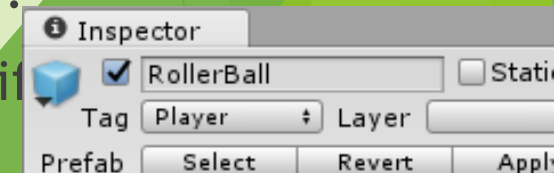
```
            Destroy (gameObject);
```

Evento que trata
Colisões em Is
Trigger = true

Este parâmetro
referencia o outro objeto
que colidiu com o dono
do script.

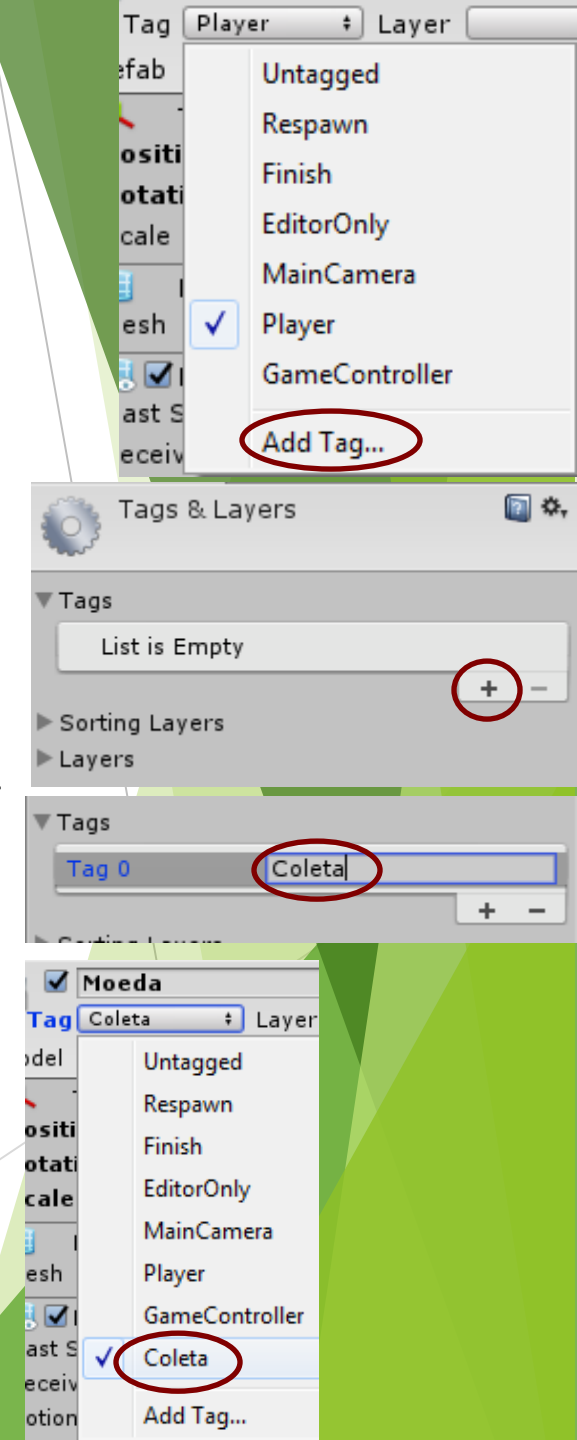
GameManager é
um script que
estudaremos
depois e que
gerenciará a
mecânica do jogo

- ▶ Como o objeto Moeda sabe que o outro objeto que colidiu com ele é o Player?
- ▶ Observe que a RollerBall tem a propriedade Tag configurada como Player. O código acima compara se o objeto do parâmetro other tem tag igual a "Player".



Roller Madness - Coleta de itens

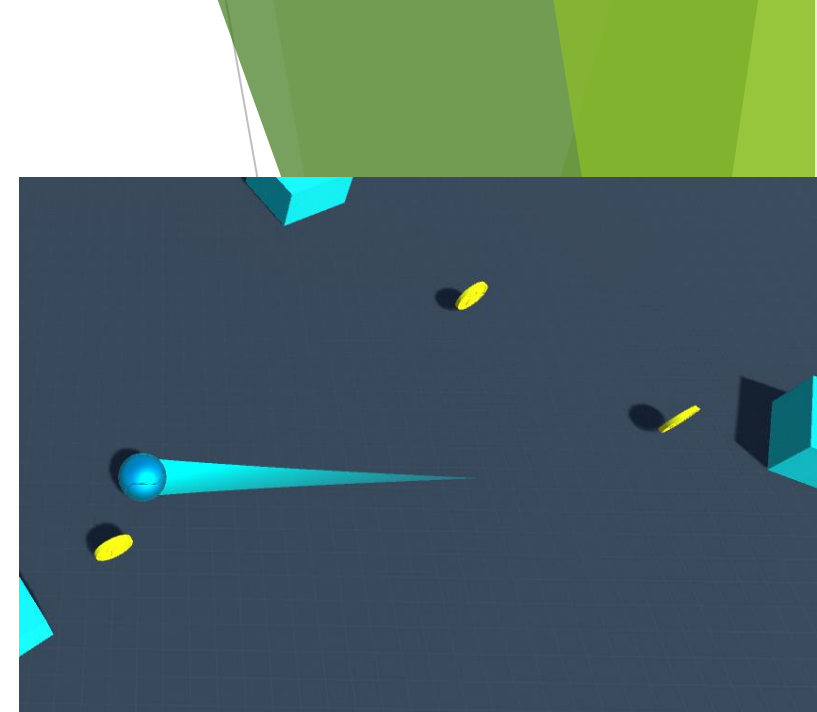
- ▶ O uso da propriedade Tag é uma forma poderosa de trabalhar nos scripts, para determinar o tipo de atividade esperada de um objeto. Você pode criar suas próprias tags, mas já existem alguns pré-definidos, como vemos ao lado.
- ▶ Seu código poderá responder aos objetos do jogo de acordo com seus nomes, mas às vezes temos várias instâncias do mesmo objeto (por exemplo, várias moedas) no jogo e o uso das tags torna o processamento mais geral e unificado para cada tipo de objeto.
- ▶ Vamos colocar uma tag em nossa moeda. Para isso, clique na Moeda, abra a lista de tags no Inspector e clique em Add Tag... . Em seguida, entramos no editor de Tags e Layers. Pressione o símbolo + e digite o nome Coleta, que é um nome mais genérico que Moeda.
- ▶ Em seguida, clique na Moeda novamente e mude sua tag para Coleta, que passou a aparecer na lista suspensa de tags.



Roller Madness - Coleta de itens

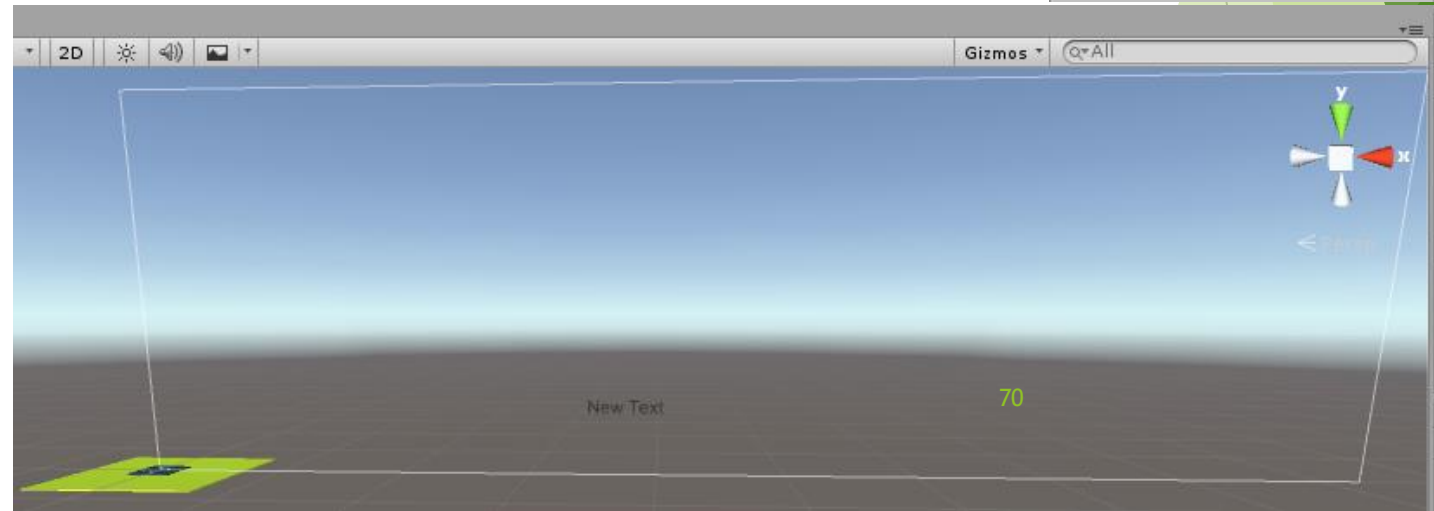
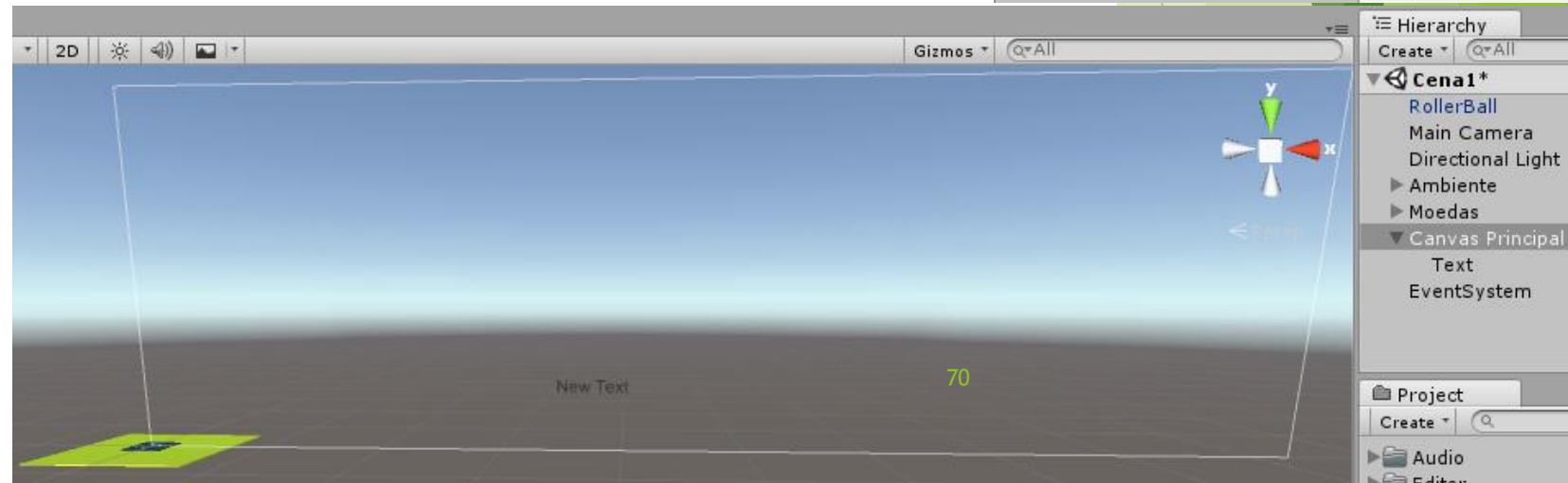
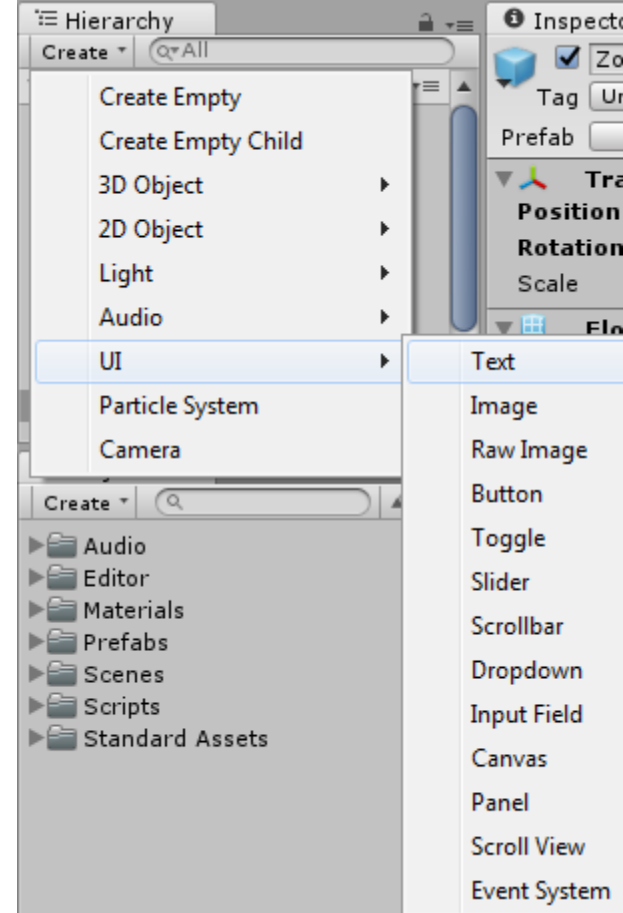
- ▶ Temos nossa Moeda configurada. Como teremos várias moedas no jogo, é interessante torná-la em um Prefab.
- ▶ Arraste a Moeda para a pasta Prefabs da janela Project e ela virará um Prefab.
- ▶ Arraste o Prefab Moeda para a cena algumas vezes, para povoá-la. Ajuste suas posições para que fiquem ao redor da bola, cercando-a mas não muito perto.
- ▶ Organize a Hierarquia do seu jogo.
- ▶ Crie um objeto vazio, faça o reset de seu transform e chame-o de Moedas
- ▶ Arraste todas as moedas para dentro dele.
- ▶ Execute seu jogo e colete todas as moedas.
- ▶ Em seguida, criaremos um painel para exibir a quantidade de moedas coletadas, bem como adicionar uma tela de Game Over.

Na execução no Editor do Unity, a recarga da cena atual poderá voltar escurecida. Para ver como impedir isso, clique [aqui](#).



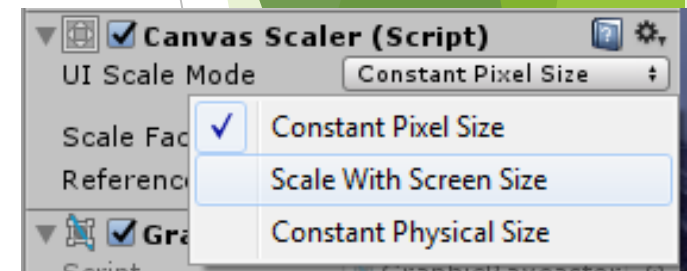
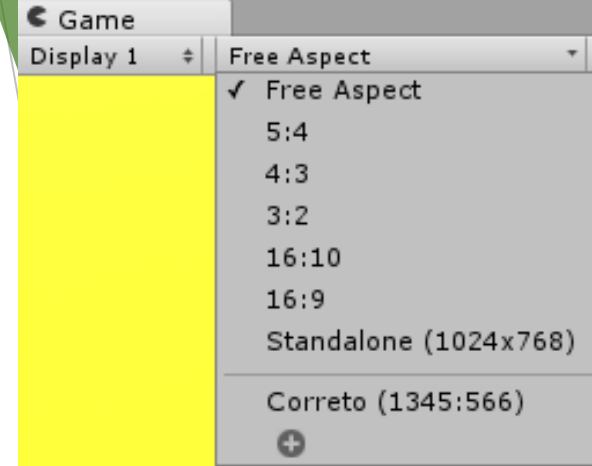
Roller Madness - UI Básica

- ▶ Agora que temos um sistema de coletas, precisamos de uma maneira de exibir os pontos obtidos pelo jogador.
- ▶ Também é aconselhável ter uma mensagem de Game Over quando ocorrer a morte do jogador ao invés de simplesmente reiniciar a cena.
- ▶ Faremos as duas coisas criando uma interface com usuário básica para nosso jogo.
- ▶ Para criar uma interface com usuário (UI) no jogo, acesse o botão [+] da Hierarquia, selecione UI e Text, para criarmos uma exibição de texto para nossas moedas.
- ▶ Após isso ter feito, Unity criou um objeto Canvas e um item filho chamado Text, além de um objeto EventSystem.
- ▶ Canvas define o espaço 2D onde os elementos de UI são gerados.
- ▶ Mude o nome do Canvas para Canvas Principal.



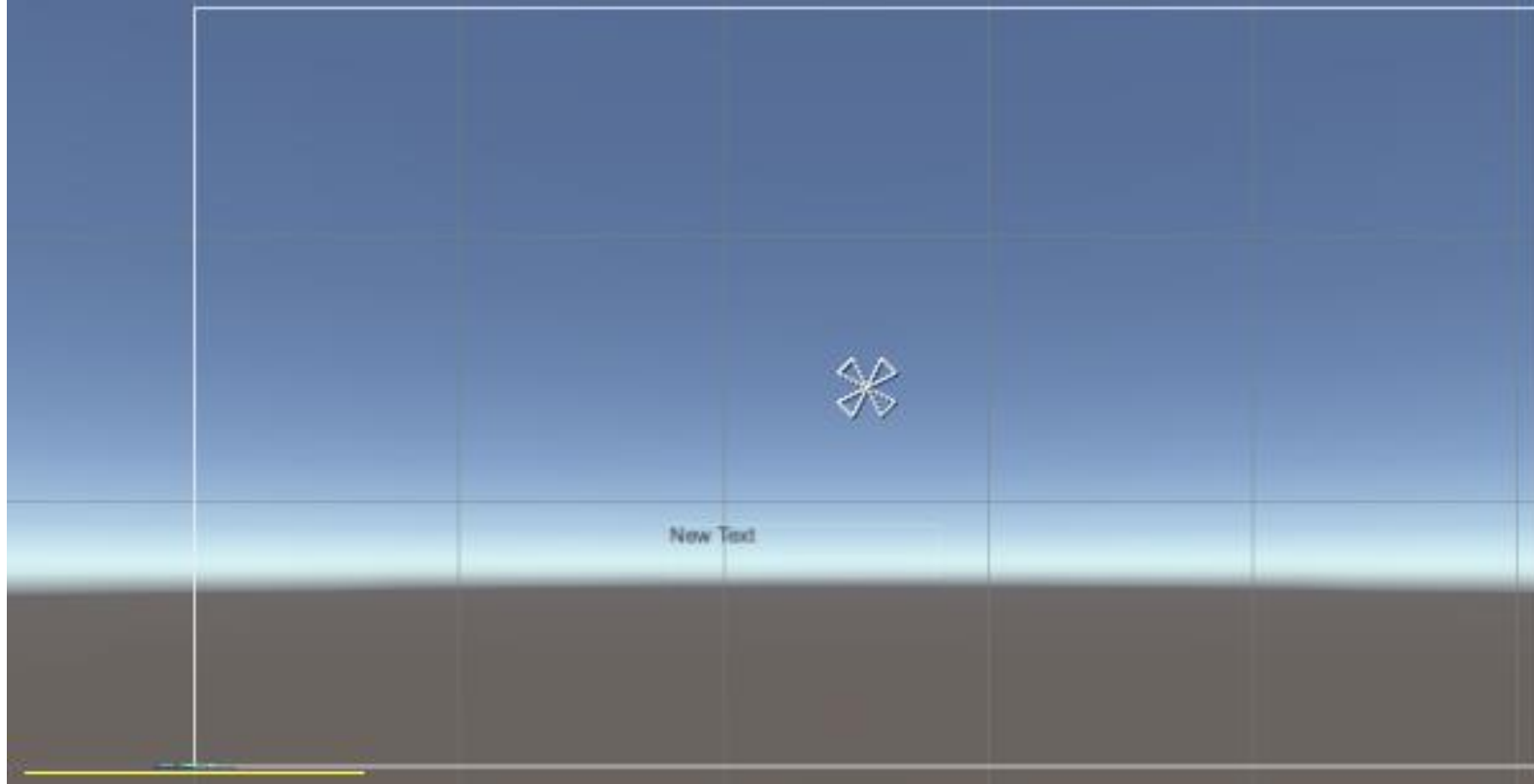
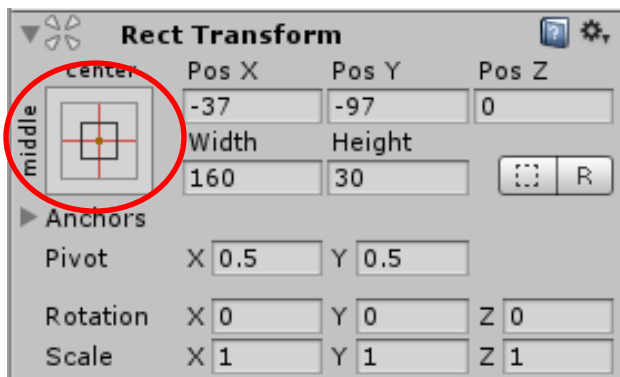
Roller Madness - UI Básica

- ▶ No Inspector do Canvas Principal há vários itens, uns dos quais é o script CanvasScaler, que define como o Canvas é escalado em função do tamanho da tela de jogo.
- ▶ Atualmente estamos usando a proporção Free Aspect mas talvez você queira usar a proporção 16:9, que exibirá mais coisas na tela.
- ▶ Além disso, vamos mudar o UI Scale Mode de “Constant Pixel Size” para “Scale with Screen Size”.
- ▶ Renomeie o objeto Text para txtScore e pressione [F] ou clique duas vezes sobre ele para aproximar a visualização desse objeto.
- ▶ Observe que ele está na cena, mas distante do objeto Chao, onde nossa ação ocorre. Também possui a string “New Text” sendo exibida. Estamos vendo esse objeto de texto no espaço 3D. No entanto, ele é um objeto bidimensional.
- ▶ No topo da janela Scene, clique a opção 2D. Faça zoom-out e note que o txtScore não está alinhado corretamente no Canvas.
- ▶ Temos que alinhá-los de forma apropriada.



Roller Madness - UI Básica

- No Inspector de txtScore, note no topo um pequeno diagrama na propriedade Rect Transform:

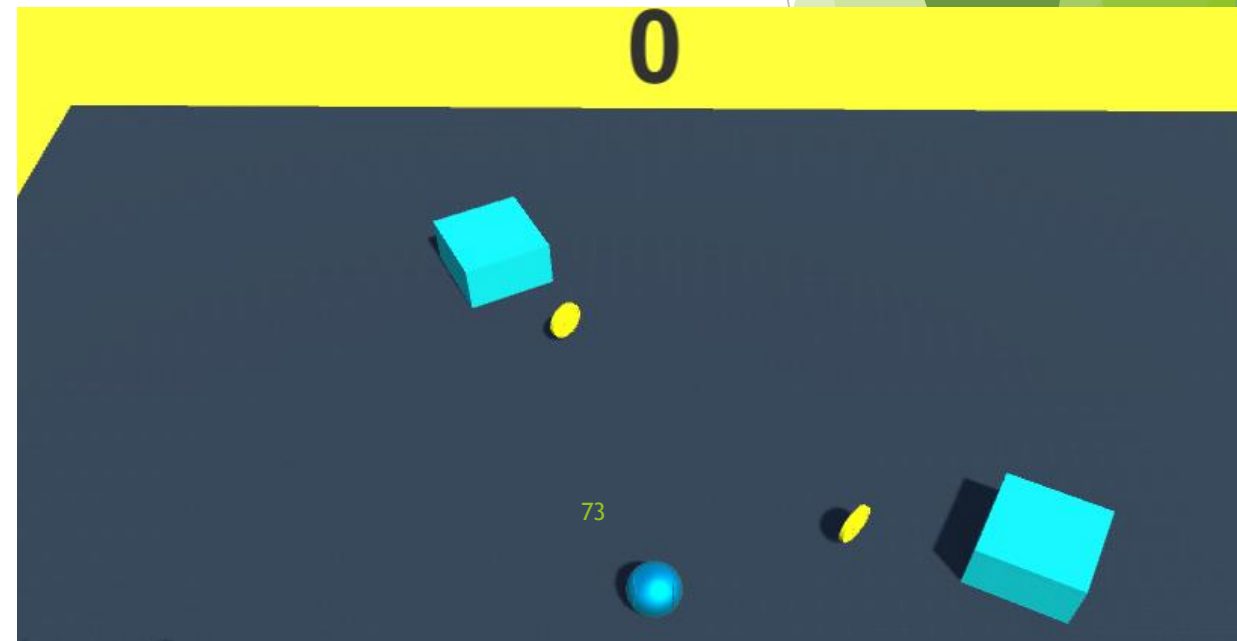


- Clicar nesse quadrado abre as opções de ancoragem.
- No topo, essa janela informa :
 - Shift : Also set pivot
 - Alt : Also set position.
- Isso significa que, se pressionar [Shift] e [Alt], poderá realizar várias coisas enquanto posiciona o txtScore.



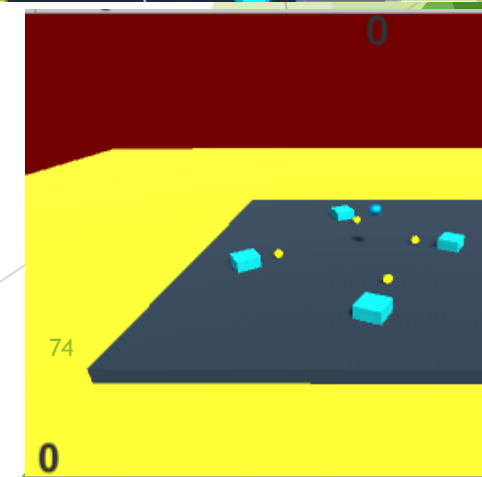
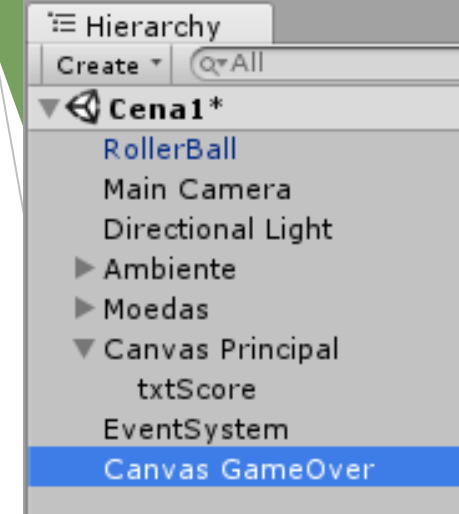
Roller Madness - UI Básica

- ▶ Pressione [Shift], [Alt] e clique no quadro central do topo da janela.
- ▶ Isso fará com que txtScore fique posicionado na parte superior e central do Canvas.
- ▶ Mude o texto para 0 (zero), o tamanho do fonte para 40 e deixe-o como bold.
- ▶ Ele desapareceu da tela porque ficou muito grande para o espaço disponível no txtScore.
- ▶ Uma forma rápida de resolver isso é mudar as propriedades Horizontal Overflow e Vertical Overflow de Wrap para Overflow.
- ▶ Se o txtScore não aparecer, clique na Camera e desligue o script SmoothFollow, ajuste a Camera e ligue o script novamente. Você também pode mudar Scale na janela Game.



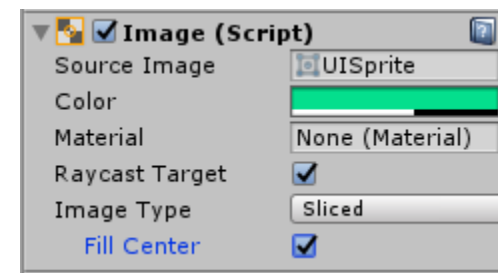
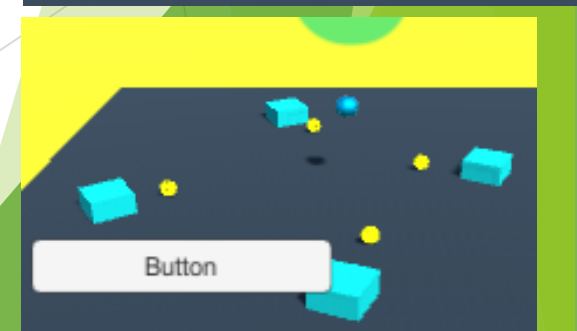
Roller Madness - UI Básica

- ▶ Vamos também criar uma mensagem de Game Over. Para isso, na janela Hierarquia, clique em [+] | UI | Canvas e criará um novo Canvas.
- ▶ Mude o nome desse novo objeto para Canvas GameOver.
- ▶ No Script Canvas Scaler mude a variável UI Scale Mode para “Scale with Screen Size”.
- ▶ Queremos colocar um texto nesse Canvas, portanto com o Canvas GameOver selecionado, clique em [+] | UI | Text e um novo campo texto será criado como filho do Canvas GameOver.
- ▶ Ao invés de configurá-lo do início, podemos copiar as propriedades do txtScore para esse novo campo de texto.
- ▶ Para isso, abra o Canvas Principal. Clique no txtScore e nos três pontinhos verticais. Selecione a opção Copy Component.
- ▶ Agora, no campo de texto recém-criado, clique na engrenagem e, em seguida, selecione a opção “Paste Component Values”. Os valores do txtScore e suas configurações serão copiados nesse outro texto.
- ▶ Mude o nome desse objeto para txtEndGameScore.



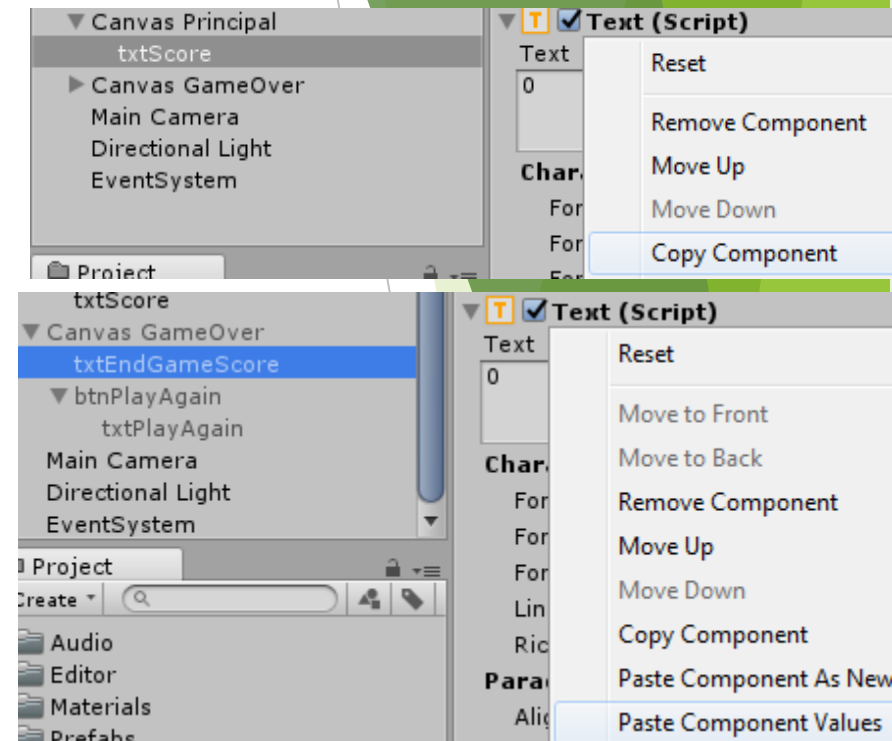
Roller Madness - UI Básica

- ▶ Centralize esse texto, clicando no quadro abaixo de Rect Transform no Inspector e, pressionando as teclas [Shift] e [Alt], clique no quadro central da ancoragem.
- ▶ Mude o tamanho do fonte para 150 e suba a coordenada y do transform do txtEndGameScore para 120. Mude Color para 04DF8D96.
- ▶ Vamos agora criar um botão para que o jogador possa escolher entre terminar o jogo ou reiniciá-lo.
- ▶ Assim, selecione o Canvas GameOver, clique em [+] da Hierarquia, selecione UI e Button.
- ▶ Um botão aparecerá no seu Canvas. Mude seu nome para btnPlayAgain.
- ▶ Vamos centralizar o botão, usando o quadro de ancoragem e as teclas [Shift] e [Alt], clicando na posição central de ancoragem.
- ▶ Se o botão se sobrepor ao txtGameOver desse Canvas, mude a coordenada y do botão para que ele fique abaixo do texto.
- ▶ Mude a cor para 04DF8D96, a mesma cor do texto.
- ▶ Fill Center faz com que o botão seja preenchido.



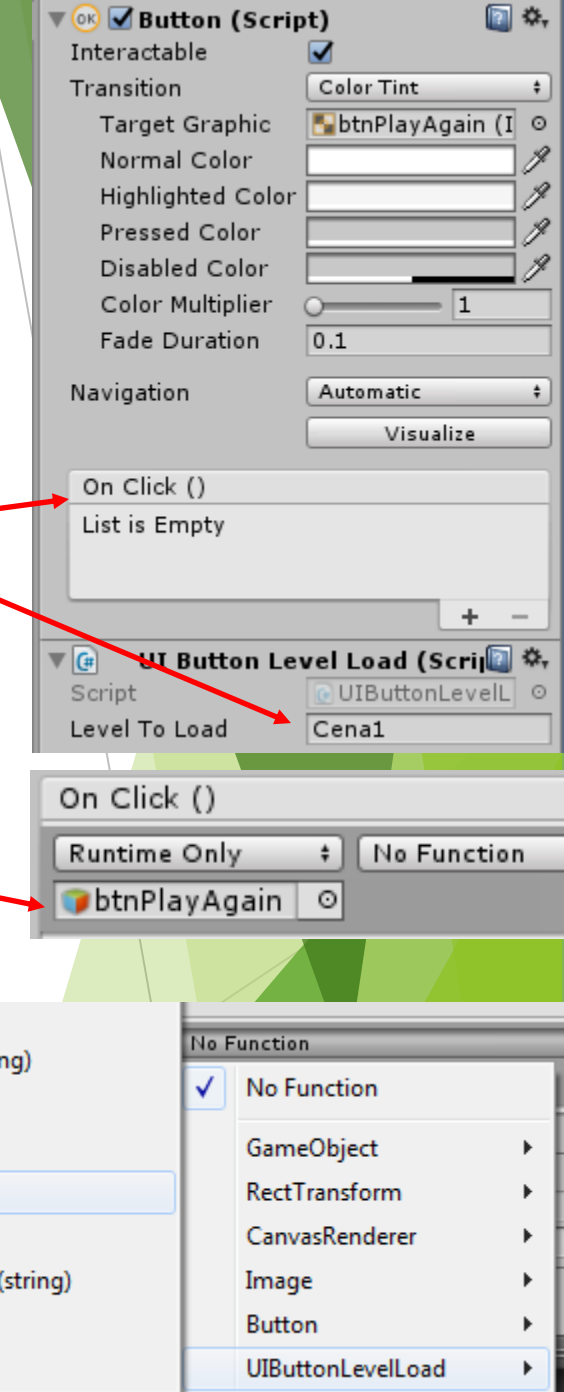
Roller Madness - UI Básica

- ▶ Vamos mudar o texto do botão. Existe um objeto Text subordinado ao botão na Hierarquia.
- ▶ Podemos copiar as propriedades de outro texto sobre esse texto e configurá-lo mais rapidamente.
- ▶ Clique no componente txtScore do Canvas Principal, clique na engrenagem do script Text e selecione Copy Component.
- ▶ Agora, selecione o txtPlayAgain do btnPlayAgain, clique na engrenagem do script Text e selecione Paste Component Values para que o texto fique igual ao do txtScore. Mude o texto exibido para “Play Again” e o tamanho do fonte para 20, por exemplo.
- ▶ Se necessário, mude os alinhamentos e ligue Align By Geometry.
- ▶ Você pode alterar as propriedades Width e Height do botão, na opção Rect Transform, caso o botão não esteja do tamanho adequado.



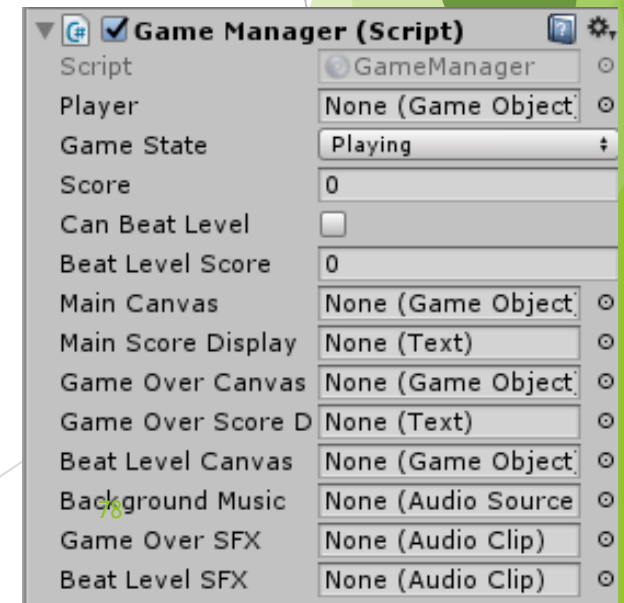
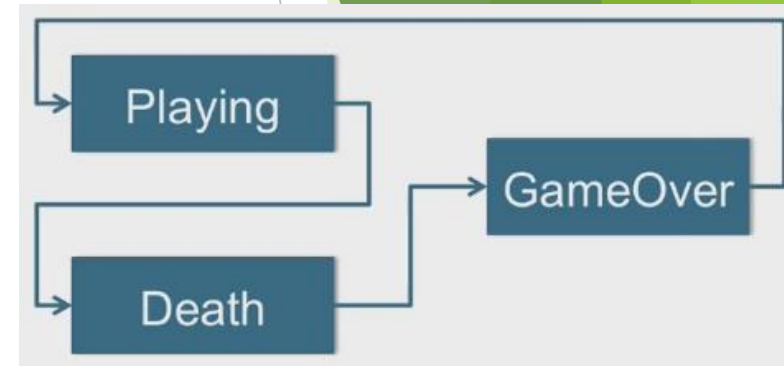
Roller Madness - UI Básica

- ▶ No botão btnPlayAgain, adicione o script UIButtonLevelLoad, que já está disponível em nosso projeto. Há também um script UIButtonQuitGame, para o caso de você desejar parar o jogo (só funcionará fora do Unity).
- ▶ Na variável LevelToLoad, digite o nome da cena atual (cena1) já que é a única que temos no momento.
- ▶ Agora, temos que fazer o botão realmente chamar esse script.
- ▶ Na figura ao lado vemos o editor do evento OnClick(), que é usado para definir que ação será realizada quando clicarmos no botão.
- ▶ Clique no botão [+] e arraste o botão btnPlayAgain da Hierarquia para a caixa abaixo da lista com as palavras Runtime Only.
- ▶ Isso trará o objeto com todas suas propriedades para esse componente e poderemos escolher o que fazer. Note que há uma lista com as palavras No Function, indicando que nenhuma ação foi determinada ainda.
- ▶ Clicando nessa lista, aparecerão os componentes.
- ▶ Escolha UIButtonLevelLoad e LoadLevel().
- ▶ Torne o Canvas GameOver inativo pelo Inspector para que ele não fique visível antes de o jogo terminar.



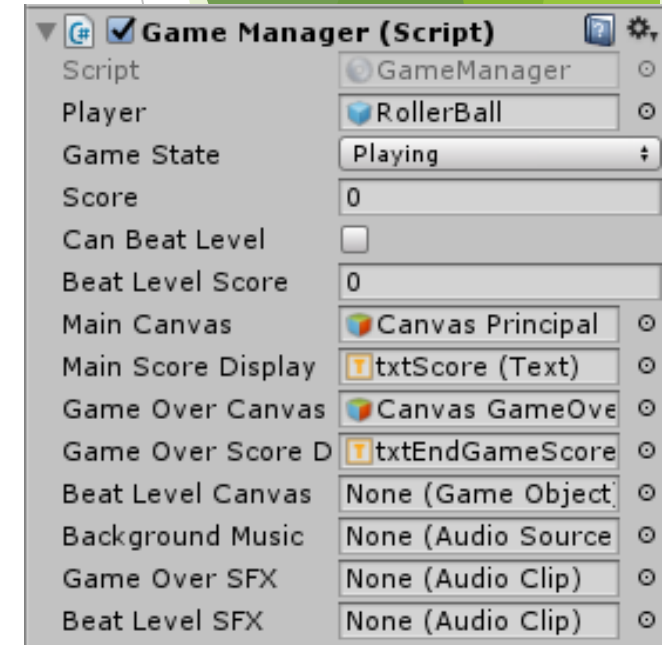
Roller Madness - Game Manager

- ▶ Vamos criar uma classe, chamada GameManager, que controlará os estados do nosso jogo e atualizará os elementos da UI de forma condizente.
- ▶ Este jogo é bastante simples, com apenas 3 estados:
 - ▶ Playing → o jogo começa nesse estado e nele fica até o player morrer.
 - ▶ Death → o jogo vai para esse estado quando o player morre e se alterna para o Canvas GameOver e a música diminui até parar.
 - ▶ GameOver → o jogo transita para esse estado e a música de GameOver toca. Fica nesse estado até que o jogador clica o botão [Play Again], o que leva à transição para o estado Playing.
- ▶ Vamos criar um game object vazio e nele incorporar o componente GameManager.
- ▶ Crie um objeto vazio na Hierarquia e o chame de Game Manager. Arraste-o para o topo da hierarquia e faça o reset de sua posição.
- ▶ Adicione o Componente de Script Game Manager no Inspector desse objeto.



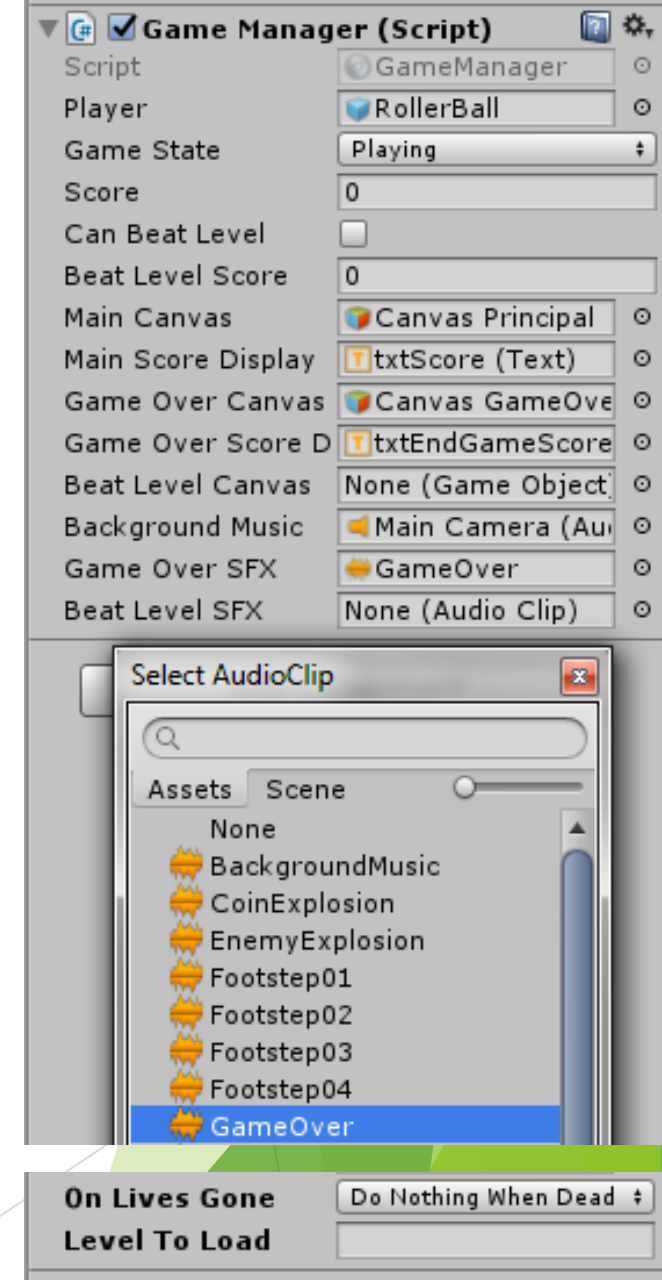
Roller Madness - Game Manager

- ▶ Este é um gerenciador de jogo básico mas que pode funcionar com vários tipos de jogos.
- ▶ Você pode ver as várias propriedades que teremos de configurar.
- ▶ Arraste RollerBall para a variável **Player** do Game Manager.
- ▶ A variável **Score** armazena o valor de pontuação do jogador.
- ▶ **Can Beat Level** igual a true permite que a fase seja ganha quando o jogador atingir a pontuação definida na variável **Beat Level Score**. Por exemplo, CanBeatLevel = true e BeatLevelScore = 5 fará com que mude a fase caso o jogador colete 5 moedas, pois cada moeda vale 1 ponto.
- ▶ **Main Canvas** deve receber nosso objeto **Canvas Principal**.
- ▶ **Main Score Display** deve receber o **txtScore**, para exibir a pontuação.
- ▶ **Game Over Canvas** deve receber nosso objeto **Canvas GameOver**.
- ▶ **Game Over Score Text** deve receber o **txtEndGameScore**.
- ▶ Se tivéssemos a possibilidade de vencer a fase, poderemos ter um Canvas específico para parabenizar o jogador e o indicariamos em **Beat Level Canvas** no Game Manager.



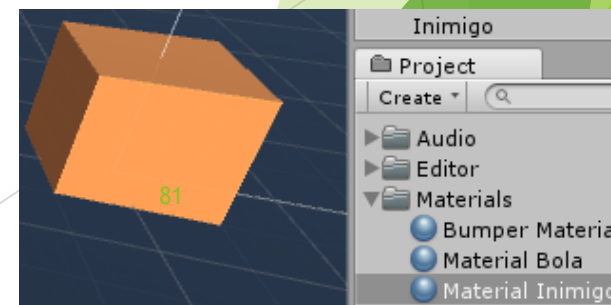
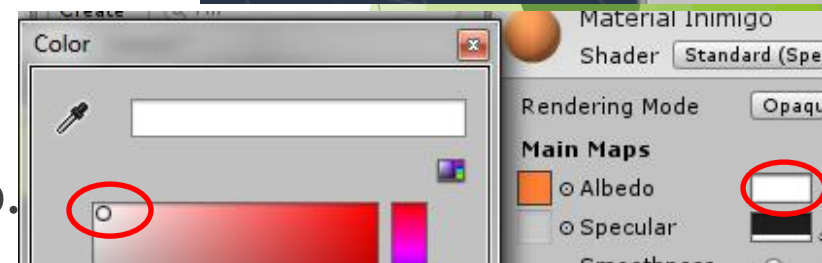
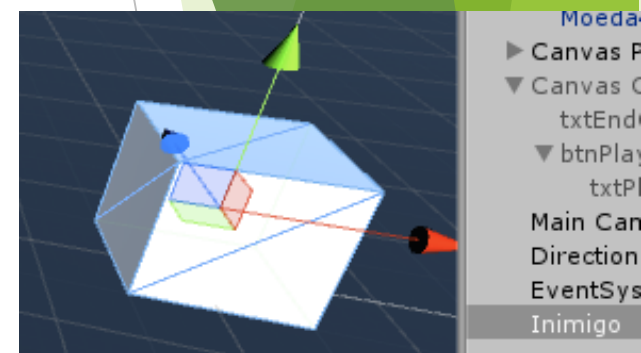
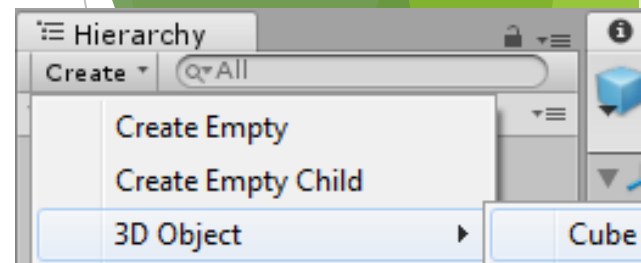
Roller Madness - Game Manager

- ▶ **Background Music** deve receber a fonte de música, ou seja, a **Main Camera**.
- ▶ Como Efeito Sonoro do Game Over (**Game Over SFX**) podemos clicar no pequeno círculo à direita dessa variável e selecionar a música **GameOver**.
- ▶ Agora temos que fazer uma pequena alteração em nosso player, ou seja, no objeto **Rollerball**.
- ▶ Quando adicionamos o script **Health** à **Rollerball**, informamos na variável **On Lives Gone** o que deveria ser feito, ou seja, “**Load Level when dead**” (carregar o nível quando morto) e indicamos **Cena1** como fase a ser carregada.
- ▶ Agora, será o **Game Manager** que cuidará disso e, portanto, vamos indicar em **On Lives Gone** que nada será feito pelo script **Health**, marcando “**Do nothing when dead**” nessa variável.
- ▶ O **Game Manager** detectará que o player morreu e exibirá a UI adequada.
- ▶ Execute e observe o resultado. Quando ocorre a morte, o jogo fala “**Game Over**” e podemos ver a pontuação obtida.



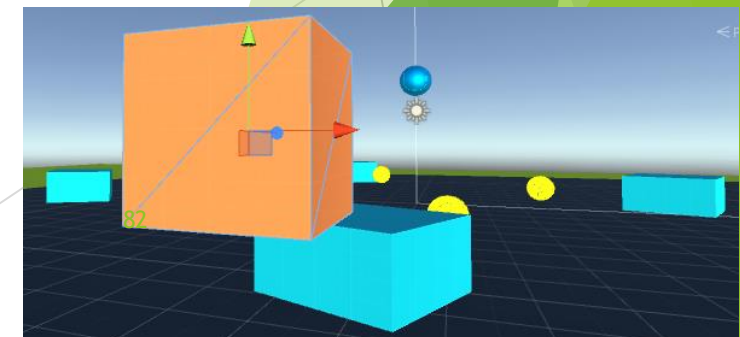
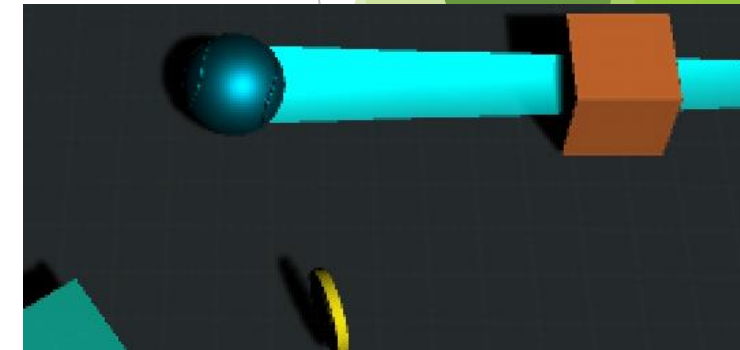
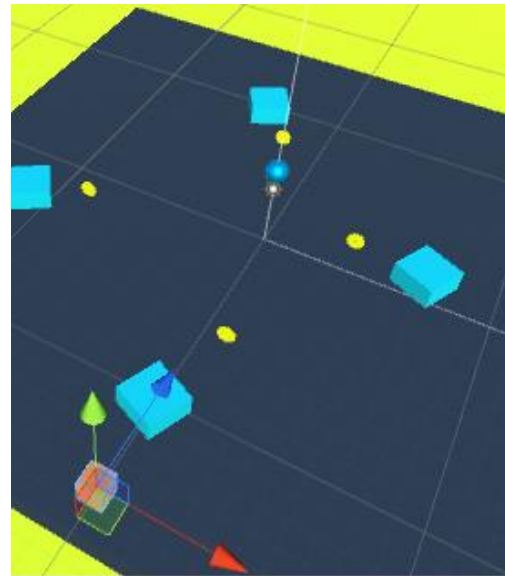
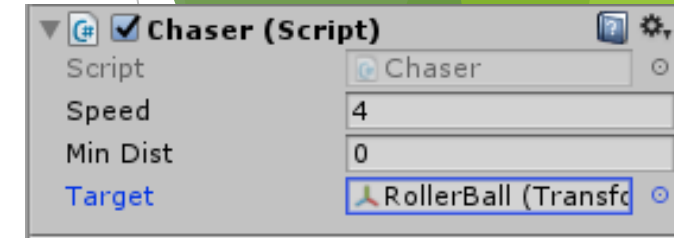
Roller Madness - Inimigos

- ▶ Nosso jogo seria muito mais desafiador e interessante se houvesse inimigos no mundo, que tivessem de ser evitados para sobrevivermos.
- ▶ Vamos aproveitar a criação de inimigos para rever alguns conceitos já estudados.
- ▶ Na hierarquia, crie um novo Cubo, usando [+] | 3D Object | Cube. Faça o reset desse cubo para que ele vá à origem do sistema de coordenadas.
- ▶ Suba-o na direção Y para que fique sobre o Chao e mude seu nome para Inimigo.
- ▶ Na pasta Materials, duplique o “Material ZonaDeMorte”. Mude o nome da cópia para “Material Inimigo”.
- ▶ Na propriedade Albedo do Material Inimigo, troque a cor por SwatchOrangeAlbedo. Troque a cor do quadrinho por branco puro.
- ▶ Arraste o novo Material para o cubo. Nosso inimigo será laranja.



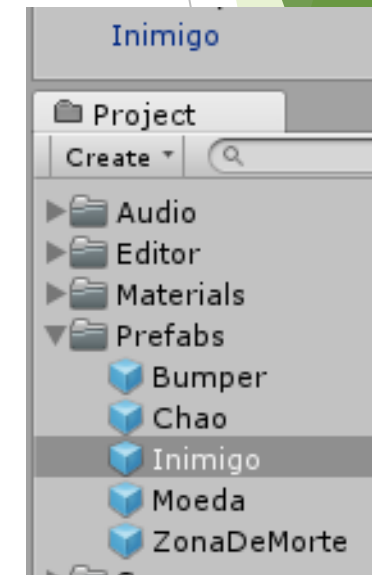
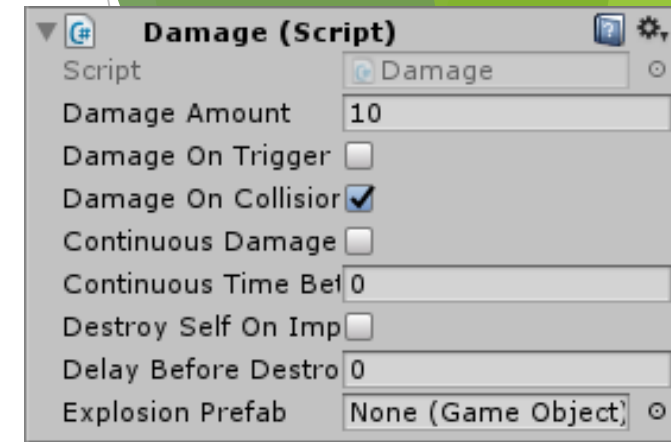
Roller Madness - Inimigos

- ▶ Ao objeto Inimigo, adicione o script Chaser, que o fará perseguir outro gameObject que, no nosso caso, será a Rollerball.
- ▶ O script Chaser possui 3 variáveis: Speed (mude para 4), Min Dist (mude para 0) e Target (arraste a Rollerball para cá).
- ▶ Esse script está escrito de tal forma que, se não houver um Target assinalado, ele procurará os objetos de jogo até encontrar um com tag igual a "Player", passando a seguir esse objeto. Estude seu código.
- ▶ Execute o jogo e perceba que o cubo laranja persegue nossa Rollerball.
- ▶ O inimigo atravessa os bumpers porque ainda não está pronto.
- ▶ Mova o inimigo para longe do player.
- ▶ Adicione um Rigidbody para que ele possa colidir com as coisas.
- ▶ Será afetado pela gravidade e outras forças.
- ▶ Mude o Physical Material do Box Collider para Bouncy e o Y para 2 metros.



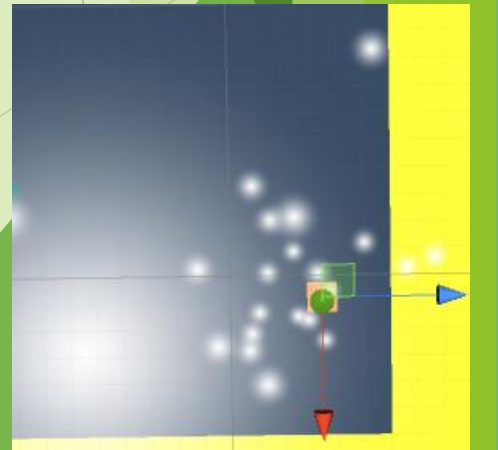
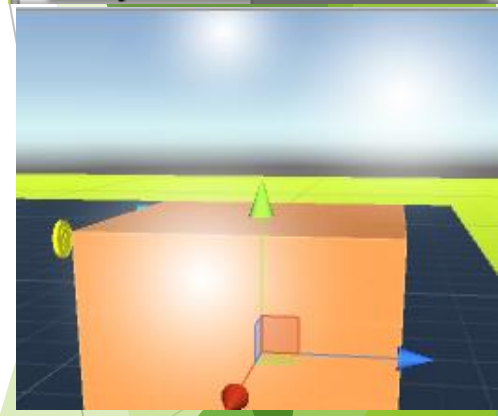
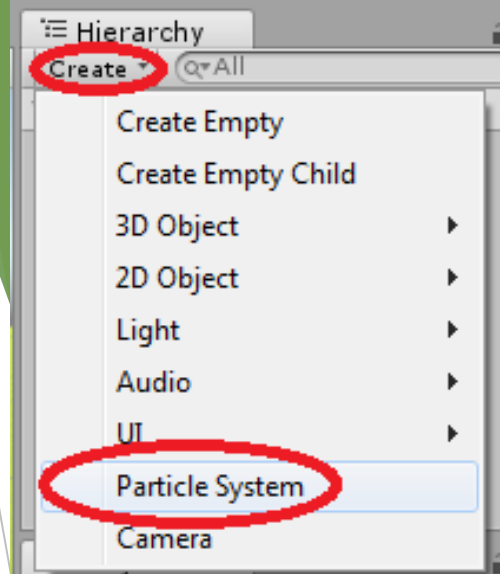
Roller Madness - Inimigos

- ▶ O Inimigo, neste momento, apenas segue nosso player, não infligindo nenhum dano a ele. Assim, vamos adicionar o script Damage ao Inimigo.
- ▶ Damage Amount pode ficar valendo 10, pois 10 é maior que 1, que é a saúde do player e queremos matar o player quando houver contato.
- ▶ Desmarque **Damage On Trigger** e marque **Damage On Collision**.
- ▶ Também queremos que o Inimigo tenha o script Health, para que sua saúde vá decaindo quando ele colidir com algum objeto, outro Inimigo ou cair para fora do Chao, como acontece com o Player.
- ▶ Adicione o Script Health ao Inimigo, deixe os valores default e execute o jogo. Se a velocidade do Inimigo estiver muito alta, você pode diminuí-la um pouco no script Chaser, para 3, por exemplo.
- ▶ Crie um Prefab a partir do objeto Inimigo.
- ▶ A partir do prefab Inimigo, crie uma nova instância do objeto na cena, em uma posição simetricamente oposta em relação ao primeiro.
- ▶ Da mesma forma que fizemos com as Moedas, seria interessante ter um objeto Vazio para armazenar e organizar os inimigos.
- ▶ Execute o jogo e observe que os inimigos se destroem caso colidam.



Roller Madness - Partículas e Efeitos Sonoros

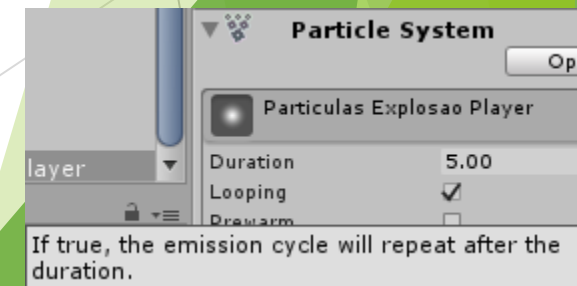
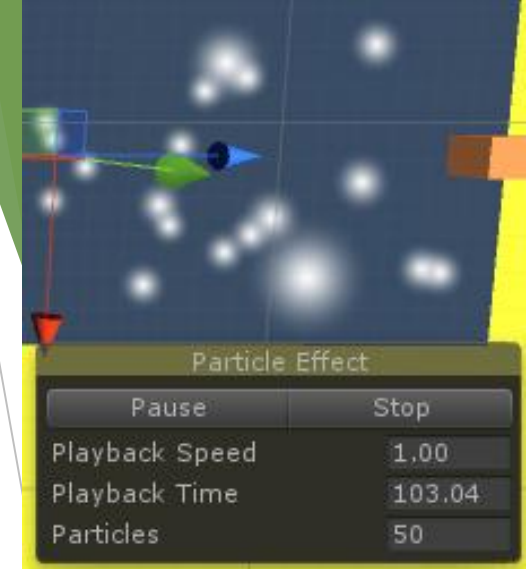
- ▶ Até este momento, temos apenas objetos sólidos em nosso jogo.
- ▶ Um Sistema de Partículas nos permite simular entidades semelhantes a fluidos, tal como fogo, fumaça e nuvens.
- ▶ Os melhores efeitos de partículas são obtidos pela combinação de imagens 2D simples ou malhas 3D movendo-se em conjunto.
- ▶ Vamos configurar, portanto, um sistema de partículas para a explosão de nosso player.
- ▶ Sob a janela Hierarchy, clique no botão [+] e selecione Effects | Particle System.
- ▶ Você verá que aparecerá uma emissão de partículas em nosso mundo de jogo, como um conjunto de bolas brilhantes flutuantes.
- ▶ Renomeie o objeto Particle System para “Particulas Explosao Player”.
- ▶ A melhor maneira de configurar um sistema de partículas é pelo seu uso, ajuste e visualização, até que atinja o padrão desejado.
- ▶ Clicando em outro objeto da cena, o sistema de partículas para de funcionar momentaneamente, até que você clique novamente na sua posição central.
- ▶ Assim, você pode testá-lo interativamente.



Roller Madness - Partículas e Efeitos Sonoros

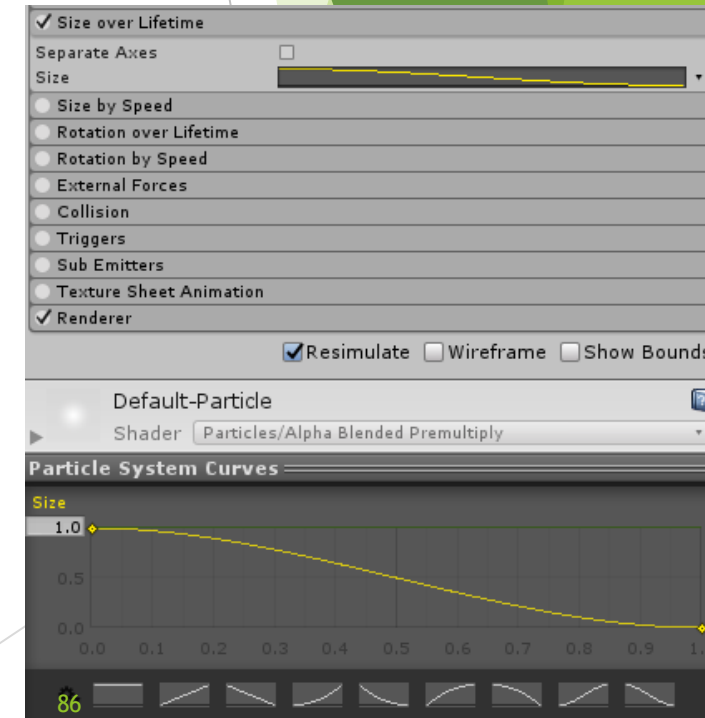
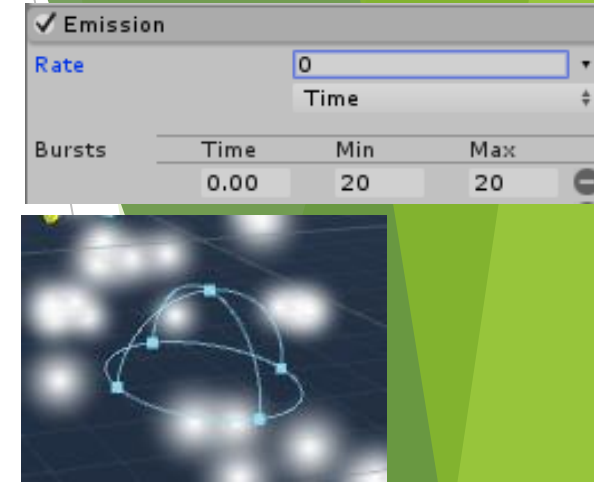
- ▶ Há, também, uma pequena janela de controle onde você pode pausar o sistema, pará-lo completamente, ajustar velocidade, tempo de reprodução e quantidade de partículas geradas.
- ▶ Observe no Inspector que o Sistema de Partículas é apenas um componente em um Game Object e que há várias abas que você pode abrir e fechar e, naturalmente, configurar suas propriedades.
- ▶ Em cada propriedade, há um menu suspenso que será exibido quando o mouse passar sobre o nome da propriedade.
- ▶ Mude a propriedade abaixo conforme definido:
 - ▶ Duration → 2
 - ▶ Looping → false
 - ▶ Start Lifetime → 1
 - ▶ Start Speed → 5
 - ▶ Start Size → 1
 - ▶ Gravity Modifier → 0.5um pouco

Você verá que as partículas sobem e caem ⁸⁵



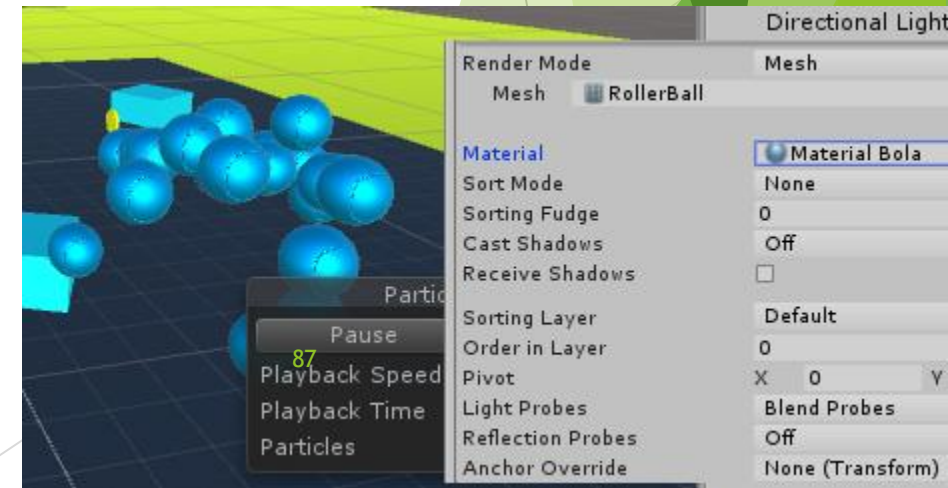
Roller Madness - Partículas e Efeitos Sonoros

- ▶ Em Emission, mude Rate para 0, clique em + e defina Burst com mínimo 20 e máximo 20. Assim, as partículas aparecerão no tempo = 0 todas de uma vez, como uma explosão.
- ▶ A próxima propriedade a alterar é Shape. Ela define o formato do conjunto de partículas. Mude de Cone para HemiSphere, para dar uma ideia mais próxima de uma explosão.
- ▶ Em seguida, selecione Size over Lifetime.
- ▶ Clique no item Size e aparecerá uma janela de configuração de curvas do sistema de partículas, com alguns itens pré-definidos na parte inferior.
- ▶ Você pode escolher a pré-definição mais à direita ou mover a curva até ela ficar semelhante à pré-definição. Ao fazer isso, notará que cada partícula individual começa maior e vai diminuindo até desaparecer.



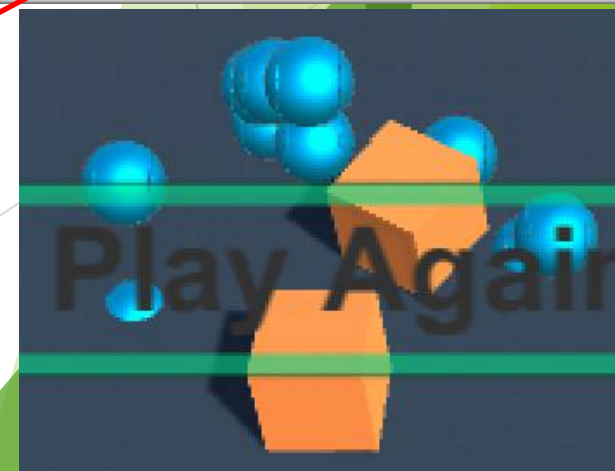
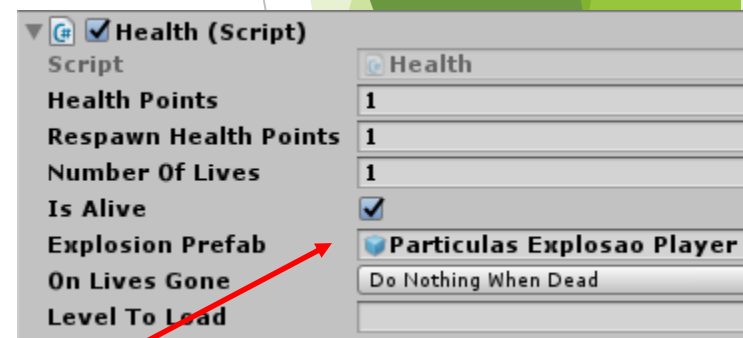
Roller Madness - Partículas e Efeitos Sonoros

- ▶ Renderer define o que será gerado para exibição. No momento, temos essas figuras brancas aparecendo.
- ▶ Atualmente, está com o valor Billboard, que gera esse grupo de imagens 2D com aparência de fumaça. Mas, para essas partículas especificamente, seria interessante gerá-las como se nossa rollerball estivesse se partindo em vários pedaços.
- ▶ Assim, vamos alterar de Billboard para Mesh (malha 3D).
- ▶ Clique no pequeno círculo ao lado da palavra Mesh e, na janela que aparecerá, selecione o item Rollerball.
- ▶ Precisamos definir o material da rollerball em explosão, pois o material está com o material default de partículas. Escolha o “Material Bola” e teremos o efeito de várias pequenas rollerball explodindo.
- ▶ Podemos continuar configurando mas está bom agora.
- ▶ Vamos adicionar um Audio Source em nossas partículas.
- ▶ Clique em [Add Component] | Audio | Audio Source.
- ▶ Na propriedade AudioClip, adicione o som PlayerExplosion.



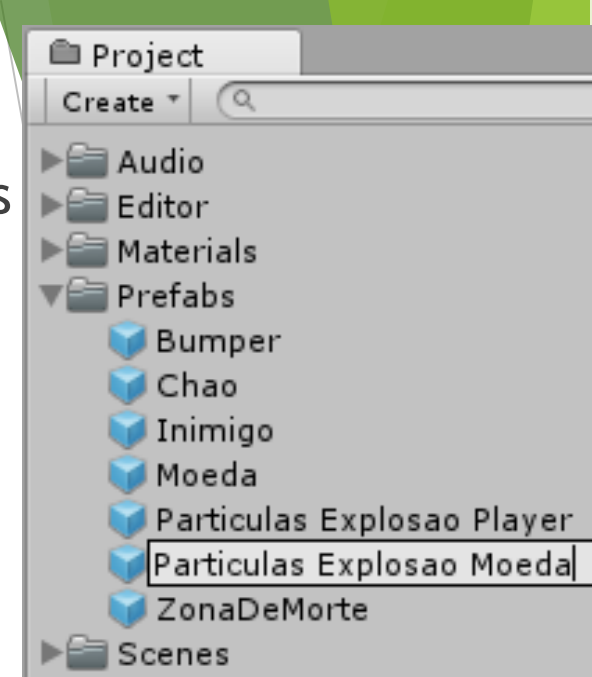
Roller Madness - Partículas e Efeitos Sonoros

- ▶ Agora, esse objeto de partículas será usado apenas quando o player morrer.
- ▶ Será importante que esse objeto seja desalocado da memória após seu uso e, para isso, vamos adicionar um script que o faça.
- ▶ Deixe selecionado o sistema de partículas e, no Inspector, clique em [Add Component] | Scripts. Selecione o script TimedObjectDestructor.
- ▶ Mude a variável Time Out para 3, de forma que o objeto destrua a si mesmo após 3 segundos.
- ▶ Agora que temos esse objeto pronto, vamos transformá-lo em um Prefab, arrastando-o da Hierarquia para a pasta prefabs da janela Project.
- ▶ Não queremos que essas partículas apareçam no jogo desde o início, apenas quando o player morrer, portanto, vamos apagá-la da Cena ou da Hierarquia, pois já temos seu modelo como um Prefab.
- ▶ Selecione a Rollerball e, no script Health, arraste o prefab “Particulas Explosao Player” para a variável “Explosion Prefab”. Quando o player morrer, o script Health disparará a geração da explosão que as partículas simulam.



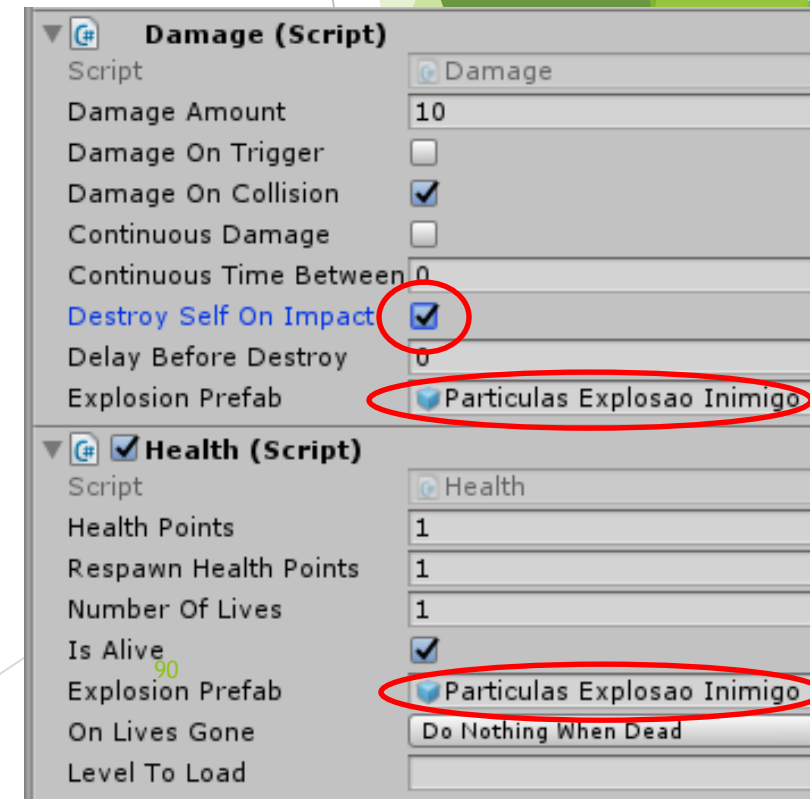
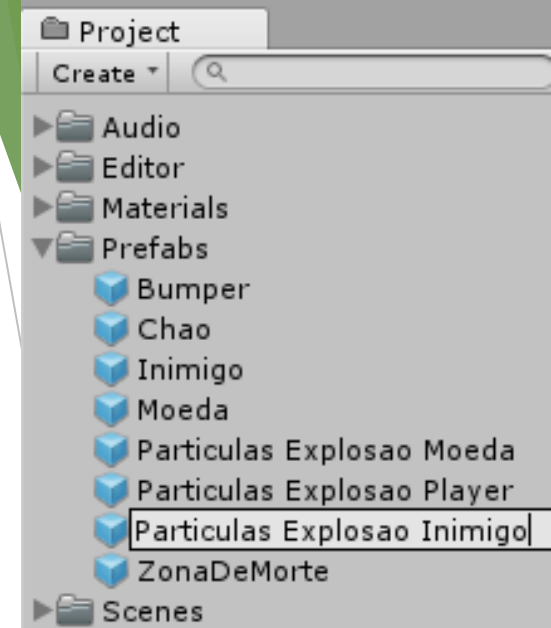
Roller Madness - Partículas e Efeitos Sonoros

- ▶ Agora que temos a explosão do player, podemos rapidamente criar sistemas de partículas para as moedas e para os inimigos.
- ▶ Assim, basicamente duplicaremos o prefab “Particulas Explosao Player” para criar outras partículas, específicas para cada situação.
- ▶ Duplique esse prefab e chame o resultado de “Particulas Explosao Moeda”.
- ▶ Você pode editar as propriedades diretamente no prefab selecionado, sem ter que fazer uma cópia dele na cena. Assim, no Inspector, altere as propriedades de Particle System abaixo:
 - ▶ Start Size → 0.2
 - ▶ Emission | Bursts → 10
 - ▶ Renderer | Material → Material Moeda
- ▶ Em Audio Source, troque AudioClip para CoinExplosion.
- ▶ Manteremos o script TimedObjectDestructor.
- ▶ Clique em uma das moedas na cena, vá ao script Treasure e, na variável Explosion Prefab, arraste o prefab que acabamos de criar (Particulas Explosao Moeda). Clique em [Apply], para que o prefab Moeda receba essas alterações e todas as demais moedas sejam atualizadas. Execute o jogo.



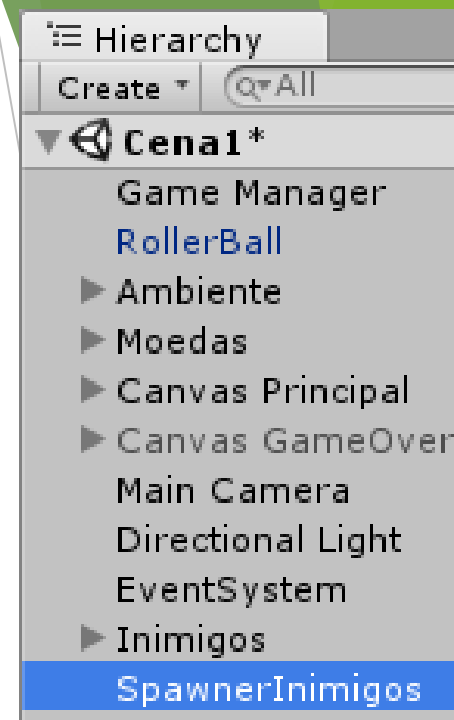
Roller Madness - Partículas e Efeitos Sonoros

- ▶ Agora já temos alguns efeitos visuais e sonoros interessantes em nosso jogo e vamos, agora, criar o sistema de partículas para a morte dos inimigos.
- ▶ Duplique o prefab “Particulas Explosao Player” e renomeie a cópia para “Particulas Explosao Inimigo”.
- ▶ No Inspector, mude a propriedade Renderer para Cube e Material para “Material Inimigo”.
- ▶ Modifique AudioSource | AudioClip para EnemyExplosion.
- ▶ No prefab Inimigo, vá ao script Damage e arraste o prefab “Particulas Explosao Inimigo” para a variável Explosion Prefab.
- ▶ Nesse script, faça Destroy Self on Impact valer true.
- ▶ No script Health, e arraste o prefab “Particulas Explosao Inimigo” para a variável Explosion Prefab.
- ▶ Execute o jogo.
- ▶ Agora, o jogo ficou um pouco mais balanceado pois, quando um inimigo colide com outro, eles se destroem.



Roller Madness - Spawners

- ▶ A última coisa que faremos para tornar nosso jogo mais interessante e eficiente é adicionar spawners para as moedas e para os inimigos.
- ▶ Dessa forma, o jogo continuamente adicionará mais moedas para serem coletadas mas, também, mais inimigos serão adicionados para aumentar o grau de desafios do jogo.
- ▶ Primeiramente, criaremos um objeto vazio para representar o spawner de inimigos. Na Hierarquia, pressione [+] | Empty Object. Mude o nome para SpawnerInimigos e faça o reset de seu transform.
- ▶ A este objeto, adicione o script Spawn Game Objects, que já foi codificado anteriormente.
- ▶ Esse script espera um prefab para ser gerado (spawned) e, portanto, associamos a variável SpawnPrefab com o objeto Inimigo da pasta Prefabs da janela Project.
- ▶ Na variável Chase Target, associamos o player, ou seja, a Rollerball para que ela seja caçada pelos inimigos gerados.
- ▶ Copie o objeto SpawnerInimigos para a pasta Prefabs de Project, de forma que ele vire um prefab.

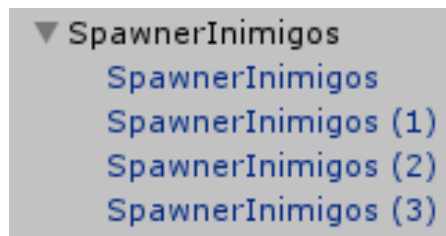


Roller Madness - Spawners

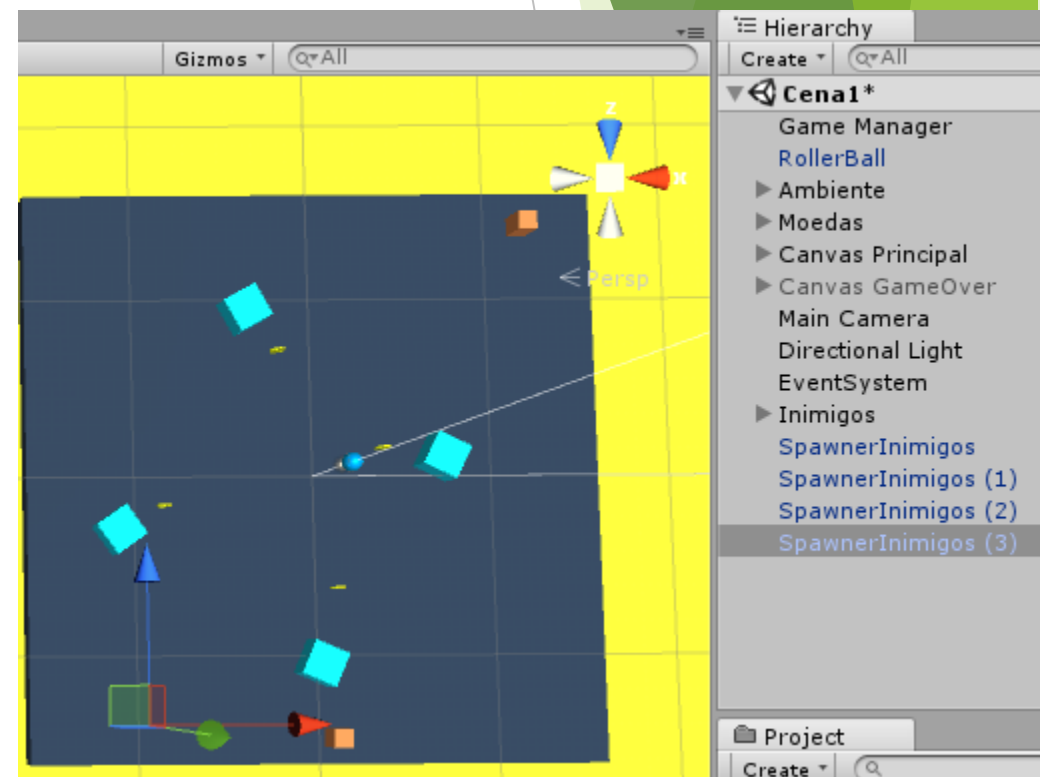
- ▶ Mude o Transform Y do objeto SpawnerInimigos da cena para 9, de forma que ele fique bem alto no jogo e, ao gerar novos inimigos, os faça cair de muito alto, o que resultará em inimigos que rebaterão bastante no jogo.
- ▶ Mude a visualização da cena para a visualização top down e mova esse objeto para o canto superior esquerdo da cena. Em seguida, faça duplicações do mesmo e as mova para os outros três cantos da cena.
- ▶ Apague os inimigos criados anteriormente e que compõe o objeto Inimigos da cena, pois apenas usaremos os que forem gerados pelos spawners.



- ▶ Copie os quatro spawners para dentro do objeto Inimigos. Mude o nome desse objeto para SpawnerInimigos.



- ▶ Salve e teste o jogo.



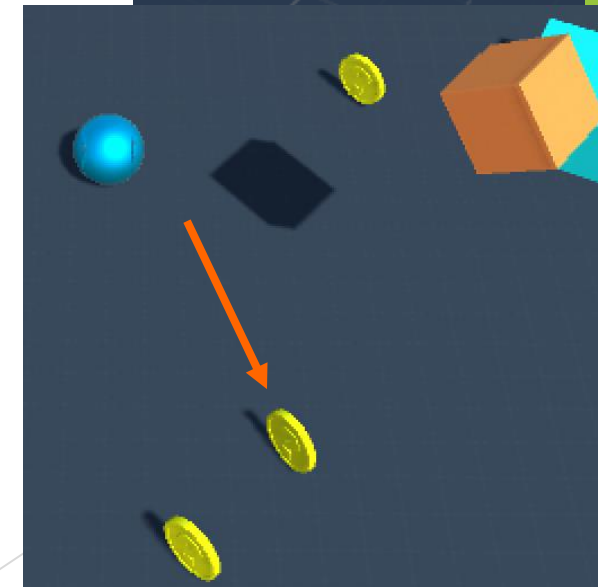
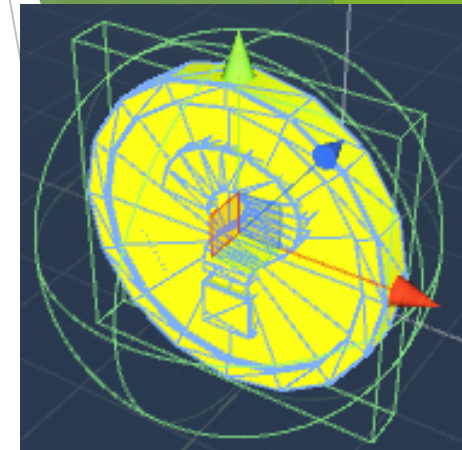
Roller Madness - Spawners

- ▶ Se você mantiver a visão top down da janela Scene enquanto joga (para isso, desative o botão [Maximize on Play] da janela Game), verá que os inimigos aparecem nos quatro cantos da área de jogo e perseguem a Rollerball.
- ▶ Vamos agora criar o spawner de moedas. Para torná-las mais interessantes, faremos com que elas interajam com Física, como os inimigos.
- ▶ Deixemos a moedas já existentes em suas posições.
- ▶ Vamos ao prefab das Moedas e o dupliquemos. Mude o nome dessa cópia para Moeda Saltitante.
- ▶ Precisamos de um Rigidbody para que as moedas saltitantes caiam.
- ▶ Portanto, observe o Rigidbody do prefab Moeda Saltitante. Mude sua propriedade Use Gravity para true.
- ▶ Arraste o prefab Moeda Saltitante para a cena e a deixe alta sobre o chão.
- ▶ Ao executar o jogo, verá que a moeda não cai. Isso é devido ao fato de que Is Kinematic ainda está ativado.
- ▶ Portanto, deixe Is Kinematic igual a false e execute o jogo. Ela cairá e passará pelo chão, desaparecendo.
- ▶ Isso ocorre porque ela tem um Sphere Collider onde Is Trigger está true.



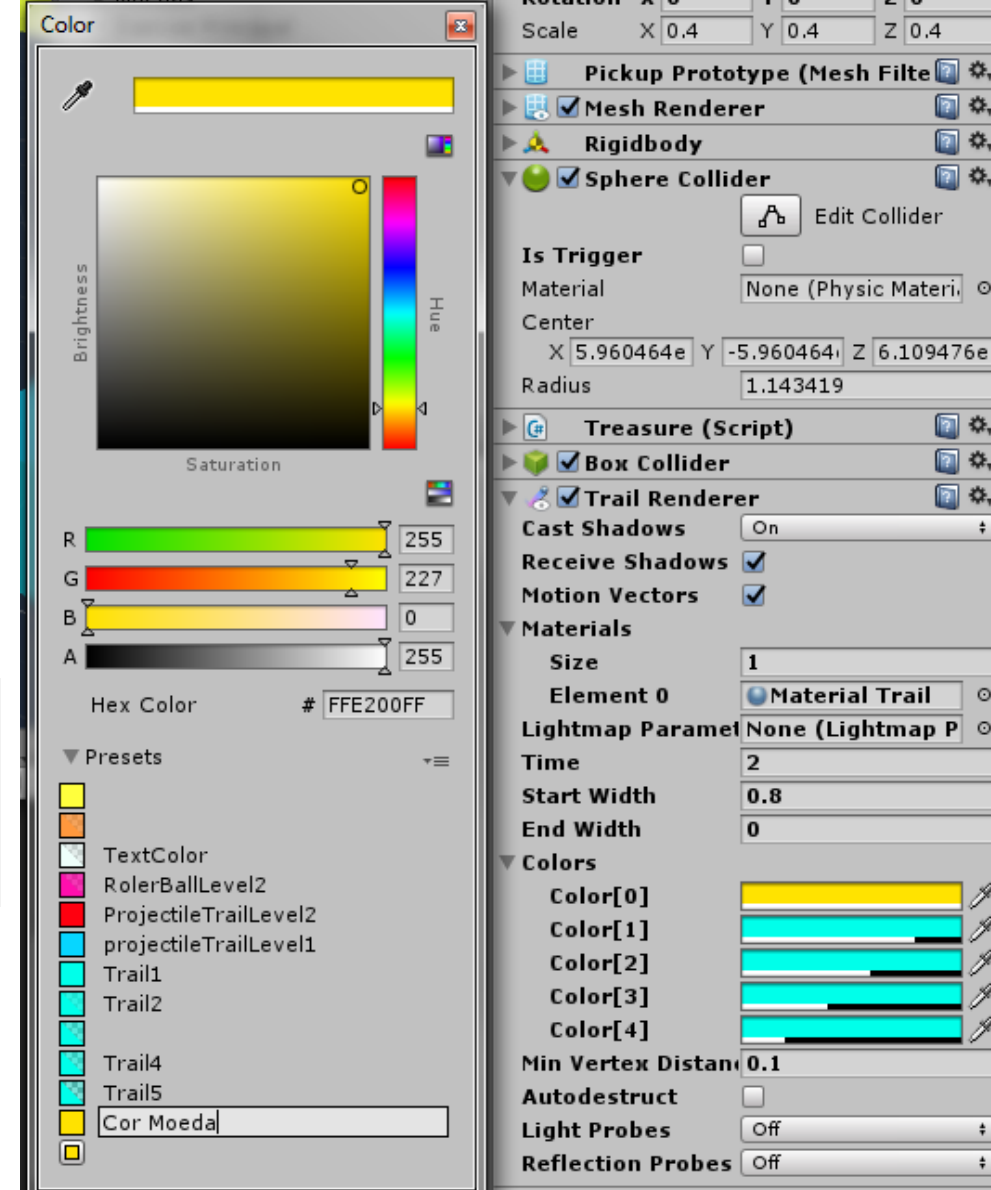
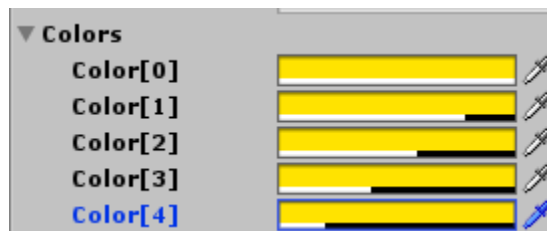
Roller Madness - Spawners

- ▶ Precisamos desse Sphere Collider true pois é com ele que ativamos o script Treasure, que conta as moedas que são coletadas.
- ▶ Portanto, para evitar que a moeda passe pelo chão, vamos adicionar um outro Collider a ela, este com Is Trigger igual a false.
- ▶ Clique em [Add Component] | Physics | Box Collider. Passaremos a ter dois colliders nesse objeto, como na figura ao lado.
- ▶ Deixe o Box Collider com Is Trigger valendo false, porque queremos que a moeda caia sobre o chão ou algum outro obstáculo da cena e salte, sem passar dentro do objeto com o qual colidiu. Dê a ela o Physic Material Bouncy para que ela fique saltitando quando tocar o chão.
- ▶ Remova o script Rotate da Moeda Saltitante e execute o jogo.
- ▶ Podemos também colocar um rastro nas moedas saltitantes que criarmos.
- ▶ Vá até o objeto Rollerball, clique no Trail Renderer de seu Inspector e copie esse componente.
- ▶ Clique na Moeda Saltitante e em alguma engrenagem do Inspector selecione Paste Component as New.



Roller Madness - Spawners

- ▶ Mude Start Width para 0.2, pois a moeda é menor que a Rollerball.
- ▶ Vamos ajustar as cores. Clique na primeira cor (ciano) e ajuste para amarelo, como vemos ao lado.
- ▶ Nomeie a nova cor como Cor Moeda.
- ▶ Em cada um das cores do Trail Renderer, selecione a cor Cor Moeda no Color Pickup e altere a parte alpha da cor (transparência).
- ▶ Terminamos o Trail Renderer.
- ▶ Como teremos várias moedas saltitantes sendo criadas, algumas poderão saltitar para fora do jogo e não mais as veremos, mas elas continuarão lá ocupando memória e tempo do processador. Para evitar isso, adicionaremos o script Timed Object Destructor à Moeda Saltitante.
- ▶ Mude TimeOut para 10 segundos e, após esse tempo, elas serão removidas da memória e deixarão de existir.



Roller Madness - Spawners

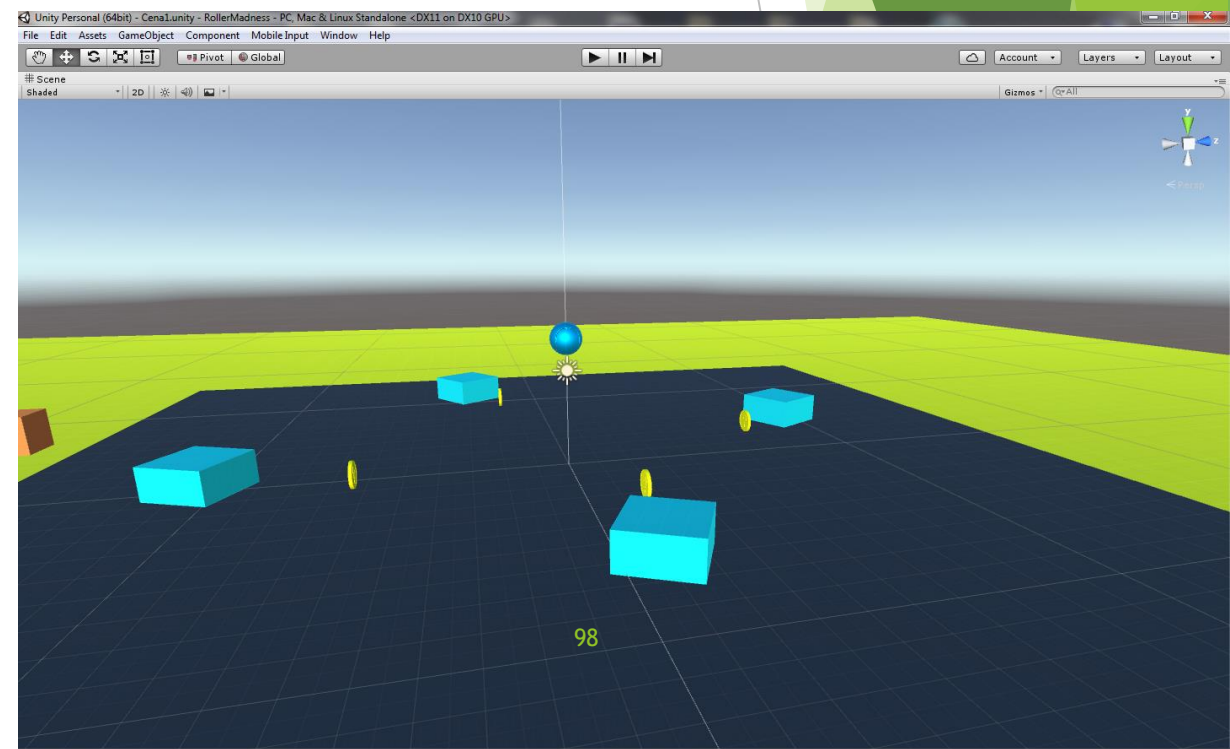
- ▶ Tendo selecionado a Moeda Saltitante, pressione o botão [Apply] no seu Inspector e, dessa forma, as alterações feitas serão aplicadas no Prefab associado a ela.
- ▶ Remova esse objeto da cena.
- ▶ Da mesma forma que fizemos com o Spawner de Inimigos, criaremos um Spawner de moedas.
- ▶ Crie um objeto vazio, chame-o de SpawnerMoedas. Faça o reset de seu transform.
- ▶ Adicione o Script Spawn Game Objects.
- ▶ Na variável Spawn Prefab, arraste o prefab Moeda Saltitante. Mude Min Seconds Between Spawning para 1 e Max Seconds Between Spawning para 5.
- ▶ A moeda não caçará nenhum objeto, portanto deixe a variável Chase Target vazia.
- ▶ Copie esse objeto SpawnerMoedas para a pasta Prefabs e faça várias cópias do novo prefab na cena, em diferentes posições, na altura 9 Y, por exemplo.
- ▶ Crie um objeto vazio chamado SpawnerMoedas para agrupar os spawners de moedas que você criou.

Roller Madness - Projeto Final

- ▶ Baseando-se no que discutimos neste estudo do jogo RollerMadness, crie uma segunda fase, que deve ser lançada quando o player atingir um certo número de coletas sem ser atingido pelos inimigos.
- ▶ Na segunda fase, coloque mais obstáculos, de diferentes formas, mude os parâmetros dos inimigos para torná-los mais letais e, caso o Player adquira as moedas necessárias para mudar de fase, mostre um Canvas de Vitória e permita reiniciar o jogo ou terminá-lo (usando o script UIButtonQuitGame).
- ▶ Esse projeto deve ser desenvolvido em dupla.

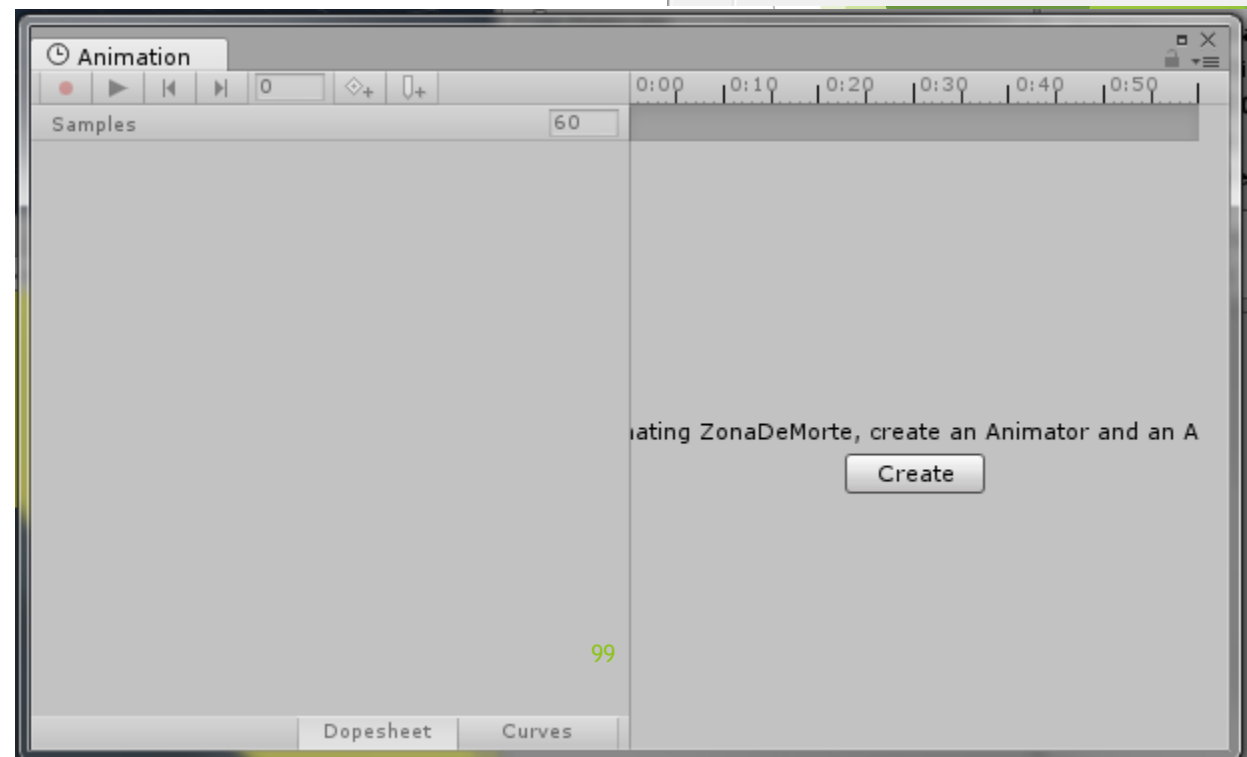
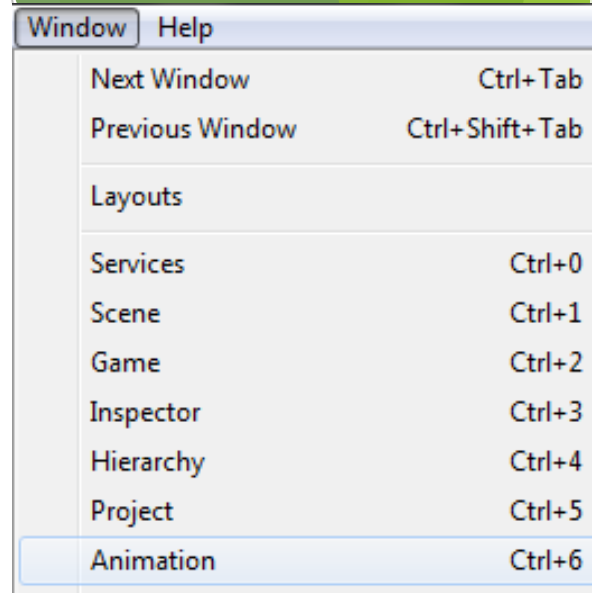
Roller Madness - Animação

- ▶ Animação é uma maneira de adicionar um polimento visual ao nosso jogo.
- ▶ Criaremos uma animação simples, explorando a Animation Timeline.
- ▶ Criaremos um clip de animação por meio de quadros chave (keyframe) no objeto ZonaDeMorte. Criaremos uma animação de luz difusa (shimmer) nesse objeto, para comunicar o fato de que é uma região perigosa para o jogador.
- ▶ Na Hierarquia, selecione o objeto “Zona de Morte” (para achá-lo, escreva a palavra “zona” no buscador ao lado do botão [+]).
- ▶ Clique em ZonaDeMorte para selecioná-lo e pressione o botão X do buscador, para mostrar toda a Hierarquia e manter selecionado ZonaDeMorte.
- ▶ Outra função de editor interessante que ainda não vimos é a fullscreen. Com o cursor sobre a janela Scene, pressione [Shift-Espaço].



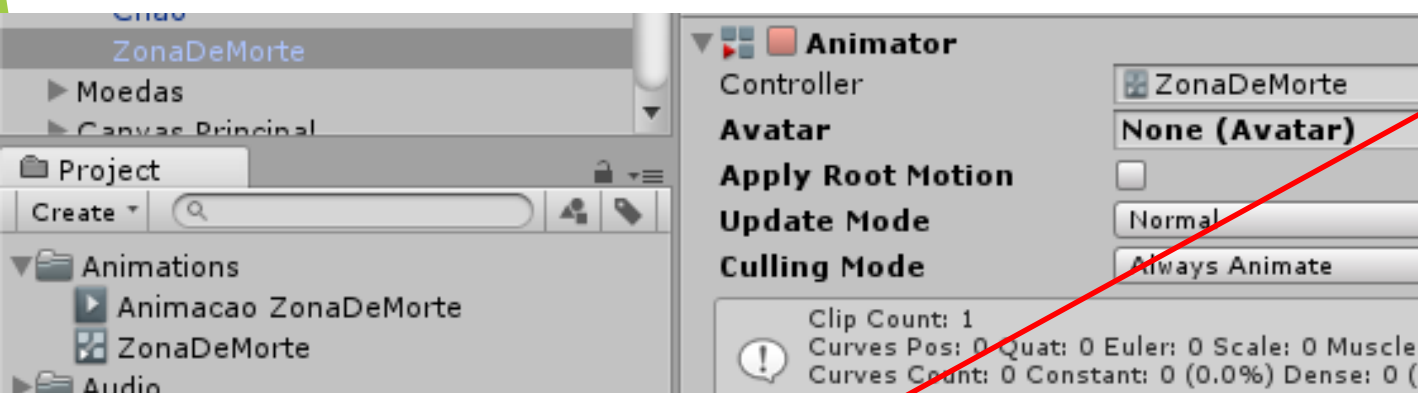
Roller Madness - Animação

- ▶ Outro [Shift-Space] volta a cena para a visualização anterior.
- ▶ Esse recurso funciona com qualquer janela do Unity.
- ▶ Se sua ZonaDeMorte estiver invisível, ative esse objeto ou seu mesh renderer, aquele que estiver desabilitado.
- ▶ Criamos animações através do Sistema de Animações.
- ▶ No menu, selecione Window | Animation. Aparecerá a janela do Editor de Animação, como vemos ao lado:
- ▶ Ela informa que, para criar uma animação, precisamos criar um Animator e um clipe de Animação.
- ▶ O sistema criará um componente Animator vinculado ao objeto ZonaDeMorte e esse Animator usa Animation Clips para criar os Quadros da animação.
- ▶ Clique no botão [Create] dessa janela.

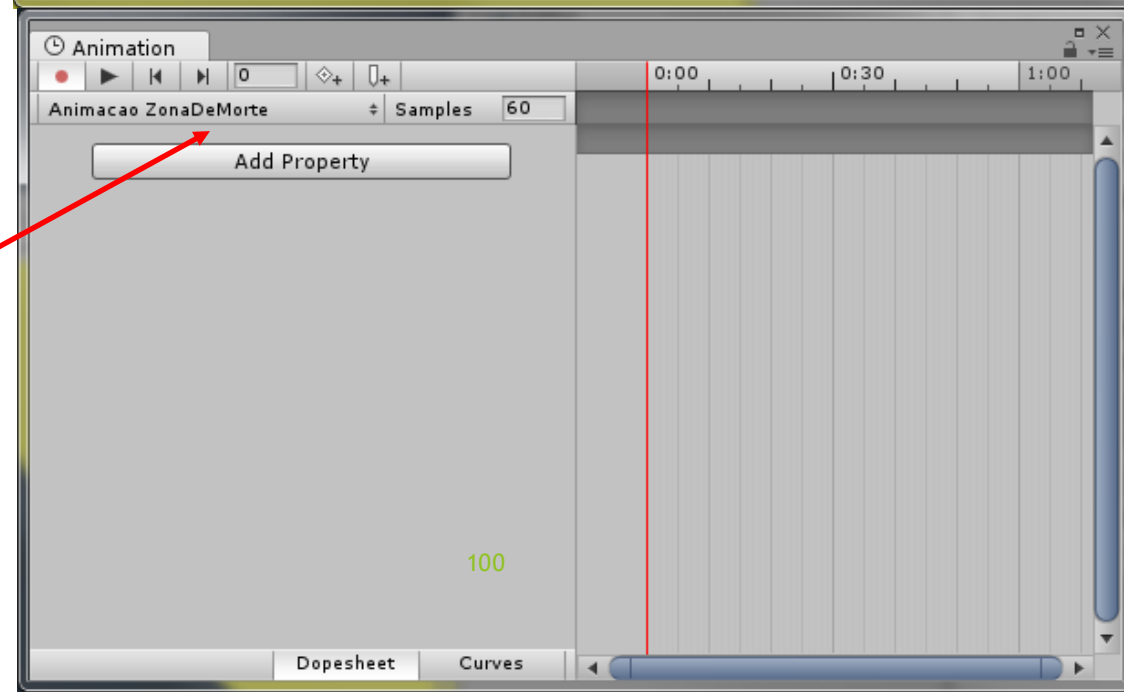
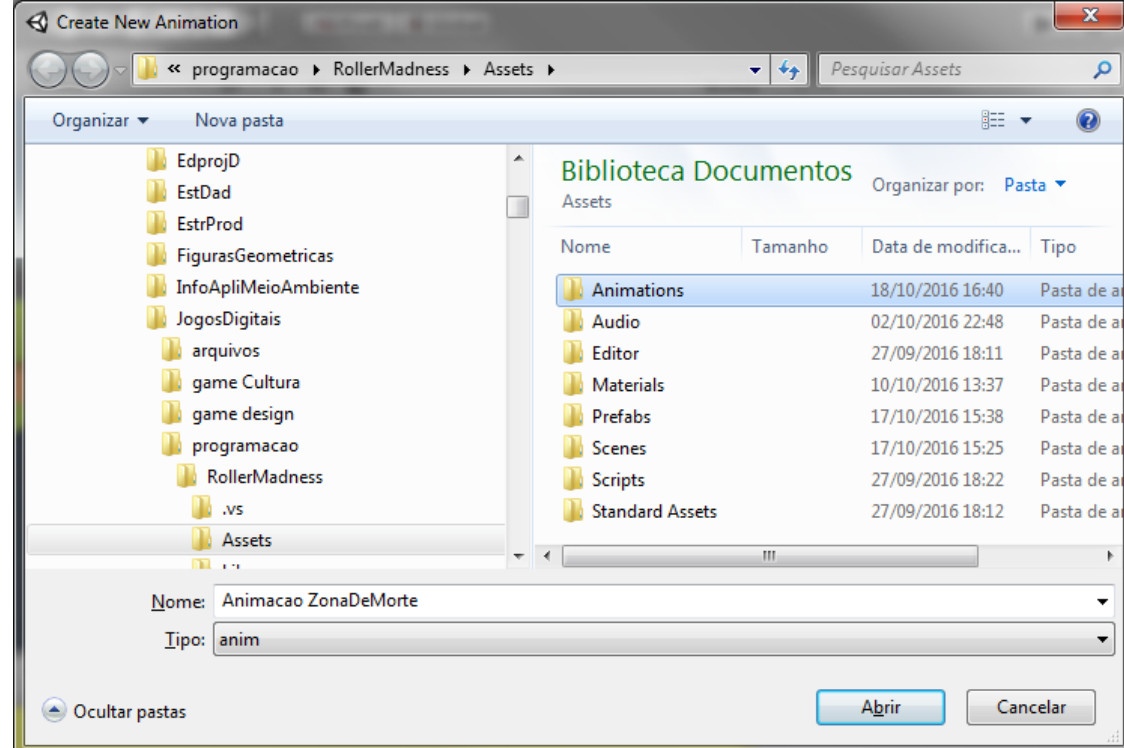


Roller Madness - Animação

- ▶ Aparecerá uma janela solicitando o nome do arquivo de animação.
- ▶ Crie uma pasta Animation nesse local, que representa o nosso projeto.
- ▶ Abra essa pasta e informe o nome do arquivo como “Animacao ZonaDeMorte”.
- ▶ Na pasta Animations, foi criado um objeto Animator e ele foi vinculado ao Inspector do game object ZonaDeMorte, que estava selecionado no momento da criação da animação.

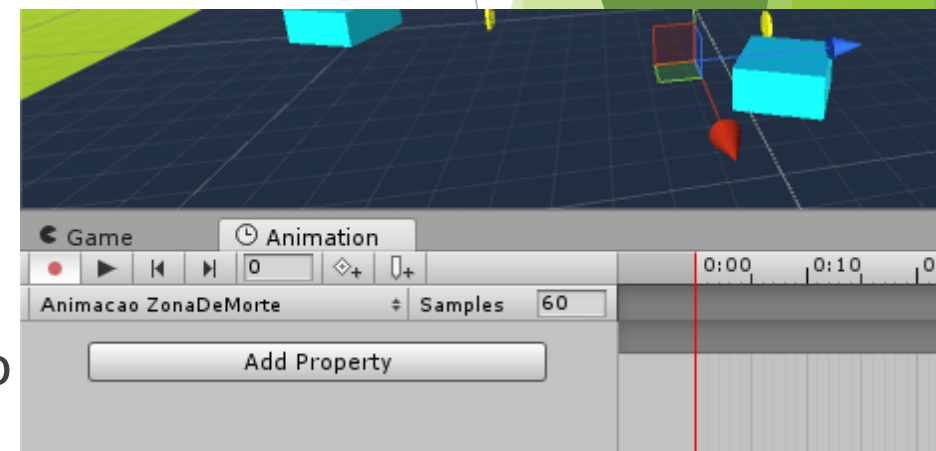
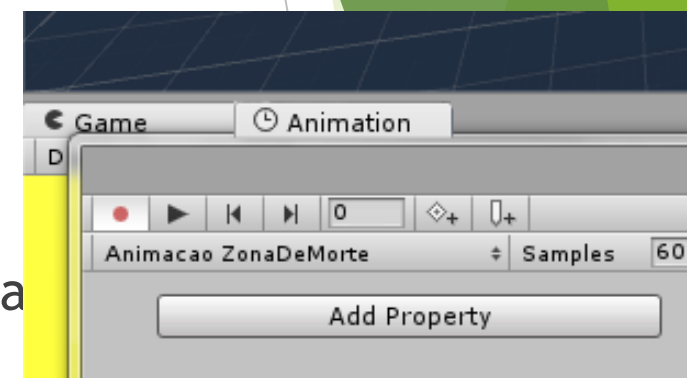
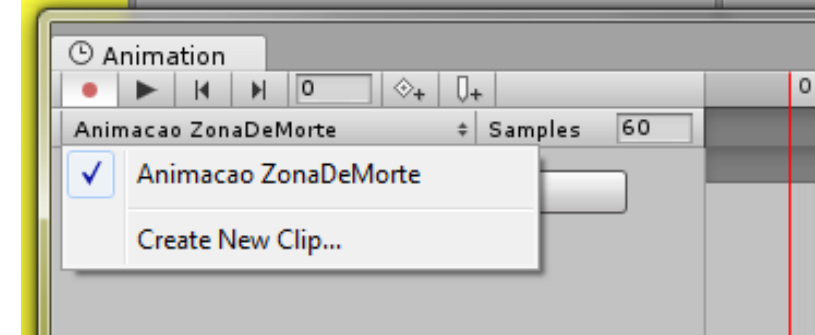


- ▶ Veja o objeto de animação no Editor de Animações.

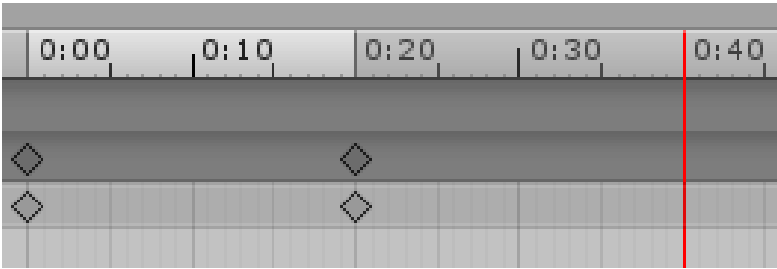


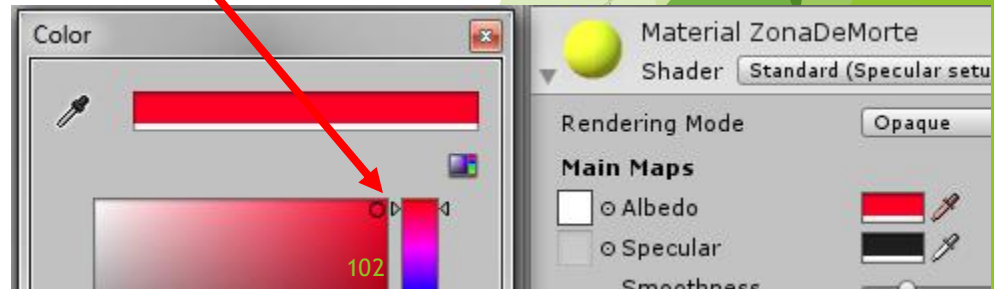
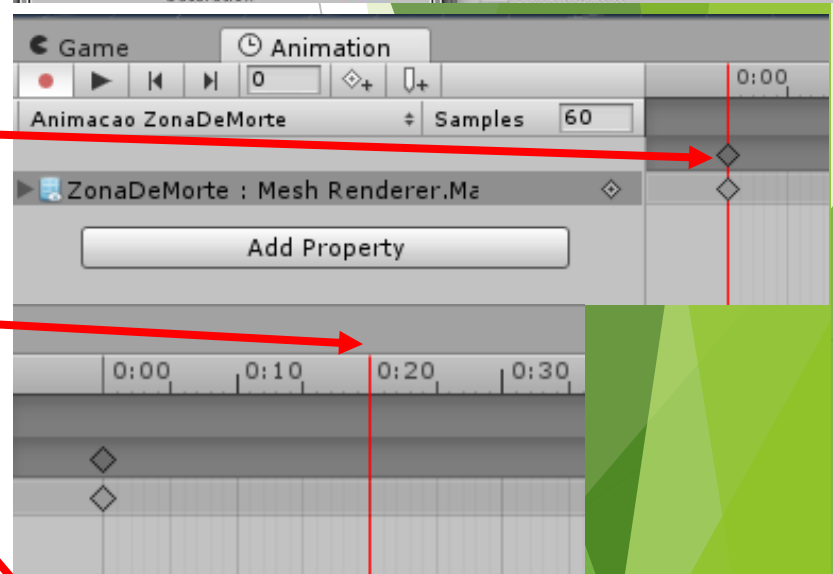
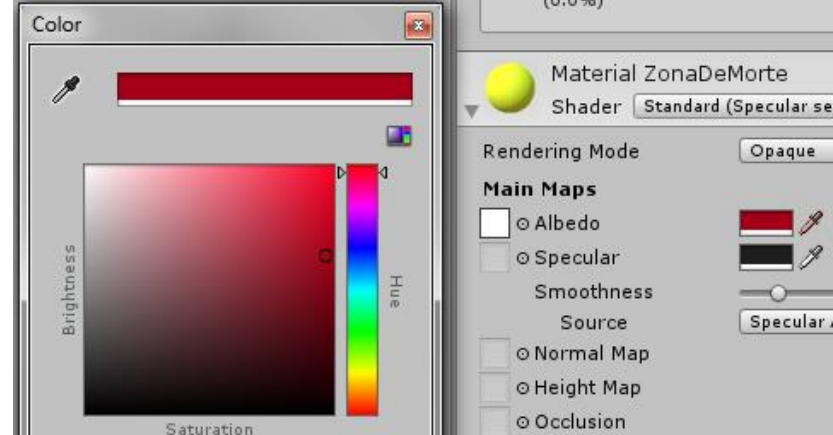
Roller Madness - Animação

- ▶ Você pode criar novos clipes de animação, na interface do Editor, vemos ao lado.
- ▶ Trabalhar com esse editor de animações seria melhor com várias monitores mas, como não os temos, vamos arrastar a janela do editor para baixo e encaixá-la ao lado da janela Game.
- ▶ O editor de animações é bastante direto.
- ▶ Temos uma timeline à direita, que começa em zero e o tempo de um em um segundo.
- ▶ Baseia-se em amostras (samples). O valor de 60 indica que serão 60 amostras por segundo.
- ▶ Note que o botão circular Vermelho (Recording) está ligado.
- ▶ Pode ser desligado mas, quando está ligado, tudo o que você fizer no editor do unity será gravado na timeline e reproduzido durante a animação.



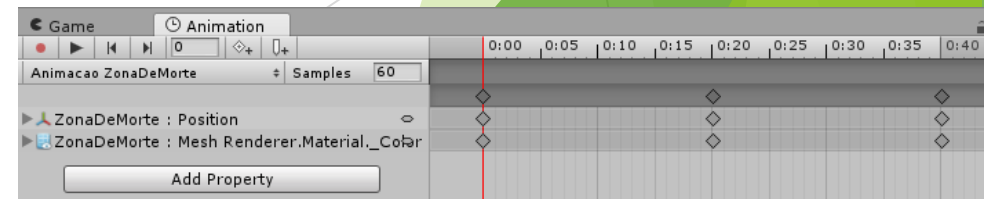
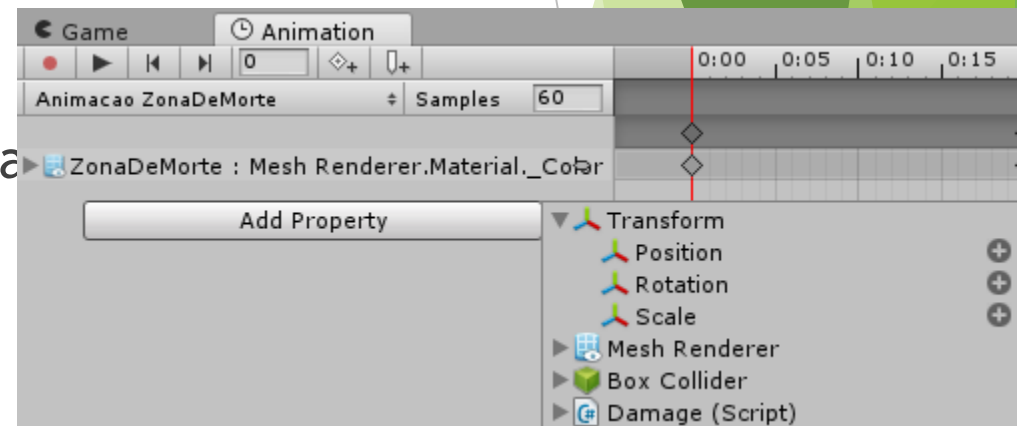
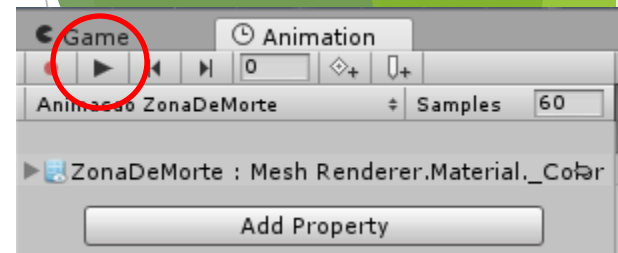
Roller Madness - Animação

- ▶ Assim, com o objeto ZonaDeMorte selecionado, acessemos a propriedade Material. No quadro de cor do Albedo, mude a cor para uma versão mais escura de vermelho (A50017FF).
 - ▶ Observem na timeline da animação que um keyframe foi marcado, pois tudo o que fizemos durante a gravação será registrado.
 - ▶ Na timeline, cliquemos na marca 0.2 para movermos adiante no tempo.
 - ▶ No Inspector do ZonaDeMorte, na propriedade Material, vamos voltar a cor para o vermelho vivo (FF0000FF) que havia antes.
 - ▶ Essa ação também foi registrada na timeline.
 - ▶ Avance agora o tempo para 0.4 na timeline.
- 
- ▶ Marque novamente a cor do 1o keyframe no Materials do Inspector.



Roller Madness - Animação

- ▶ Se você clicar na seta ao lado do botão de gravação, poderá ver o resultado da animação, quando ZonaDeMorte ficará piscando entre tonalidades de vermelho.
- ▶ Em geral, em animações, o sistema de animação faz uma interpolação entre o valor estabelecido de uma propriedade num determinado momento do tempo e o valor estabelecido dessa propriedade no tempo seguinte. Em outras palavras, no caso da mudança de cor, ela não ocorre apenas no tempo 0.0 0.2, 0.4 e 0.0 mas, ao contrário o sistema calcula valores intermediários da propriedade nos intervalos de tempo entre as medidas (key frames), de forma que suaviza a transição de uma cor para outra.
- ▶ Trabalhamos com a animação de uma única propriedade mas podemos incluir outras propriedades e a
- ▶ Com o botão [Record] ainda selecionado na janela Animation, clique em [Add Property] e selecione Transform | Position.
- ▶ Clique no tempo 0:00, mude o valor de Y de -2 para -2.1.
- ▶ Clique no tempo 0.2 e mude o valor de Y para -2. Clique no tempo 0.4 e mude Y para -2.1.
- ▶ Desligue o botão [Record], pois temos nossa animação pronta.



Roller Madness - Animação

- Observe que o objeto ZonaDeMorte tem um box collider e estamos movendo-o com animation, provavelmente deveríamos adicionar um Rigidbody nele para propósitos de otimização de desempenho do jogo.
- Adicione um Rigidbody em ZonaDeMorte.
- Como o movemos por meio de Animation, certamente não desejamos que a gravidade atue nesse objeto e, portanto, desligamos o checkbox Use Gravity.
- Por estamos movendo-o com animação, ligamos Is Kinematic,
- Esses atos aumentarão o desempenho do jogo e diminuirão a carga na CPU e na GPU, tornando seu uso mais eficiente.

